

Debugging Flutter

Guía completa de debugging y optimización en Flutter: mentalidad anti-dependencia de la IA, debugger de VS Code (launch.json, breakpoints avanzados, Debug Console), DevTools (Inspector, Performance, Memory, CPU Profiler, Network, Logging, App Size), fundamentos de rendimiento, optimización de rebuilds y rendering, detección de memory leaks, debugging de código asíncrono, playbook sistemático y ejecución aislada de cada capa de Clean Architecture.

ENFOQUE

Debugging y profiling

STACK

**Flutter + Dart +
DevTools**

NIVEL

Básico a Avanzado

TIEMPO ESTIMADO

35-45 horas

VS Code

DevTools

Rendimiento

Memory

Network

Debugging

Clean Architecture

Contenido

31 capítulos · Debugging con VS Code + DevTools + Rendimiento + Maestría (Flutter)

PARTE 1 · DEBUGGING CON VS CODE

01	01 — Fundamentos del Debugging en VS Code
02	02 — Configuración de `launch.json` para Flutter
03	03 — Breakpoints Avanzados
04	04 — Inspección de Datos y Debug Console
05	05 — Multi-target y Debugging Remoto
06	06 — Cheatsheet: Debugging en VS Code
07	07 — Prácticas y Ejercicios de Debugging en VS Code

PARTE 2 · FLUTTER DEVTOOLS

08	08 — Fundamentos de Flutter DevTools
09	09 — Flutter Inspector y Debug de Layout
10	10 — La Performance View: FPS, Frames y Jank
11	11 — CPU Profiler: Funciones Costosas
12	12 — Memory Profiler: Leaks y Uso de Memoria
13	13 — Network View: Requests HTTP y WebSocket
14	14 — Debugger View: Control de Ejecución
15	15 — Logging View y Estrategias de Logging
16	16 — App Size: Analizando el Tamaño del Bundle
17	17 — Cheatsheet de DevTools
18	18 — Prácticas y Ejercicios con DevTools

PARTE 3 · RENDIMIENTO Y OPTIMIZACIÓN

19	19 — Fundamentos de Rendimiento
20	20 — Optimizando Rebuilds Innecesarios
21	21 — Detección y Prevención de Memory Leaks
22	22 — Rendering Complejo: Listas, Slivers e Imágenes
23	23 — Cheatsheet de Optimización de Rendimiento
24	24 — Prácticas de Optimización de Rendimiento

PARTE 4 · DEBUGGING AVANZADO

25	25 — Debugging Asíncrono en Dart/Flutter
26	26 — Workflow de Debugging por Tipo de Bug

PARTE 5 · MAESTRÍA: MENTALIDAD Y DEBUGGING POR CAPA

27	27 — Mentalidad de Debugging: Piensa como un Debugger, no como un Prompt
28	28 — Debugging Programático: Herramientas de Debug desde el Código
29	29 — Playbook de Debugging Sistemático
31	31 — Debugging por Capa: Ejecución Aislada de cada Capa con el Debugger

PARTE 6 · REFERENCIA RÁPIDA

30	30 — Referencia Rápida: Todo el Debugging en una Página
----	---

01 — Fundamentos del Debugging en VS Code

Por qué el debugging es una habilidad clave en Flutter, cómo funcionan los modos de compilación y qué herramientas te ofrece VS Code.

0. Mentalidad del debugger

Antes de aprender herramientas, establece la mentalidad. Lee [27-mentalidad-debugging.md](#) para entender la filosofía: "No adivines, observa".

Los 3 principios clave:

1. **No supongas, verifica** — Usa breakpoints y datos, no intuición
2. **Reduce el espacio de búsqueda** — Binary search debugging (a la mitad en cada paso)
3. **Reproduce antes de arreglar** — Si no puedes reproducir, no puedes arreglar

Y la regla anti-dependencia-AI:

"¿Podrías explicarle a otro developer exactamente qué está pasando?" Si la respuesta es NO, no entiendes el bug todavía. Usa el debugger para entenderlo antes de pedir ayuda.

1. ¿Qué es el debugging?

Debugging es el proceso de **encontrar y corregir errores** en tu código. Pero no se trata solo de cazar bugs: es entender el comportamiento de tu aplicación en tiempo real, inspeccionar el estado del programa en cualquier momento y razonar sobre por qué algo no funciona como esperabas.

El debugger es tu lupa para ver dentro de la máquina. Sin él, estás adivinando. Con él, estás **observando**.

2. ¿Por qué el debugging en Flutter es especial?

Flutter tiene particularidades que lo hacen diferente de otros frameworks:

1. **Build methods**: se ejecutan muchas veces, no sabes cuándo
2. **State management (BLoC/Cubit)**: el estado vive en objetos que se crean/destruyen

3. **Hot Reload**: cambias código sin reiniciar, pero el estado persiste
4. **Two threads**: UI thread (Dart) y Raster thread (GPU)
5. **Widgets son inmutables**: se recrean en cada rebuild

Esto significa que necesitas herramientas específicas para entender qué está pasando.

3. Modos de compilación en Flutter

Flutter tiene 3 modos de compilación, y cada uno impacta el debugging:

3.1 Debug Mode

```
flutter run
```

- **Para qué**: desarrollo diario
- **Optimizaciones**: NINGUNA (todo es lento)
- **Assertions**: HABILITADOS (validan constraints e invariantes)
- **Hot Reload**: SÍ funciona
- **Debugging**: COMPLETO (breakpoints, stepping, inspección)
- **Rendimiento**: NO REPRESENTATIVO (no medir performance aquí)

3.2 Profile Mode

```
flutter run --profile
```

- **Para qué**: medir performance real
- **Optimizaciones**: MAYORÍA habilitadas
- **Assertions**: DESHABILITADOS
- **Hot Reload**: NO funciona (Hot Restart sí)
- **Debugging**: PARCIAL (breakpoints sí, pero más lento)
- **Rendimiento**: REPRESENTATIVO (medir aquí)

3.3 Release Mode

```
flutter run --release  
flutter build apk
```

- **Para qué**: producción
- **Optimizaciones**: TODAS habilitadas
- **Assertions**: DESHABILITADOS
- **Hot Reload**: NO funciona

- **Debugging:** NO DISPONIBLE
- **Rendimiento:** ÓPTIMO

Regla de oro: Para debugging usa `debug`. Para medir performance usa `profile`. Nunca midas performance en `debug`.

4. Interfaz del debugger de VS Code

Cuando presionas F5 y empieza una sesión de debugging, VS Code muestra 5 componentes principales:

1. Debug Toolbar (barra flotante) [▶ Continue] [⏮ Step Over] [⏴ Step Into] [⏵ Step Out]	
2. Run and Debug View (sidebar) - CALL STACK - VARIABLES - WATCH - BREAKPOINTS	Editor de código (con breakpoints marcados)
3. Debug Console (panel inferior) > Output del debugger, expresiones evaluadas	
4. Status Bar (barra de estado - color naranja) [Debug: flutter (debug)] ← indica sesión activa	

4.1 Componentes detallados

Debug Toolbar

Botones flotantes para controlar la sesión:

Botón	Atajo	Qué hace
Continue / Pause	<code>F5</code>	Reanuda hasta el próximo breakpoint, o pausa
Step Over	<code>F10</code>	Ejecuta la siguiente línea sin entrar en funciones
Step Into	<code>F11</code>	Entra en la siguiente función
Step Out	<code>Shift+F11</code>	Sale de la función actual
Restart	<code>Ctrl+Shift+F5</code>	Reinicia la sesión de debugging
Stop	<code>Shift+F5</code>	Detiene la sesión

Run and Debug View (Sidebar)

Panel izquierdo con 4 secciones:

- **CALL STACK:** Muestra la pila de llamadas. Cuando hay multi-target, cada sesión es un top-level element.
- **VARIABLES:** Variables locales y del scope actual. Se actualizan con el stack frame seleccionado.
- **WATCH:** Expresiones que defines para monitorear. Se evalúan en cada pausa.
- **BREAKPOINTS:** Lista de todos los breakpoints. Puedes habilitar/deshabilitar/eliminar.

Debug Console

Panel inferior donde:

- Se muestra stdout/stderr del debugger
- Puedes evaluar expresiones Dart en tiempo real
- Funciona como REPL (Read-Eval-Print Loop)
- Soporta autocompletado y sintaxis

Status Bar

Cuando hay una sesión activa, la barra de estado cambia a **color naranja** (o el color accent de tu tema). Muestra el nombre de la configuración de debug activa.

5. Flujo completo de debugging

1. Abrir proyecto en VS Code
 - └─ Asegurarse de que la extensión Flutter está instalada
2. Configurar launch.json (si es necesario)
 - └─ Ver archivo 02-configuracion-launch-json.md
3. Poner breakpoints
 - └─ Click en el gutter (margen izquierdo) o F9
4. Iniciar sesión
 - └─ F5 o Run > Start Debugging
5. Seleccionar dispositivo
 - └─ Emulador, simulador o dispositivo físico
6. Ejecutar la acción que activa el bug
 - └─ La app se pausa en el breakpoint
7. Inspeccionar
 - └─ Variables, Watch, Call Stack, Debug Console
8. Navegar por el código
 - └─ Step Into, Step Over, Step Out
9. Encontrar el problema
 - └─ Modificar código si es necesario
10. Continuar ejecución
 - └─ F5 para continuar, Shift+F5 para detener

6. Ejemplo: primer breakpoint

Imagina que tienes un BLoC de login:


```
// lib/presentation/bloc/login/login_bloc.dart
class LoginBloc extends Bloc<LoginEvent, LoginState> {
  final LoginUseCase loginUseCase;

  LoginBloc({required this.loginUseCase}) : super(LoginInitial()) {
    on<LoginSubmitted>(_onLoginSubmitted);
  }

  Future<void> _onLoginSubmitted(
    LoginSubmitted event,
    Emitter<LoginState> emit,
  ) async {
    emit(LoginLoading()); // <-- PON AQUÍ UN BREAKPOINT

    final result = await loginUseCase(
      LoginParams(
        email: event.email,
        password: event.password,
      ),
    );

    result.fold(
      (failure) => emit(LoginError(failure.message)), // <-- Y AQUÍ
      (user) => emit(LoginSuccess(user)), // <-- Y AQUÍ
    );
  }
}
```

Pasos:

1. Haz click en el gutter de la línea del `emit(LoginLoading())` (aparece un círculo rojo)
2. Presiona F5
3. La app compila y se abre en el emulador
4. Escribe email y password, presiona "Login"
5. **La app se pausa** en esa línea
6. En el panel VARIABLES verás `event.email` y `event.password`
7. Presiona F10 para avanzar línea por línea
8. Observa cómo el estado cambia de `LoginInitial` a `LoginLoading`

7. Ejemplo: Debug Console

Durante una sesión, puedes abrir la Debug Console (`Ctrl+Shift+Y`) y escribir expresiones:

```
// En la Debug Console, puedes evaluar:
> event.email
"usuario@ejemplo.com"

> state.runtimeType
"LoginLoading"

> await loginUseCase(LoginParams(email: "test@test.com", password: "123"))
// Esto ejecuta el UseCase en tiempo real y te muestra el resultado
```

Esto es extremadamente útil para:

- Verificar que un objeto tiene los valores esperados
- Ejecutar código sin modificar el archivo
- Probar comportamientos "qué pasaría si..."

8. Debugging con Hot Reload vs Hot Restart

8.1 Hot Reload (`r` en la terminal o Debug Console)

- **Qué hace:** Reinyecta código Dart compilado sin reiniciar la app
- **Estado:** SE MANTIENE (BLoC, variables globales, etc.)
- **Breakpoints:** SE MANTIENEN
- **Uso:** Cambios visuales rápidos (colores, texto, layout)

8.2 Hot Restart (`R` en la terminal o `Ctrl+Shift+F5`)

- **Qué hace:** Reinicia completamente la app
- **Estado:** SE PIERDE todo
- **Breakpoints:** SE LIMPIAN (hay que volver a ponerlos)
- **Uso:** Cambios en initializers, `main()` , rutas, `initState`

Consejo: Si tu bug depende del estado inicial de la app, usa Hot Restart. Si es un cambio visual, usa Hot Reload.

Nota sobre atajos: `Ctrl+Shift+F5` en VS Code ejecuta el comando **Debug: Restart** (que en Flutter equivale al hot restart). El **hot reload** no tiene atajo por defecto: se dispara con `r` en la terminal o Debug Console, o con el botón **Hot Reload** del Debug Toolbar.

Resumen

Concepto	Descripción
Debug Mode	Compilación sin optimizaciones, debugging completo
Profile Mode	Compilación optimizada, para medir performance
Release Mode	Producción, sin debugging
F5	Iniciar/continuar debugging
F9	Toggle breakpoint

Concepto	Descripción
F10	Step over
F11	Step into
Shift+F11	Step out
Debug Console	REPL para evaluar expresiones
Hot Reload	Cambio sin reiniciar, estado se mantiene
Hot Restart	Reinicio completo, estado se pierde

Referencias

- [Flutter | Debugging tools](#) — Debugging desde código y herramientas de Flutter
- [Flutter | VS Code](#) — Guía oficial de VS Code para Flutter
- [Flutter | Hot reload](#) — Qué se puede y no se puede hot reload
- [Flutter | Build modes](#) — Debug, Profile y Release en detalle
- [VS Code | Debugging](#) — Conceptos generales del debugger
- [27-mentalidad-debugging.md](#) — Filosofía de debugging: piensa como un debugger
- [28-debugging-programatico-flutter.md](#) — Herramientas de debug desde código

 **Siguiente:** [02-configuracion-launch-json.md](#) — Cómo configurar `launch.json` para lanzar y adjuntar sesiones de depuración

02 — Configuración de `launch.json` para Flutter

Cómo lanzar, adjuntar y combinar sesiones de depuración de Flutter desde VS Code con `launch.json`.

1. ¿Qué es `launch.json`?

`launch.json` es el archivo de configuración que le dice a VS Code **cómo ejecutar y debuggear** tu aplicación. Vive en `.vscode/launch.json` y define:

- Qué comando ejecutar (`flutter run` , `flutter run --profile` , etc.)
- Qué plataforma usar (Android, iOS, Web, Desktop)
- Qué argumentos pasar a `flutter run` y al programa
- A qué proceso adjuntarse (attach)

2. Estructura básica

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Flutter (Debug)",
      "type": "dart",
      "request": "launch",
      "program": "lib/main.dart"
    }
  ],
  "compounds": [
    {
      "name": "App + Worker",
      "configurations": ["Flutter (Debug)", "Worker"]
    }
  ]
}
```

- `configurations` : lista de configuraciones individuales
- `compounds` : combinaciones de configuraciones que se lanzan juntas

3. Configuración mínima para Flutter

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Flutter (Debug)",
      "type": "dart",
      "request": "launch",
      "program": "lib/main.dart"
    }
  ]
}
```

Con esto, presionando F5 se ejecuta tu app en debug mode. Para crear el archivo desde cero: panel **Run and Debug** → **create a launch.json file**.

4. Propiedades importantes

4.1 name

Nombre que aparece en el dropdown de debugging. Usa nombres descriptivos.

4.2 type

Siempre `"dart"` para Flutter (usa la extensión Dart-Code).

4.3 request

- `"launch"`: lanza una nueva sesión (`flutter run`)
- `"attach"`: se adjunta a un proceso ya en ejecución

4.4 program

Punto de entrada. Por defecto `"lib/main.dart"`.

4.5 flutterMode

- `"debug"` (default)
- `"profile"`
- `"release"` (sin debugging; útil solo para verificar builds)

4.6 args

Argumentos que recibe el **programa** que se ejecuta:

```
"args": ["--my-program-flag"]
```

En tests, `args` son argumentos del test runner (`--name`, `--plain-name`, etc.).

4.7 `toolArgs`

Argumentos que se pasan a la herramienta `flutter run`:

```
"toolArgs": ["-d", "emulator-5554", "--verbose"]
```

4.8 `dartDefine`

Arreglo de pares `key=value` que se convierten en `--dart-define` para `flutter run`:

```
"dartDefine": ["SUPABASE_URL=http://localhost:54321", "ENV=dev"]
```

Equivalente en `toolArgs`: `"toolArgs": ["--dart-define=ENV=dev"]`. Prefiere `dartDefine` por legibilidad.

4.9 `deviceId`

Fuerza un dispositivo concreto (sobrescribe el selector de F5):

```
"deviceId": "emulator-5554"
```

4.10 `env`

Variables de entorno del proceso:

```
"env": {  
  "SUPABASE_URL": "http://localhost:54321"  
}
```

5. Configuración multi-plataforma

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Flutter (Debug)",
      "type": "dart",
      "request": "launch",
      "program": "lib/main.dart"
    },
    {
      "name": "Flutter (Profile)",
      "type": "dart",
      "request": "launch",
      "program": "lib/main.dart",
      "flutterMode": "profile"
    },
    {
      "name": "Android (Debug)",
      "type": "dart",
      "request": "launch",
      "program": "lib/main.dart",
      "flutterMode": "debug",
      "toolArgs": ["-d", "android"]
    },
    {
      "name": "iOS (Debug)",
      "type": "dart",
      "request": "launch",
      "program": "lib/main.dart",
      "flutterMode": "debug",
      "toolArgs": ["-d", "ios"]
    },
    {
      "name": "Chrome (Debug)",
      "type": "dart",
      "request": "launch",
      "program": "lib/main.dart",
      "flutterMode": "debug",
      "toolArgs": ["-d", "chrome"]
    },
    {
      "name": "macOS (Debug)",
      "type": "dart",
      "request": "launch",
      "program": "lib/main.dart",
      "flutterMode": "debug",
      "toolArgs": ["-d", "macos"]
    },
    {
      "name": "Linux (Debug)",
      "type": "dart",
      "request": "launch",
      "program": "lib/main.dart",
      "flutterMode": "debug",
      "toolArgs": ["-d", "linux"]
    },
    {
      "name": "Windows (Debug)",
      "type": "dart",
      "request": "launch",
```

```
    "program": "lib/main.dart",
    "flutterMode": "debug",
    "toolArgs": ["-d", "windows"]
  }
]
}
```

6. Configuraciones con Supabase

Para apps con Supabase es útil tener configuraciones por ambiente:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "App (Dev - Local Supabase)",
      "type": "dart",
      "request": "launch",
      "program": "lib/main_development.dart",
      "dartDefine": ["SUPABASE_URL=http://localhost:54321"]
    },
    {
      "name": "App (Staging)",
      "type": "dart",
      "request": "launch",
      "program": "lib/main_staging.dart",
      "dartDefine": ["SUPABASE_URL=https://staging.supabase.co"]
    },
    {
      "name": "App (Production)",
      "type": "dart",
      "request": "launch",
      "program": "lib/main_production.dart",
      "dartDefine": ["SUPABASE_URL=https://prod.supabase.co"]
    }
  ]
}
```

Seguridad: nunca pongas claves secretas (p. ej. la `anon key` de producción) directamente en `launch.json`. Úsalas vía `String.fromEnvironment` desde un `.env` o desde las variables de entorno del sistema.

7. Configuración con Flutter Flavors

Si usas flavors (development, staging, production):


```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Development",
      "type": "dart",
      "request": "launch",
      "program": "lib/main.dart",
      "toolArgs": ["--flavor", "development", "-t", "lib/main_development.dart"]
    },
    {
      "name": "Production",
      "type": "dart",
      "request": "launch",
      "program": "lib/main.dart",
      "toolArgs": ["--flavor", "production", "-t", "lib/main_production.dart"],
      "flutterMode": "profile"
    }
  ]
}
```

`--flavor` y `-t` son banderas de `flutter run`, por eso van en `toolArgs`.

8. Attach a un proceso en ejecución

Hay dos formas de adjuntarte a una app que ya corre.

8.1 Desde VS Code

Configuración `attach`: la extensión **descubre automáticamente** la app en ejecución (no hace falta una URI del VM service):

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Attach to running app",
      "type": "dart",
      "request": "attach",
      "program": "lib/main.dart"
    }
  ]
}
```

Si hay varios procesos y quieres forzar uno, usa `deviceId` (o el selector de la barra de estado al iniciar el attach).

8.2 Desde la terminal

```
# Terminal 1: lanza la app
flutter run

# Terminal 2: se adjunta a la app en ejecución
flutter attach
```

`flutter attach` descubre la app por sí solo. Opciones útiles:

```
# Filtrar por puerto del VM service
flutter attach --device-vm-service-port 8181

# Evitar el prompt si hay varios procesos (Android: package name / iOS: bundle id)
flutter attach --app-id com.example.miapp
```

Nota: la app debe estar corriendo en modo `debug` o `profile` para poder adjuntarse. En `release` no hay VM service al que conectar.

9. Configuración para testing

Para ejecutar tests con debugging:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Test (all)",
      "type": "dart",
      "request": "launch",
      "program": "test/widget_test.dart"
    },
    {
      "name": "Test (specific)",
      "type": "dart",
      "request": "launch",
      "program": "test/features/auth/login_test.dart",
      "args": ["--name", "login exitoso"]
    }
  ]
}
```

10. Compounds: ejecutar múltiples configuraciones

Los compounds permiten lanzar varias configuraciones simultáneamente:

```

{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "App Flutter",
      "type": "dart",
      "request": "launch",
      "program": "lib/main.dart"
    },
    {
      "name": "Worker Dart",
      "type": "dart",
      "request": "launch",
      "program": "bin/worker.dart"
    }
  ],
  "compounds": [
    {
      "name": "App + Worker",
      "configurations": ["App Flutter", "Worker Dart"],
      "stopAll": true,
      "preLaunchTask": ""
    }
  ]
}

```

- `stopAll` : detiene todas las configuraciones al terminar una
- `preLaunchTask` : ejecuta una task de VS Code (de `.vscode/tasks.json`) antes de lanzar

11. Variable Substitution

`launch.json` soporta variables que se resuelven en tiempo de ejecución:

11.1 Variables comunes

Variable	Qué resuelve	Ejemplo
<code>\${workspaceFolder}</code>	Raíz del workspace	<code>/home/user/mi-app</code>
<code>\${file}</code>	Archivo activo en el editor	<code>lib/main.dart</code>
<code>\${fileBasename}</code>	Solo el nombre del archivo	<code>main.dart</code>
<code>\${fileDirname}</code>	Directorio del archivo activo	<code>lib</code>
<code>\${cwd}</code>	Directorio de trabajo actual	<code>/home/user/mi-app</code>
<code>\${env:NAME}</code>	Variable de entorno	<code>\${env:HOME}</code> → <code>/home/user</code>
<code>\${command:يد}</code>	Resultado de un comando	<code>\${command:flutter}</code>

11.2 Ejemplo práctico

```
{
  "name": "Debug current file",
  "type": "dart",
  "request": "launch",
  "program": "${file}"
}
```

Esto lanza siempre el archivo que tengas abierto. Útil para debugging rápido.

12. Platform-specific properties

Puedes definir propiedades que solo apliquen a una plataforma:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Flutter (Debug)",
      "type": "dart",
      "request": "launch",
      "program": "lib/main.dart",
      "windows": {
        "program": "lib/main_windows.dart"
      },
      "linux": {
        "program": "lib/main_linux.dart"
      },
      "osx": {
        "program": "lib/main_macos.dart"
      }
    }
  ]
}
```

13. presentation — Ordenar y agrupar configuraciones

Controla cómo aparecen en el dropdown de debugging:

```
{
  "name": "Flutter (Debug)",
  "type": "dart",
  "request": "launch",
  "program": "lib/main.dart",
  "presentation": {
    "order": 1,
    "group": "Flutter",
    "hidden": false
  }
}
```

Propiedad	Qué hace
<code>order</code>	Orden en el dropdown (menor = primero)
<code>group</code>	Agrupar bajo un submenu
<code>hidden</code>	Ocultar del dropdown (accesible solo vía Command Palette)

14. `preLaunchTask` y `postDebugTask`

Integra con `tasks.json` para ejecutar tareas antes/después de debug:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Flutter (Debug)",
      "type": "dart",
      "request": "launch",
      "program": "lib/main.dart",
      "preLaunchTask": "flutter pub get",
      "postDebugTask": "flutter analyze"
    }
  ]
}
```

Requiere un `tasks.json` correspondiente:

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "flutter pub get",
      "type": "shell",
      "command": "flutter",
      "args": ["pub", "get"],
      "presentation": { "reveal": "silent" }
    },
    {
      "label": "flutter analyze",
      "type": "shell",
      "command": "flutter",
      "args": ["analyze"],
      "presentation": { "reveal": "always" }
    }
  ]
}
```

15. Mejores prácticas

1. **Nunca guardes secrets en `launch.json`**: usa variables de entorno o `.env`
 2. **Usa `flutterMode` en lugar de banderas sueltas**: es más limpio que `toolArgs: ["--profile"]`
 3. **Nombra tus configuraciones descriptivamente**: "App (Dev)" no "Debug 1"
 4. **Ten una configuración por plataforma**: para no cambiar cada vez
 5. **Commit el `launch.json`**: es parte del proyecto, todos lo necesitan
 6. **Usa `dartDefine` en vez de `--dart-define` dentro de `toolArgs`**: es más legible
-

16. Scratchpads: lanzar un entrypoint aislado por capa

Cuando desarrollas en modo andamiaje (ver [31-debugging-por-capa.md](#)), creas entrypoints aislados en `tool/scratch/` para **correr cada capa de Clean Architecture por separado con el debugger**. Añade una configuración por cada scratchpad:

```
{
  "name": "Scratch: UseCase (dart)",
  "type": "dart",
  "request": "launch",
  "program": "tool/scratch/main_usecase.dart"
},
{
  "name": "Scratch: DataSource (flutter)",
  "type": "dart",
  "request": "launch",
  "program": "tool/scratch/main_datasource.dart",
  "dartDefine": [
    "SUPABASE_URL=http://localhost:54321",
    "SUPABASE_ANON_KEY=eyJ..."
  ]
}
```

Tip: Agrúpalos con `presentation` (§13) en un submenú `Scratch: *` para no ensuciar la lista de configuraciones de la app.

Capas puras sin Flutter (entity, model, usecase → `dart run`) vs **capas con Flutter** (datasource, cubit, page → `flutter run -t`). La única diferencia en el `launch.json` es el `program`; el runtime lo decide el contenido de la capa.

12. Ejemplo completo de `launch.json` para un proyecto real

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Flutter (Debug)",
      "type": "dart",
      "request": "launch",
      "program": "lib/main.dart"
    },
    {
      "name": "Flutter (Profile)",
      "type": "dart",
      "request": "launch",
      "program": "lib/main.dart",
      "flutterMode": "profile"
    },
    {
      "name": "Android (Debug)",
      "type": "dart",
      "request": "launch",
      "program": "lib/main.dart",
      "toolArgs": ["-d", "android"]
    },
    {
      "name": "Chrome (Debug)",
      "type": "dart",
      "request": "launch",
      "program": "lib/main.dart",
      "toolArgs": ["-d", "chrome"]
    },
    {
      "name": "Dev (Local Supabase)",
      "type": "dart",
      "request": "launch",
      "program": "lib/main_development.dart",
      "dartDefine": ["SUPABASE_URL=http://localhost:54321"]
    },
    {
      "name": "Attach to running app",
      "type": "dart",
      "request": "attach",
      "program": "lib/main.dart"
    },
    {
      "name": "Test (all)",
      "type": "dart",
      "request": "launch",
      "program": "test/widget_test.dart"
    }
  ]
}
```


Resumen

Propiedad	Descripción	Ejemplo
name	Nombre de la config	"Flutter (Debug)"
type	Tipo de debugger	"dart"
request	Launch o attach	"launch"
program	Punto de entrada	"lib/main.dart"
flutterMode	Modo de compilación	"debug", "profile", "release"
toolArgs	Args de flutter run	["-d", "android"]
dartDefine	Valores --dart-define	["ENV=dev"]
deviceId	Dispositivo forzado	"emulator-5554"
env	Variables de entorno	{"KEY": "value"}
args	Args del programa/test	["--name", "login"]

Referencias

- Flutter | VS Code — Run and debug — Documentación oficial de debugging en VS Code
 - VS Code | Debugging — Conceptos de launch.json y configuraciones
 - VS Code | Debugging configuration — Variable substitution, platform-specific, presentation
 - VS Code | Tasks — Para preLaunchTask y postDebugTask
 - Dart-Code | Debugging — Guía oficial de la extensión Dart-Code
-  **Siguiente:** 03-breakpoints-avanzados.md — Todos los tipos de breakpoints disponibles en VS Code

03 — Breakpoints Avanzados

Todos los tipos de breakpoints que ofrece VS Code y cómo aplicarlos en un flujo de trabajo con BLoC/Cubit y streams.

1. Tipos de breakpoints

VS Code soporta varios tipos de breakpoints, cada uno para un caso de uso específico. Los breakpoints son la herramienta fundamental del debugging: pausan la ejecución en un punto exacto del código para que puedas inspeccionar el estado.

Tipo	Pausa?	Cuándo usar
Básico (línea)	Sí	Punto de pausa general
Condicional (Expression)	Sí	Cuando se cumple una condición
Hit Count	Sí	Después de N ocurrencias
Triggered (Wait for)	Sí	Cuando otro breakpoint se activa primero
Inline	Sí	En una columna específica de una línea
Function	Sí	Cuando se llama una función por nombre
Data	Sí	Cuando cambia el valor de una variable
Logpoint	No	Logging sin pausar

2. Breakpoints básicos

2.1 Cómo crearlos

- **Click en el gutter** (margen izquierdo del editor, junto al número de línea)
- **F9** en la línea actual
- **Click derecho** > "Add Breakpoint"

2.2 Visual

```
15: emit(LoginLoading()); // Círculo ROJO lleno = breakpoint activo
16: final result = await loginUseCase(params);
```

2.3 Estados visuales

Icono	Estado	Significado
Círculo rojo lleno	Activo	Pausará aquí
Círculo gris lleno	Disabled	Existe pero no pausa
Círculo gris hueco	No registrado	El debugger no puede registrar este breakpoint

2.4 Gestionar desde la sección BREAKPOINTS

En el panel lateral Run and Debug > BREAKPOINTS:

- Click en el círculo para habilitar/deshabilitar
- Click derecho para editar/eliminar
- Drag para reordenar

3. Breakpoints condicionales

Pausan solo cuando se cumple una condición. Útiles cuando un breakpoint se ejecuta muchas veces pero solo te interesa una.

3.1 Expression (expresión)

Pausa cuando la expresión evalúa a `true`.

Crear:

1. Click derecho en el gutter > "Add Conditional Breakpoint"
2. Seleccionar "Expression"
3. Escribir la expresión

Ejemplos en Flutter:

```
// Pausar solo cuando el email contiene "@gmail.com"
// Condición: event.email.contains("@gmail.com")

// Pausar solo cuando el estado es error
// Condición: state.is>LoginError

// Pausar solo cuando la lista tiene más de 100 items
// Condición: items.length > 100

// Pausar solo cuando un ID es específico
// Condición: user.id == "abc-123"
```

3.2 Hit count (conteo de veces)

Pausa después de que el breakpoint se ejecuta N veces.

Crear:

1. Click derecho > "Add Conditional Breakpoint"
2. Seleccionar "Hit Count"
3. Escribir la condición: `> 5`, `== 10`, `>= 100`

Ejemplos:

```
> 10    // Pausa después de la 10.ª vez
== 1    // Pausa solo la primera vez
>= 50   // Pausa a partir de la 50.ª vez
```

Caso de uso real: un listener se ejecuta en cada frame de animación, pero solo quieres ver el estado en el frame 30.

3.3 Wait for breakpoint (triggered)

Pausa cuando OTRO breakpoint se activa primero.

Crear:

1. Click derecho > "Add Conditional Breakpoint"
2. Seleccionar "Wait for Breakpoint"
3. Seleccionar qué breakpoint debe activarse primero

Caso de uso real: quieres inspeccionar el estado DESPUÉS de que se ejecuta una función específica.

4. Inline breakpoints

Pausan en una columna específica de una línea. Útiles para líneas con múltiples statements.

4.1 Cómo crearlos

- **Ctrl+Shift+F9** (Windows/Linux) o **Shift+F9** (macOS) durante una sesión de debugging
- Click derecho > "Add Inline Breakpoint"

4.2 Ejemplo

```
// Si esta línea tiene múltiples statements:  
return User(name: getName(), email: getEmail(), age: getAge());  
  
// Un inline breakpoint pausa en un statement específico:  
return User(name: getName(), ← breakpoint aquí  
           email: getEmail(),  
           age: getAge());
```

4.3 Visual

```
42:   return User(name: getName(), | email: getEmail(), | age: getAge());  
    | breakpoint 1             | breakpoint 2       | breakpoint 3
```

5. Function breakpoints

Pausan cuando se invoca una función por nombre. Útiles cuando no tienes el source code o la función está en un paquete externo.

5.1 Cómo crearlos

1. En el panel BREAKPOINTS, click en el icono 
2. Seleccionar "Function Breakpoint"
3. Escribir el nombre de la función

5.2 Ejemplos en Flutter

```
// Pausar cuando se llama setState  
setState  
  
// Pausar cuando se llama Navigator.push  
Navigator.push  
  
// Pausar cuando se llama print  
print  
  
// Pausar en un método de un paquete  
http.get
```

Nota: el soporte de function breakpoints depende del debugger. En el debugger de Dart funciona mejor con nombres completos; si no se resuelve, usa un breakpoint de línea en la primera línea de la función.

5.3 Visual

En el panel BREAKPOINTS aparece un triángulo rojo:

```
▲ setState
▲ Navigator.push
```

6. Data breakpoints

Pausan cuando el valor de una variable CAMBIA. Son extremadamente poderosos para encontrar dónde se modifica un dato.

6.1 Cómo crearlos

1. En el panel VARIABLES, busca la variable
2. Click derecho > "Break on Value Change"
3. Opciones: "Break on Value Change", "Break on Value Read", "Break on Access"

6.2 Tipos

Tipo	Qué hace	Cuándo usar
Value Change	Pausa cuando el valor CAMBIA	Encontrar dónde se modifica un dato
Value Read	Pausa cuando el valor SE LEE	Encontrar quién consume un dato
Value Access	Pausa cuando el valor SE ACCESA (read o write)	Auditoría completa de acceso

Nota: los data breakpoints requieren soporte del debugger y de la plataforma. En Flutter el debugger de Dart tiene soporte limitado; si no está disponible para tu configuración, usa breakpoints condicionales en las líneas de escritura como alternativa.

6.3 Ejemplo real

```
// Tienes un UserEntity con un campo 'name'
class UserEntity {
    String name;
    // ...
}

// Quieres saber QUIÉN cambia el nombre del usuario
// 1. En VARIABLES, busca 'user'
// 2. Click derecho > "Break on Value Change" en 'name'
// 3. Ahora pausará cada vez que alguien haga user.name = "..."
```

6.4 Visual

En el panel BREAKPOINTS aparece un hexágono rojo:

- user.name (Change)
- user.email (Read)

7. Logpoints (breakpoints de logging)

No pausan la ejecución. Solo imprimen un mensaje en la Debug Console. Son como `print()` pero sin modificar el código.

7.1 Cómo crearlos

1. Click derecho en el gutter > "Add Logpoint"
2. Escribir el mensaje (puede incluir expresiones con `{}`)

7.2 Ejemplos

```
// Log simple
"El usuario hizo login"

// Log con variable
"Email: {event.email}"

// Log con expresión compleja
"Estado: {state.runtimeType}, Items: {state.items.length}"

// Log condicional (solo si la expresión no es null)
"User ID: {user?.id ?? 'no user'}"
```

7.3 Visual

```
15: emit(LoginLoading()); // Diamante AZUL = logpoint
16: final result = await loginUseCase(params);
```

7.4 Comparación con print()

Aspecto	Logpoint	print()
Modifica el código	NO	SÍ
Se puede habilitar/deshabilitar	SÍ	NO
Tiene condiciones	SÍ	NO (hay que escribir if)
Se ve en Debug Console	SÍ	SÍ (en terminal)
Se commit a git	NO	SÍ (hay que eliminarlo)

Regla: usa logpoints en lugar de `print()` para debugging temporal. No contaminan el código.

8. Práctica guiada: breakpoints en un BLoC

8.1 Escenario

Tienes un BLoC de carrito de compras y el total no se calcula correctamente.


```
// lib/presentation/bloc/cart/cart_bloc.dart
class CartBloc extends Bloc<CartEvent, CartState> {
  final CalculateTotalUseCase calculateTotalUseCase;

  CartBloc({required this.calculateTotalUseCase}) : super(CartInitial()) {
    on<AddItemToCart>(_onAddItem);
    on<RemoveItemFromCart>(_onRemoveItem);
    on<RecalculateTotal>(_onRecalculate);
  }

  Future<void> _onAddItem(
    AddItemToCart event,
    Emitter<CartState> emit,
  ) async {
    final currentState = state as CartLoaded;
    final updatedItems = [...currentState.items, event.item];

    final total = await calculateTotalUseCase(updatedItems); // LÍNEA 28

    emit(CartLoaded(
      items: updatedItems,
      total: total,
    ));
  }

  Future<void> _onRemoveItem(
    RemoveItemFromCart event,
    Emitter<CartState> emit,
  ) async {
    final currentState = state as CartLoaded;
    final updatedItems = currentState.items
      .where((item) => item.id != event.itemId)
      .toList();

    final total = await calculateTotalUseCase(updatedItems); // LÍNEA 43

    emit(CartLoaded(
      items: updatedItems,
      total: total,
    ));
  }
}
}
```

8.2 Estrategia de debugging

Breakpoint 1: Condición en línea 28

Condición: `event.item.price < 0`
 Mensaje: `"Item con precio negativo detectado: {event.item.name}"`

Esto pausa si alguien agrega un item con precio negativo.

Breakpoint 2: Logpoint en línea 43

Mensaje: `"Calculando total para {updatedItems.length} items"`

Esto te muestra cuántos items hay al momento de recalcular.

Breakpoint 3: Condicional en `total` (línea donde se emite el estado)

```
Condición: total < 0
```

Esto pausa si el total calculado es inválido.

9. Práctica: debugging de un stream

9.1 Escenario

Un stream de Supabase no está emitiendo los eventos esperados.

```
// En tu datasource
final stream = supabase
    .from('messages')
    .stream(primaryKey: ['id'])
    .order('created_at');

stream.listen((data) {
    // LÍNEA 8: Este listener no se ejecuta
    debugPrint('Nuevo mensaje: $data');
});
```

9.2 Estrategia

Breakpoint condicional en línea 8:

```
Condición: data.isEmpty
```

Esto pausa solo si llega un stream vacío (posible problema).

Logpoint antes del listen:

```
Mensaje: "Suscribiéndose al stream de mensajes"
```

Breakpoint de línea en el `listen`: Esto pausa cada vez que el stream emite nuevos datos (para confirmar que los eventos llegan).

Resumen

Tipo	Pausa?	Cuándo usar
Básico	Sí	Punto de pausa general

Tipo	Pausa?	Cuándo usar
Condición - Expression	Sí	Cuando se cumple una condición
Condición - Hit Count	Sí	Después de N veces
Triggered	Sí	Cuando otro breakpoint se activa
Inline	Sí	En una columna específica
Function	Sí	Cuando se llama una función
Data	Sí (soporte limitado)	Cuando cambia un valor
Logpoint	No	Logging sin pausar

Referencias

- [VS Code | Breakpoints](#) — Tipos de breakpoints en VS Code
- [Dart-Code | Debugging](#) — Detalles del debugger Dart/Flutter
- [VS Code | Keyboard shortcuts](#) — Cómo personalizar atajos

 **Siguiente:** [04-inspeccion-datos-consola.md](#) — Inspeccionar variables y evaluar expresiones en la Debug Console

04 — Inspección de Datos y Debug Console

Cómo inspeccionar variables, monitorear expresiones con WATCH y usar la Debug Console como REPL durante una sesión de debugging.

1. Inspeccionar durante debugging

Cuando la app se pausa en un breakpoint, VS Code te muestra información detallada sobre el estado del programa. Esta información se distribuye en tres áreas principales: el panel **VARIABLES**, el panel **WATCH** y la **Debug Console**.

2. Panel VARIABLES

Muestra las variables del scope actual. Se actualiza automáticamente cuando cambias de stack frame en CALL STACK.

2.1 Qué muestra

- **Local**: variables declaradas en la función actual
- **Closure**: variables capturadas por closures
- **Global**: variables globales del archivo
- **Fields**: campos de la instancia actual (si estás en un método)

2.2 Interactuar con variables

Acción	Cómo	Qué hace
Expandir objeto	Click en flecha	Muestra campos internos
Ver valor	Hover sobre variable	Muestra tooltip con valor
Copiar valor	Click derecho > Copy Value	Copia el valor al clipboard
Copiar como expresión	Click derecho > Copy as Expression	Copia una expresión para usar en WATCH
Modificar valor	Click derecho > Set Value	Cambia el valor en tiempo real

Acción	Cómo	Qué hace
Filtrar	<code>Ctrl+Alt+F</code> (Win/Linux) / <code>Alt+Cmd+F</code> (macOS)	Busca por nombre o valor

2.3 Ejemplo de inspección

```
void _onLoginSubmitted(LoginSubmitted event, Emitter<LoginState> emit) {
  emit(LoginLoading());
  // BREAKPOINT AQUÍ
}
```

En VARIABLES verás:

```
event
  email: "usuario@ejemplo.com"
  password: "secreto123"
state
  LoginLoading (runtimeType)
this
  loginUseCase: LoginUseCase
  repository: AuthRepositoryImpl
```

2.4 Modificar variables en tiempo real

Imagina que quieres probar qué pasa si el email está vacío:

1. Click derecho en `event.email`
2. Seleccionar "Set Value"
3. Escribir `""` (string vacío)
4. Continuar con F5
5. La app usará el email vacío

Esto es extremadamente útil para:

- Probar edge cases sin modificar el código
- Simular datos inválidos
- Verificar que la validación funciona

3. Panel WATCH

Permite monitorear expresiones personalizadas que se evalúan en cada pausa.

3.1 Cómo agregar expresiones

1. En el panel WATCH, click en `+`
2. Escribir una expresión Dart válida

3. Presionar Enter

3.2 Ejemplos de expresiones en WATCH

```
// Ver el tipo del estado actual
state.runtimeType

// Ver la cantidad de items en una lista
state.items.length

// Ver si un usuario está autenticado
user != null

// Ver el total formateado
'\${total.toStringAsFixed(2)}'

// Ver todos los nombres de usuarios
users.map((u) => u.name).toList()

// Ver el tiempo transcurrido
DateTime.now().difference(startTime).inSeconds

// Ver si un stream tiene listeners
streamController.hasListener
```

3.3 Consejos para WATCH

- **Expresiones Dart válidas:** puedes usar cualquier expresión que sea válida en Dart
- **Se evalúan en cada pausa:** si el valor cambia, se actualiza automáticamente
- **Puedes agregar múltiples expresiones:** no hay límite
- **Click derecho para opciones:** copiar resultado, remover expresión

4. Debug Console como REPL

La Debug Console es un REPL (Read-Eval-Print Loop) que te permite ejecutar código Dart en el contexto actual de la sesión de debugging.

4.1 Abrir la Debug Console

- `Ctrl+Shift+Y` (Windows/Linux)
- `Cmd+Shift+Y` (macOS)
- Click en "DEBUG CONSOLE" en el panel inferior

4.2 Qué puedes hacer

1. Evaluar expresiones simples

```
> event.email
"usuario@ejemplo.com"

> state.runtimeType
"LoginLoading"

> items.length
42
```

2. Llamar métodos

```
> user.toString()
"User(id: 123, name: Juan, email: juan@test.com)"

> items.isEmpty
true

> total.toStringAsFixed(2)
"1234.50"
```

3. Ejecutar código complejo

```
> items.where((i) => i.price > 100).map((i) => i.name).toList()
["Laptop", "Tablet", "Monitor"]

> DateTime.now().difference(order.createdAt).inHours
48
```

4. Modificar variables

```
> email = ""
> items = []
> user = null
```

5. Invocar funciones

```
> await calculateTotal(items)
1500.00
```

4.3 Multiline input

Para escribir código en múltiples líneas:

1. Escribe la primera línea
2. Presiona **Shift+Enter** para nueva línea
3. Cuando termines, presiona **Enter** para ejecutar

Ejemplo:

```
> final result = items
  .where((i) => i.isActive)
  .map((i) => i.name)
  .toList();
> result
["Laptop", "Tablet"]
```

4.4 Sugerencias de autocompletado

La Debug Console soporta autocompletado:

- Escribe el nombre de una variable y presiona **Tab** o **Enter**
- Escribe **.** después de un objeto para ver métodos disponibles
- Usa **Ctrl+Space** para forzar autocompletado

5. Ejemplo práctico completo

5.1 Escenario

Tienes un BLoC de carrito y el total no es correcto. Quieres diagnosticar.

```
class CartBloc extends Bloc<CartEvent, CartState> {
  // ...
  Future<void> _onAddItem(AddItemToCart event, Emitter<CartState> emit) async {
    final currentState = state as CartLoaded;
    final updatedItems = [...currentState.items, event.item];
    final total = await calculateTotalUseCase(updatedItems);
    // BREAKPOINT AQUÍ
    emit(CartLoaded(items: updatedItems, total: total));
  }
}
```

5.2 Paso 1: Inspeccionar en VARIABLES

```
currentState
  items: [Item(Laptop, 999.99), Item(Mouse, 29.99)]
  total: 1029.98

event
  item: Item(Keyboard, 79.99)

updatedItems
  [Item(Laptop, 999.99), Item(Mouse, 29.99), Item(Keyboard, 79.99)]
```


5.3 Paso 2: Agregar a WATCH

```
updatedItems.length      → 3
total                    → 1109.97
updatedItems.map((i) => i.price).toList() → [999.99, 29.99, 79.99]
```

5.4 Paso 3: Evaluar en Debug Console

```
> updatedItems.fold<double>(0, (sum, item) => sum + item.price)
1109.97

> total == updatedItems.fold<double>(0, (sum, item) => sum + item.price)
true
```

5.5 Paso 4: Modificar y probar

```
> event.item = Item(Keyboard, -10.00)
// Continuar con F5 y ver qué pasa con precio negativo
```

6. Inspeccionar objetos complejos

6.1 Listas

```
// En VARIABLES, expande la lista
items
  [0] → Item(Laptop, 999.99)
  [1] → Item(Mouse, 29.99)
  [2] → Item(Keyboard, 79.99)

// En Debug Console
> items[0].name
"Laptop"

> items.where((i) => i.price > 50).toList()
[Item(Laptop, 999.99), Item(Keyboard, 79.99)]
```

6.2 Mapas

```
// En VARIABLES
response
  "users" → List (3 items)
  "total" → 150
  "page" → 1

// En Debug Console
> response['users'].length
3

> (response['users'][0] as Map)['name']
"Juan"
```

6.3 Objetos con referencia circular

```
// Si un objeto tiene referencia circular, VS Code lo maneja
user
  id: 123
  name: "Juan"
  orders → [Order, Order, Order]
    [0]
      user → * [Circular] // Muestra que es circular
```

7. Consejos avanzados

7.1 Usar Debug Console para testing rápido

Antes de crear un unit test, prueba la lógica en Debug Console:

```
> final result = validateEmail("test@test.com")
> result.isValid
true
```

7.2 Inspeccionar errores

```
> try { await apiCall(); } catch (e) { e.toString(); }
"SocketException: Connection timed out"
```

7.3 Verificar el estado de un BLoC

```
> bloc.state  
CartLoaded(items: 3, total: 1109.97)  
  
> bloc.state.runtimeType  
"CartLoaded"  
  
> bloc.isClosed  
false
```

7.4 Medir tiempo de ejecución

```
> final sw = Stopwatch()..start();  
> await heavyComputation();  
> sw.stop();  
> sw.elapsedMilliseconds  
342
```

Resumen

Herramienta	Para qué	Atajo
VARIABLES	Ver/inspeccionar variables del scope actual	Click en panel
WATCH	Monitorear expresiones personalizadas	+ en panel WATCH
Debug Console	Ejecutar código Dart en tiempo real	Ctrl+Shift+Y
Set Value	Modificar variables durante ejecución	Click derecho en VARIABLES
Copy as Expression	Copiar expresión para WATCH	Click derecho en VARIABLES
Filter Variables	Buscar variables por nombre/valor	Ctrl+Alt+F

Referencias

- [VS Code | Debugging](#) — Paneles VARIABLES, WATCH y Debug Console
- [Dart | dart:developer](#) — Utilidades de debugging del SDK
- [Flutter | DevTools](#) — Herramientas complementarias del ecosistema

 **Siguiente:** 05-multi-target-remoto.md — Debugging multi-target, attach y trabajo remoto

05 — Multi-target y Debugging Remoto

Depurar varios procesos a la vez, adjuntarte a una app en ejecución y trabajar contra dispositivos y backends remotos.

1. Multi-target debugging

Permite depurar múltiples procesos simultáneamente: tu app Flutter + un script Dart, o dos instancias de la misma app.

1.1 Depurar Dart CLI + Flutter App en paralelo

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Flutter App",
      "type": "dart",
      "request": "launch",
      "program": "lib/main.dart"
    },
    {
      "name": "Worker Dart",
      "type": "dart",
      "request": "launch",
      "program": "bin/worker.dart"
    }
  ],
  "compounds": [
    {
      "name": "App + Worker",
      "configurations": ["Flutter App", "Worker Dart"]
    }
  ]
}
```

1.2 ¿Cuándo usar compounds?

- App frontend + worker isolate procesando datos
- Microservicio Dart local + cliente que lo consume
- Dos instancias de app para probar comunicación
- Migración de datos que corre en paralelo con la app

2. Attach a un proceso existente

2.1 Desde VS Code

La app ya está corriendo (p. ej. lanzada con `flutter run`) y quieres adjuntarte desde VS Code:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Attach to running Flutter",
      "type": "dart",
      "request": "attach",
      "program": "lib/main.dart"
    }
  ]
}
```

La extensión descubre automáticamente el **VM Service** de la app en ejecución. No hace falta copiar ninguna URI.

2.2 Pausar la app en el arranque

Si necesitas que la app espere a que un debugger se conecte (útil en CI o lanzando fuera de VS Code):

```
flutter run --start-paused
```

Luego adjunta un debugger desde VS Code (`request: "attach"`).

2.3 Forzar puertos del VM Service / DDS

Normalmente no hace falta, pero si tu firewall o tooling lo exige:

```
# Fijar el puerto del Dart Development Service (DDS)
flutter run --dds-port 8181

# Al adjuntar, buscar el VM service en un puerto específico
flutter attach --device-vm-service-port 8181
```

Nota: la bandera antigua `--observatory-port` (y su sucesor `--vm-service-port`) fue reemplazada. Hoy el proceso correcto es el DDS sobre el VM Service, y la detección automática hace innecesario fijar puertos en la mayoría de los casos.

3. Chrome remoto (web)

3.1 Configuración de launch

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Flutter Web Remote",
      "type": "dart",
      "request": "launch",
      "program": "lib/main.dart",
      "deviceId": "chrome",
      "toolArgs": ["--web-port=8080", "--web-hostname=0.0.0.0"]
    }
  ]
}
```

Acceder desde otra máquina:

```
http://192.168.1.50:8080
```

`deviceId: "chrome"` (antes `device: "chrome"`) selecciona el navegador. Las banderas de servidor web van en `toolArgs` porque son de `flutter run`.

3.2 Chrome DevTools remoto

Para depurar el front-end web con el debugger de Chrome:

```
# En la máquina remota, inicia Chrome con depuración remota:
google-chrome --remote-debugging-port=9222 --user-data-dir=/tmp/chrome-debug

# Desde tu máquina, haz port forwarding por SSH:
ssh -L 9222:localhost:9222 usuario@maquina-remota
```

4. Dispositivos físicos remotos

4.1 ADB inalámbrico (Android 11+)

Empareja el dispositivo por red (Android Developer Options → **Wireless debugging**):

```
# Pairing code
adb pair 192.168.1.100:37000

# Conectar
adb connect 192.168.1.100:5555

# Verificar
adb devices
```

Luego lanza la app desde VS Code o terminal:

```
flutter run --debug -d 192.168.1.100:5555
```

4.2 Dispositivo físico por USB

Sin configuración extra: `flutter run` detecta el dispositivo conectado por USB.

5. Debugging con Docker y Supabase local

5.1 Supabase local (CLI)

El entorno local de Supabase corre en contenedores. La forma recomendada es el CLI oficial:

```
# Inicia los contenedores de Supabase local (Postgres, Auth, Realtime, Storage, Functions)
supabase start

# Detener los contenedores (mantiene los datos)
supabase stop

# Estado y URLs del proyecto local
supabase status
```

`supabase start` levanta el API en `http://localhost:54321` y la base de datos en el puerto `54322`.

5.2 Conectar la app Flutter al backend local

```
// En tu cliente Supabase (modo desarrollo):
final supabase = SupabaseClient(
  'http://localhost:54321', // API local de Supabase
  supabaseAnonKey,        // Anon key local de supabase status
);
```

5.3 `launch.json` para desarrollo con Supabase local

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Flutter + Supabase Local",
      "type": "dart",
      "request": "launch",
      "program": "lib/main.dart",
      "dartDefine": ["SUPABASE_URL=http://localhost:54321"]
    }
  ]
}
```

5.4 Levantar Supabase antes de depurar (tasks)

Crea una task en `.vscode/tasks.json` :

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "supabase: start",
      "type": "shell",
      "command": "supabase start",
      "problemMatcher": []
    }
  ]
}
```

Y referénciala con `preLaunchTask` en el compound:

```
{
  "compounds": [
    {
      "name": "Supabase Local + App",
      "configurations": ["Flutter + Supabase Local"],
      "preLaunchTask": "supabase: start"
    }
  ]
}
```


6. Depuración de tests

6.1 Test individual

```
{
  "name": "Debug Single Test",
  "type": "dart",
  "request": "launch",
  "program": "test/widget_test.dart",
  "args": ["--plain-name", "test name exacto"]
}
```

6.2 Test con patrón regex

```
{
  "name": "Debug Test Pattern",
  "type": "dart",
  "request": "launch",
  "program": "test/unit/auth_test.dart",
  "args": ["--name", "login.*error"]
}
```

6.3 Todos los tests de un archivo

```
{
  "name": "Debug All Auth Tests",
  "type": "dart",
  "request": "launch",
  "program": "test/unit/auth_test.dart"
}
```

6.4 Usando la Testing Side Bar de VS Code

1. Click en el icono de Testing (matraz) en la sidebar
 2. Hover sobre un test → aparece "Debug Test"
 3. Pon breakpoints en el test → ejecuta con debug
-

7. Debugging de isolates

7.1 Crear isolate con nombre identificable

```
import 'dart:isolate';

// Crear isolate con nombre identificable
await Isolate.spawn(
  _isolateEntryPoint,
  message,
  debugName: 'image-processor', // ← Visible en VS Code
);
```

7.2 VS Code detecta automáticamente los isolates

- En **CALL STACK** verás cada isolate como entry separado
- Click en un isolate diferente → inspeccionar sus variables
- Cada isolate tiene su propio punto de ejecución

7.3 Pausar todos los isolates al arrancar

```
{
  "name": "Flutter with Isolates",
  "type": "dart",
  "request": "launch",
  "program": "lib/main.dart",
  "toolArgs": ["--start-paused"]
}
```

8. Depuración de errores de plataforma

8.1 Logs de Android (native)

```
# Logs de Flutter desde terminal
adb logcat -s flutter

# Buscar errores específicos
adb logcat | grep -E "(Exception|Error|FATAL)"
```

8.2 Logs de iOS (native)

```
# Simulator logs
xcrun simctl spawn booted log stream --level=debug --predicate 'process == "Runner"'

# Device logs
idevicesyslog -u DEVICE_ID
```

8.3 Task de VS Code para logs

```
{
  "label": "Android Logs",
  "type": "shell",
  "command": "adb",
  "args": ["logcat", "-s", "flutter"],
  "isBackground": true,
  "problemMatcher": []
}
```

9. El VM Service y DevTools

Cuando corres la app en debug, Flutter expone un **VM Service** (por defecto sobre el DDS) que usan VS Code y DevTools para inspeccionar la app. Puedes abrir DevTools de varias formas:

```
# Mientras corre flutter run, presiona "d" en la terminal
d

# Desde otra terminal, adjuntando el URI que imprime flutter run:
dart devtools http://127.0.0.1:8181/XXXXXX/
```

`dart devtools` acepta el URI del VM Service como argumento posicional (o lanza una página que se conecta sola a la app descubierta).

10. Resumen de patrones de conexión

Escenario	Método	Puerto/Tipo
Local emulator	Directo	Automático
Dispositivo físico USB	Directo	USB
Dispositivo WiFi/inalámbrico	<code>adb connect</code> / wireless pairing	WiFi
Chrome web	<code>deviceId: "chrome"</code> + <code>toolArgs</code>	Automático

Escenario	Método	Puerto/Tipo
Backend Supabase local	<code>supabase start</code>	54321/54322
SSH remote	Port forwarding	SSH tunnel
Dos procesos Dart	Compound launch	Múltiples
Tests	Program launch	Test runner

Comandos útiles de terminal

```
# Ver todos los dispositivos conectados
flutter devices

# Lanzar la app esperando un debugger
flutter run --start-paused

# Adjuntarse a una app en ejecución
flutter attach

# Adjuntarse filtrando por app (Android package / iOS bundle id)
flutter attach --app-id com.example.miapp

# Abrir DevTools contra la app descubierta
dart devtools
```

Resumen

Escenario	Cómo conectarse	Nota
App Flutter + script Dart	Compound launch	Breakpoints en ambos procesos
App en ejecución	<code>flutter attach</code>	Mismo codebase y target
Dispositivo físico / WiFi	<code>adb</code> / wireless pairing	USB o red local
Chrome web	<code>deviceId: "chrome"</code>	Depuración en el navegador
Backend Supabase local	<code>supabase start</code>	Puerto 54321/54322
Máquina remota	SSH port forwarding	Tunnel hacia el VM Service
Tests	Program launch	Breakpoints en tests
Isolates	Detección automática	<code>--start-paused</code> pausa todos

Referencias

- [Flutter | VS Code — Run and debug](#) — Configuraciones de launch/attach
 - [Flutter | Debugging tools](#) — Panorama de herramientas de debugging
 - [Flutter | DevTools CLI](#) — Cómo abrir DevTools desde la terminal
 - [Supabase | Local development](#) — `supabase start` y el stack local
-

 **Siguiente:** [06-cheatsheet-vscode.md](#) — Cheatsheet de atajos, expresiones y plantillas de VS Code

06 — Cheatsheet: Debugging en VS Code

Referencia rápida de atajos, expresiones, plantillas de `launch.json` y errores comunes al depurar Flutter en VS Code.

1. Atajos de teclado esenciales

Atajo	Acción	Descripción
<code>F5</code>	Start / Continue	Iniciar depuración o continuar hasta el próximo breakpoint
<code>Ctrl+F5</code>	Run Without Debugging	Ejecutar sin pausar (sin sesión de debug)
<code>Shift+F5</code>	Stop	Detener depuración
<code>Ctrl+Shift+F5</code>	Restart	Reiniciar depuración (en Flutter equivale al hot restart)
<code>F10</code>	Step Over	Ejecutar siguiente línea sin entrar en funciones
<code>F11</code>	Step Into	Entrar dentro de función/método
<code>Shift+F11</code>	Step Out	Salir de la función actual
<code>F9</code>	Toggle Breakpoint	Agregar/quitar breakpoint en la línea actual
<code>Ctrl+Shift+F9</code>	Inline Breakpoint	Pausar en un statement específico de la línea (Win/Linux; en macOS: <code>Shift+F9</code>)

Nota: "Run to Cursor", "Conditional Breakpoint" y "Disable All Breakpoints" no tienen atajo por defecto: se acceden con click derecho o desde la Command Palette. Puedes asignarles atajos en el editor de Keyboard Shortcuts.

1.1 Navegación de paneles

Atajo	Panel
<code>Ctrl+Shift+D</code>	Run and Debug (sidebar)
<code>Ctrl+Shift+Y</code>	Debug Console
<code>Ctrl+Shift+E</code>	Explorer
<code>Ctrl+Shift+F</code>	Search

Atajo	Panel
<code>Ctrl+Shift+P</code>	Command Palette

2. Panel VARIABLES

Sección	Qué contiene
Local	Variables declaradas en la función actual
Closure	Variables capturadas por closures
Global	Variables globales del archivo
Fields	Campos de la instancia actual (en un método)

2.1 Acciones útiles

Acción	Cómo
Expandir objeto	Click en la flecha
Ver valor	Hover sobre la variable
Copiar valor	Click derecho > Copy Value
Copiar como expresión	Click derecho > Copy as Expression
Modificar valor	Click derecho > Set Value
Filtrar	<code>Ctrl+Alt+F</code> (Win/Linux) / <code>Alt+Cmd+F</code> (macOS)

2.2 CALL STACK

Elemento	Significado
► Frame actual	Punto de ejecución donde está pausado
⏏ Frame pausado	Pausado en un breakpoint
🧵 Hilo	Hilo de ejecución (en Dart: isolate)
🏠 Isolate	Aislamiento de memoria con su propio stack

3. Sintaxis de expresiones (WATCH / Debug Console)

3.1 Acceso a propiedades

```
variable.property  
variable.property.subProperty
```

3.2 Índices y colecciones

```
myList[0]  
myMap["key"]  
mySet.first
```

3.3 Métodos de inspección

```
myVariable.toString()  
myVariable.runtimeType  
myList.length  
myList.isEmpty  
myMap.keys.toList()
```

3.4 Operaciones condicionales

```
counter > 0 ? "positive" : "negative"  
user?.name ?? "Anonymous"  
list.isNotEmpty
```

3.5 Cuidado con side effects

```
// Llamar métodos puede ejecutar lógica y cambiar estado  
user.isValid()  
controller.text
```

4. Tipos de breakpoints — Referencia rápida

Tipo	Icono	Cómo crearlo	Cuándo usarlo
Línea	●	Click gutter o F9	Punto de parada básico
Condicional (Expression)	● + 📝	Click derecho → Conditional	Cuando se cumple una expresión

Tipo	Icono	Cómo crearlo	Cuándo usarlo
Hit Count		Click derecho → Conditional → Hit Count	Cada N ocurrencias
Logpoint (Log Message)	 diamante	Click derecho → Add Logpoint	Logging sin pausar
Inline	 pequeño	<code>Ctrl+Shift+F9</code>	En un statement específico de la línea
Function		Panel BREAKPOINTS → 	Al entrar a una función por nombre
Data		VARIABLES → click derecho	Cuando cambia un valor (soporte limitado)

5. `launch.json` — Plantillas comunes

5.1 Debug Flutter básico

```
{
  "name": "Flutter Debug",
  "type": "dart",
  "request": "launch",
  "program": "lib/main.dart"
}
```

5.2 Flutter con `--dart-define`

```
{
  "name": "Flutter + Dart Define",
  "type": "dart",
  "request": "launch",
  "program": "lib/main.dart",
  "dartDefine": ["ENV=dev"]
}
```

5.3 Flutter test

```
{
  "name": "Debug Test",
  "type": "dart",
  "request": "launch",
  "program": "test/widget_test.dart",
  "args": ["--plain-name", "test name"]
}
```

5.4 Attach a un proceso existente

```
{
  "name": "Attach",
  "type": "dart",
  "request": "attach",
  "program": "lib/main.dart"
}
```

La extensión descubre automáticamente la app en ejecución. No hace falta especificar una URI del VM Service.

5.5 Multiple targets (compound)

```
{
  "compounds": [
    {
      "name": "App + Worker",
      "configurations": ["Flutter App", "Dart Worker"]
    }
  ]
}
```

6. Debug Console — Comandos útiles

6.1 Inspección directa

```
// Ver tipo de variable
myVar.runtimeType

// Ver contenido de lista
myList → [1, 2, 3, 4, 5]

// Ver contenido de mapa
myMap → { "key": "value", "count": 42 }
```

6.2 Expresiones evaluables

```
// Operaciones matemáticas
counter + 1
items.length * 2

// Strings
"Count: $counter"
user.name.toUpperCase()

// Condicionales
isActive && isVerified ? "Active User" : "Pending"

// Colecciones
users.where((u) => u.age > 18).toList()
products.map((p) => p.name).join(", ")
```

6.3 Llamadas con side effects (usar con cuidado)

```
// Imprimir valor (se ve en la consola)
print("DEBUG: $variable")

// Llamar un getter
controller.text
```

7. Hot Reload durante debugging

Acción	Cómo	Efecto
Hot Reload	r en terminal/Debug Console o botón  del toolbar	Mantiene estado, actualiza UI
Hot Restart	R en terminal o Ctrl+Shift+F5 (Debug: Restart)	Resetea estado completo
Run without debugging	Ctrl+F5	Ejecuta sin sesión de debug

7.1 Hot Reload preserva:

- Estado actual de BLoC/Cubit/Provider
- Posición de scroll
- Contenido de campos de texto
- Resultado de cálculos previos

7.2 Hot Restart resetea:

- Todos los estados a valores iniciales
- Conexiones de streams
- Datos en memoria

- Controllers (dispose + recrear)

Importante: `Ctrl+F5` es "Run Without Debugging", NO hot reload. El hot reload no tiene atajo por defecto: se dispara con `r` o desde el toolbar.

8. Errores comunes y soluciones

8.1 "Could not find a Flutter app to attach to"

Causa: No hay ninguna app en ejecución con VM service activo
Solución: Lanza la app con ``flutter run`` (modo debug) antes de hacer attach

8.2 "Application finished"

Causa: La app terminó o no está en modo debug
Solución: Verifica que estés en modo `--debug`, no `--release`

8.3 "Function breakpoints not working"

Causa: El debugger de Dart no resuelve el nombre de la función
Solución: Usa un breakpoint de línea en la primera línea de la función

8.4 "Variables show "

Causa: El compilador optimizó la variable (ocurre en profile/release)
Solución: Usa `--debug`, o agrega un `print()/logpoint` antes del breakpoint

9. Command Palette (`Ctrl+Shift+P`) — Debug

Comando	Acción
Debug: Start Debugging	Iniciar (F5)
Debug: Stop Debugging	Detener (Shift+F5)
Debug: Restart Debugging	Reiniciar (Ctrl+Shift+F5)
Debug: Open Debug Console	Abrir Debug Console

Comando	Acción
Debug: Toggle Breakpoint	Breakpoint en el cursor
Debug: Remove All Breakpoints	Limpiar todos
Debug: Disable All Breakpoints	Deshabilitar todos
Flutter: Hot Reload	Hot reload
Flutter: Hot Restart	Hot restart
Dart: Open DevTools Inspector	Abrir el Flutter Inspector en DevTools

10. Variable Substitution — Referencia rápida

Variable	Ejemplo de resolución
<code>\${workspaceFolder}</code>	<code>/home/user/mi-app</code>
<code>\${file}</code>	<code>lib/features/login/presentation/pages/login_page.dart</code>
<code>\${fileBasename}</code>	<code>login_page.dart</code>
<code>\${fileDirname}</code>	<code>lib/features/login/presentation/pages</code>
<code>\${env:HOME}</code>	<code>/home/user</code>
<code>\${env:PATH}</code>	<code>/usr/bin:/usr/local/bin:...</code>

Uso: `"program": "${file}"` → lanza el archivo activo en el editor.

11. Platform-specific & Presentation — Referencia rápida

Platform-specific

```
"windows": {"program": "lib/main_windows.dart"},
"linux": {"program": "lib/main_linux.dart"},
"osx": {"program": "lib/main_macos.dart"}
```

Presentation

```
"presentation": {"order": 1, "group": "Flutter", "hidden": false}
```

- `order`: posición en el dropdown (menor = primero)

- `group` : agrupar en submenu
 - `hidden` : ocultar del dropdown
-

12. Flujo de debugging típico

1. F5 (Start)
↓
2. Breakpoint alcanzado
↓
3. Inspeccionar VARIABLES
↓
4. Agregar WATCH si es necesario
↓
5. F10 (Step Over) para recorrer código
↓
6. F11 (Step Into) para ver dentro de una función
↓
7. Debug Console para evaluar expresiones
↓
8. F5 (Resume) para continuar hasta el siguiente BP
↓
9. Repetir hasta encontrar el bug
↓
10. Fix + Hot Restart

Resumen

- **Atajos clave:** `F5` (start/continue), `Ctrl+F5` (sin debug), `Shift+F5` (stop), `F9` (toggle BP), `F10` (step over), `F11` (step into), `Ctrl+Shift+F5` (hot restart).
 - **Templates:** `flutter`, `dart`, `flutter test` y `Flutter Attach` cubren el 90% de los casos.
 - **Debug Console:** evalúa expresiones y llama funciones en vivo; `Set Value` modifica variables en caliente.
 - **Hot Reload** preserva estado; **Hot Restart** lo resetea (widgets `StatefulWidget` e instancias de repositorios).
 - **Errores típicos:** "Could not find a Flutter app to attach to" (codebase incorrecto), "Variables show " (modo profile, no debug).
-



Referencias

- [VS Code | Default keyboard shortcuts](#) — Atajos por defecto de VS Code
- [VS Code | Debugging](#) — Conceptos del debugger
- [Dart-Code | Debugging](#) — Detalles del debugger Dart/Flutter

- [Flutter | VS Code](#) — Guía oficial de VS Code para Flutter
-

 **Siguiente:** [07-practicas-vscode.md](#) — Prácticas y ejercicios de debugging con VS Code

07 — Prácticas y Ejercicios de Debugging en VS Code

Escenarios reales de debugging con BLoC/Cubit, Supabase, GoRouter y UI, más un ejercicio integrador de pagos y un checklist.

1. Escenario 1: Debugging de BLoC/Cubit con estado

1.1 Contexto

Tienes un `AuthCubit` que maneja login de usuarios. El estado no se actualiza correctamente después del login.

1.2 Setup del proyecto

`lib/features/auth/presentation/cubit/auth_cubit.dart`

```
import 'package:flutter_bloc/flutter_bloc.dart';

part 'auth_state.dart';

class AuthCubit extends Cubit<AuthState> {
  final LoginUseCase _loginUseCase;

  AuthCubit(this._loginUseCase) : super(AuthInitial());

  Future<void> login(String email, String password) async {
    emit(AuthLoading());
    try {
      final user = await _loginUseCase(email, password);
      emit(AuthSuccess(user)); // ← Breakpoint aquí
    } catch (e) {
      emit(AuthFailure(e.toString())); // ← Y aquí
    }
  }

  void logout() {
    emit(AuthInitial()); // ← Y aquí
  }
}
```

`lib/features/auth/presentation/cubit/auth_state.dart`


```

part of 'auth_cubit.dart';

sealed class AuthState extends Equatable {
  @override
  List<Object?> get props => [];
}

final class AuthInitial extends AuthState {}
final class AuthLoading extends AuthState {}

final class AuthSuccess extends AuthState {
  final dynamic user;
  AuthSuccess(this.user);
  @override
  List<Object?> get props => [user];
}

final class AuthFailure extends AuthState {
  final String message;
  AuthFailure(this.message);
  @override
  List<Object?> get props => [message];
}

```

1.3 Ejercicio paso a paso

Paso 1: Configurar launch.json

```

{
  "name": "Debug Auth Flow",
  "type": "dart",
  "request": "launch",
  "program": "lib/main.dart",
  "dartDefine": ["ENV=debug"]
}

```

Paso 2: Breakpoints condicionales

En `auth_cubit.dart`, sobre el `emit(AuthLoading())`:

- Click derecho → Add Conditional Breakpoint → Expression
- Expresión: `email.isEmpty || password.isEmpty`
- **Resultado:** solo pausa si intentan loguear sin credenciales

Paso 3: Agregar watches

En el panel WATCH, agregar:

```

email
password
state
state.runtimeType

```

Paso 4: Ejecutar y depurar

1. **F5** para iniciar
2. Intentar login con email vacío → debe pausar en el breakpoint condicional
3. **F10** (Step Over) para ver la transición de estados
4. En Debug Console: `state.props` para ver los datos del estado actual

Paso 5: Detectar el bug

Si el estado queda en `AuthLoading()` y nunca llega a `AuthSuccess`, usar:

- **F11** (Step Into) en `await _loginUseCase(email, password)` para entrar al use case
- Verificar si la excepción se está lanzando correctamente
- Verificar que `emit(AuthSuccess(user))` se ejecuta

2. Escenario 2: Debugging de llamada API con error inesperado

2.1 Contexto

Una llamada HTTP a Supabase retorna status 200 pero el body está vacío o en formato inesperado.

2.2 Setup del proyecto

`lib/features/products/data/datasources/product_remote_datasource.dart`

```
import 'dart:convert';
import 'package:http/http.dart' as http;

class ProductRemoteDatasource {
  final String baseUrl;
  final String apiKey;

  ProductRemoteDatasource({required this.baseUrl, required this.apiKey});

  Future<List<ProductModel>> getProducts() async {
    final response = await http.get(
      Uri.parse('$baseUrl/rest/v1/products'),
      headers: {
        'apikey': apiKey,
        'Authorization': 'Bearer $apiKey',
        'Content-Type': 'application/json',
      },
    );

    // ← Breakpoint aquí

    if (response.statusCode == 200) {
      final List<dynamic> jsonList = json.decode(response.body);
      return jsonList.map((json) => ProductModel.fromJson(json)).toList();
    } else {
      throw Exception('Failed to load products: ${response.statusCode}');
    }
  }
}
```

2.3 Ejercicio paso a paso

Paso 1: Agregar breakpoint condicional

En la línea `if (response.statusCode == 200) :`

- Click derecho → Add Conditional Breakpoint → Expression
- Expresión: `response.statusCode != 200 || response.body.isEmpty`
- **Resultado:** pausa si status no es 200 O si el body está vacío

Paso 2: Logpoint (alternativa)

- Click derecho → Add Logpoint
- Mensaje: `Status: {response.statusCode}, Body length: {response.body.length}`
- No pausa la ejecución pero logea en Debug Console

Paso 3: Ejecutar y diagnosticar

1. `F5` para iniciar
2. El logpoint muestra: `Status: 200, Body length: 0`
3. Agregar watch: `response.body`
4. En Debug Console:

```
response.body.isEmpty // true
response.headers      // ver content-type
```

Paso 4: El bug está en...

Response viene vacío → verificar:

- API key correcta
- RLS policies en Supabase (¡causa común!)
- Filtros en la query

Paso 5: Fix y verificación

```
// Agregar verificación explícita
if (response.statusCode == 200) {
    if (response.body.isEmpty) {
        throw Exception('Empty response - check RLS policies');
    }
    final List<dynamic> jsonList = json.decode(response.body);
    return jsonList.map((json) => ProductModel.fromJson(json)).toList();
}
```

3. Escenario 3: Debugging de GoRouter y navegación

3.1 Contexto

La navegación con GoRouter no funciona como se espera. El usuario hace click en un botón pero la ruta no cambia.

3.2 Setup del proyecto

`lib/core/router/app_router.dart`

```
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';

final goRouter = GoRouter(
  initialLocation: '/',
  routes: [
    GoRoute(
      path: '/',
      builder: (context, state) => const HomeScreen(),
    ),
    GoRoute(
      path: '/products/:id',
      builder: (context, state) {
        final productId = state.pathParameters['id']!;
        // ← Breakpoint aquí
        return ProductScreen(productId: productId);
      },
    ),
    GoRoute(
      path: '/products/:id/reviews',
      builder: (context, state) {
        final productId = state.pathParameters['id']!;
        final filter = state.uri.queryParameters['filter'] ?? 'all';
        // ← Breakpoint aquí
        return ReviewsScreen(productId: productId, filter: filter);
      },
    ),
  ],
);
```

3.3 Ejercicio paso a paso

Paso 1: Breakpoint condicional

En la ruta `/products/:id`:

- Click derecho → Add Conditional Breakpoint → Expression
- Expresión: `productId.isEmpty || productId == "null"`
- **Resultado:** pausa si el ID no se extrae correctamente

Paso 2: Usar Log Message para debug no intrusivo

```
// En el builder de las rutas (solo en debug):
if (kDebugMode) {
  debugPrint('[GoRouter] Navegando a: ${state.uri}');
  debugPrint('[GoRouter] Path params: ${state.pathParameters}');
  debugPrint('[GoRouter] Query params: ${state.queryParameters}');
}
```

Paso 3: Watches útiles

```
state.uri
state.pathParameters
state.queryParameters
state.matchedLocation
state.name
```

Paso 4: Diagnóstico típico

Si `productId` es `"null"` (string):

- El parámetro `:id` no se está pasando en la navegación
- Verificar `context.go('/products/${product.id}')` en el widget que navega
- Verificar que el `id` no sea null en el modelo

4. Escenario 4: Debugging de UI y layouts

4.1 Contexto

Un widget se renderiza con tamaños incorrectos. Necesitas ver las dimensiones exactas.

4.2 Setup

```
lib/features/home/presentation/widgets/product_card.dart
```

```

class ProductCard extends StatelessWidget {
  final Product product;

  const ProductCard({super.key, required this.product});

  @override
  Widget build(BuildContext context) {
    // ← Breakpoint aquí
    return Container(
      width: double.infinity,
      height: 120,
      padding: const EdgeInsets.all(16),
      child: Row(
        children: [
          Expanded(
            flex: 2,
            child: Image.network(product.imageUrl),
          ),
          Expanded(
            flex: 3,
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(product.name, style: Theme.of(context).textTheme.titleLarge),
                Text(product.description),
                Text('\${product.price}'),
              ],
            ),
          ),
        ],
      ),
    );
  }
}

```

4.3 Ejercicio paso a paso

Paso 1: Usar el Widget Inspector para entender el árbol

1. Poner breakpoint en `build`
2. `F5` hasta que pare
3. En VARIABLES, expandir `this` → ver propiedades del widget
4. Abrir DevTools → Widget Inspector para ver el árbol completo

Paso 2: Debug Console para inspeccionar

```

// En Debug Console durante el breakpoint
product.name           // ver nombre
product.imageUrl       // ver URL
product.price          // ver precio
product.description.length // ver longitud

```

Paso 3: Verificar constraints

Agregar temporalmente:

```
LayoutBuilder(  
  builder: (context, constraints) {  
    // ← Breakpoint aquí  
    debugPrint('Constraints: $constraints');  
    return Container(/* ... */);  
  },  
)
```

En Debug Console:

```
constraints.maxWidth    // ancho máximo disponible  
constraints.minWidth    // ancho mínimo  
constraints.maxHeight   // alto máximo
```

Paso 4: Usar Flutter DevTools Inspector

1. Abrir DevTools desde VS Code (`Ctrl+Shift+D` → DevTools)
2. Seleccionar el widget en el árbol
3. Ver propiedades de layout: width, height, constraints
4. Activar "Toggle Debug Paint" (Toggle Repaint Rainbow / Debug Paint) para ver borders

5. Escenario 5: Debugging de Supabase Realtime

5.1 Contexto

Las actualizaciones en tiempo real no llegan al widget.

5.2 Setup

```
lib/features/chat/data/datasources/chat_realtime_datasource.dart
```

```

import 'dart:async';
import 'package:supabase_flutter/supabase_flutter.dart';

class ChatRealtimeDatasource {
  final SupabaseClient _client;
  final _messagesController = StreamController<Map<String, dynamic>>.broadcast();

  Stream<Map<String, dynamic>> get messagesStream => _messagesController.stream;

  ChatRealtimeDatasource(this._client) {
    _subscribeToMessages();
  }

  void _subscribeToMessages() {
    _client
      .from('messages')
      .stream(primaryKey: ['id'])
      .order('created_at', ascending: false)
      .limit(50)
      .listen((data) {
        // ← Breakpoint aquí
        _messagesController.add(data);
      }, onError: (error) {
        // ← Breakpoint aquí también
        _messagesController.addError(error);
      });

    // ← Breakpoint aquí para verificar la suscripción
  }

  Future<void> sendMessage(String content) async {
    await _client.from('messages').insert({
      'content': content,
      'user_id': _client.auth.currentUser?.id,
      'created_at': DateTime.now().toIso8601String(),
    });
  }

  void dispose() {
    _client.channel('messages').unsubscribe();
    _messagesController.close();
  }
}

```

5.3 Ejercicio paso a paso

Paso 1: Verificar conexión

Poner breakpoint en el `.listen((data) {...})`:

- ¿Se ejecuta alguna vez? Si no → problema de suscripción
- ¿Se ejecuta con error? → problema de conexión/RLS

Paso 2: Watches para diagnóstico


```
data
data.length
data.isEmpty
_client.auth.currentUser?.id
```

Paso 3: Logs detallados

```
_client
  .from('messages')
  .stream(primaryKey: ['id'])
  .order('created_at', ascending: false)
  .limit(50)
  .listen((data) {
    debugPrint('=== REALTIME UPDATE ===');
    debugPrint('Records: ${data.length}');
    debugPrint('First record: ${data.firstOrNull}');
    debugPrint('Channel status: ${_client.channel('messages').status}');
    _messagesController.add(data);
  }, onError: (error) {
    debugPrint('=== REALTIME ERROR ===');
    debugPrint('Error: $error');
    debugPrint('Type: ${error.runtimeType}');
    _messagesController.addError(error);
  });
```

Paso 4: Diagnóstico común

1. ¿Las RLS políticas permiten SELECT? → `supabase.from('messages').select()`
2. ¿La tabla tiene replica identity? → `ALTER TABLE messages REPLICA IDENTITY FULL;`
3. ¿El channel se suscribió correctamente?
4. ¿Hay errores en Supabase Dashboard → Database → Replication?

6. Ejercicio Integrador: Debug completo de feature de pagos

6.1 Contexto

App Flutter con arquitectura limpia que integra pasarela de pagos. Los usuarios reportan que el pago "se queda cargando" sin completarse.

6.2 Archivos del ejercicio

```
lib/features/payments/presentation/bloc/payment_bloc.dart
```

```

import 'package:flutter_bloc/flutter_bloc.dart';

part 'payment_event.dart';
part 'payment_state.dart';

class PaymentBloc extends Bloc<PaymentEvent, PaymentState> {
  final ProcessPaymentUseCase _processPayment;
  final VerifyPaymentUseCase _verifyPayment;

  PaymentBloc(this._processPayment, this._verifyPayment)
    : super(PaymentInitial()) {
    on<ProcessPayment>(_onProcessPayment);
    on<VerifyPaymentStatus>(_onVerifyPayment);
  }

  Future<void> _onProcessPayment(
    ProcessPayment event,
    Emitter<PaymentState> emit,
  ) async {
    emit(PaymentProcessing());

    try {
      // ← Breakpoint 1: ¿Se llega aquí?
      final result = await _processPayment(PaymentParams(
        amount: event.amount,
        currency: event.currency,
        productId: event.productId,
      ));

      // ← Breakpoint 2: ¿Se obtiene resultado?

      if (result.requires3DS) {
        emit(Payment3DSRequired(result.paymentUrl));
        // ← Breakpoint 3: ¿Se emite este estado?
        return;
      }

      if (result.status == PaymentStatus.success) {
        emit(PaymentSuccess(result.transactionId));
      } else {
        emit(PaymentFailure(result.errorMessage ?? 'Unknown error'));
      }
    } catch (e, stackTrace) {
      // ← Breakpoint 4: ¿Se captura error?
      emit(PaymentFailure(e.toString()));
    }
  }

  Future<void> _onVerifyPayment(
    VerifyPaymentStatus event,
    Emitter<PaymentState> emit,
  ) async {
    emit(PaymentVerifying());
    try {
      final status = await _verifyPayment(event.transactionId);
      // ← Breakpoint 5
      if (status == PaymentStatus.success) {
        emit(PaymentSuccess(event.transactionId));
      } else if (status == PaymentStatus.pending) {
        // ← Breakpoint 6: ¿Se queda en loop aquí?
        emit(PaymentVerifying());
      } else {
        emit(PaymentFailure('Payment failed'));
      }
    }
  }
}

```

```

    }
  } catch (e) {
    emit(PaymentFailure(e.toString()));
  }
}
}

```

6.3 Ejercicio paso a paso

Paso 1: Mapear los breakpoints

Agregar breakpoints en los 6 puntos marcados con comentarios.

Paso 2: Configurar watches iniciales

```

event.amount
event.currency
event.productId
state
state.runtimeType

```

Paso 3: Ejecutar escenarios

Escenario A — Pago exitoso:

1. Iniciar debugging
2. Llenar formulario de pago
3. Verificar: ¿Pasa por Breakpoint 1? ¿2? ¿Resultado?
4. Si falla en 2 → el use case lanza excepción

Escenario B — Pago 3DS:

1. Usar tarjeta que requiere 3DS
2. Verificar: ¿Se emite `Payment3DSRequired`?
3. Después de 3DS → verificar `VerifyPaymentStatus`

Escenario C — Pago quedando en "cargando":

1. Usar tarjeta que causa timeout
2. Verificar: ¿Llega a Breakpoint 3 o 4?
3. Si se queda en `PaymentProcessing()` → problema en el use case

Paso 4: Diagnosticar causa raíz

Causa más común: el verify loop

```

// BUG: Si el verify retorna pending, se re-emite PaymentVerifying
// pero NO se re-intenta el verify → loop infinito de UI
if (status == PaymentStatus.pending) {
  emit(PaymentVerifying());
  // ← FALTA: await Future.delayed(...) y re-enviar el evento
}

```

Fix:

```
if (status == PaymentStatus.pending) {  
  emit(PaymentVerifying());  
  // Re-verificar después de 3 segundos  
  await Future.delayed(const Duration(seconds: 3));  
  if (!isClosed) {  
    add(VerifyPaymentStatus(event.transactionId));  
  }  
}
```

Paso 5: Verificar con Debug Console

```
// Durante el debugging, en Debug Console:  
state.runtimeType      // Ver estado actual  
event.amount           // Ver monto  
event.productId        // Ver producto  
result.transactionId   // Ver ID de transacción  
result.requires3DS      // Ver si requiere 3DS  
result.errorMessage    // Ver mensaje de error
```

7. Checklist de debugging

Antes de depurar cualquier bug:

- ☐ ¿Puedo reproducir el bug consistentemente?
- ☐ ¿Entiendo el flujo de datos? (BLoC → UseCase → DataSource)
- ☐ ¿Los breakpoints están en los puntos correctos del flujo?
- ☐ ¿Tengo watches configurados para variables clave?
- ☐ ¿Verifiqué los logs en Debug Console?
- ☐ ¿Revisé el estado del BLoC/Cubit?
- ☐ ¿Verifiqué la respuesta de la API (Network en DevTools)?
- ☐ ¿Revisé las RLS policies si uso Supabase?

Resumen

- Depurar con **breakpoints + watches** siguiendo el flujo completo: BLoC/Cubit → UseCase → DataSource → UI.
- Usar **DevTools (Network view)** para validar respuestas de Supabase y revisar las **RLS policies** cuando hay datos inesperados.
- En layouts, usar el **Inspector** para detectar sobreflujo (**RenderFlex overflowed**) y revisar constraints.
- Con **Realtime**, verificar suscripción, canal y políticas RLS antes de tocar la UI.

- El ejercicio integrador de **pagos** combina todos los escenarios; el **checklist** es la puerta de entrada antes de depurar cualquier bug.



Referencias

- [Flutter | DevTools](#) — Widget Inspector, Network view y más
- [Supabase | Realtime](#) — Postgres Changes y suscripciones
- [GoRouter | API](#) — Path y query parameters
- [flutter_bloc | API](#) — Bloc y Cubit

 **Siguiente:** [08-fundamentos-devtools.md](#) — Introducción a Flutter DevTools y sus vistas

08 — Fundamentos de Flutter DevTools

Qué es Flutter DevTools, cómo iniciarlo, cómo se conecta con tu app y qué vistas te ofrece para diagnosticar problemas.

1. ¿Qué es Flutter DevTools?

Es la suite de herramientas de diagnóstico para Flutter y Dart. No es un debugger tradicional sino un complemento para análisis **visual** de la app: árbol de widgets, métricas de rendimiento, memoria, red y más.

1.1 Vistas principales

Herramienta	Propósito
Flutter Inspector	Explorar el árbol de widgets, propiedades y layout
Performance	Medir FPS, frame rendering y jank
CPU Profiler	Analizar uso de CPU con flame chart
Memory	Monitorear uso de memoria, leaks y snapshots
Network	Inspeccionar requests HTTP y WebSocket
Debugger	Breakpoints y control de ejecución (alternativa a VS Code)
Logging	Ver logs de la app en tiempo real
App Size	Analizar el tamaño del bundle

Además, DevTools soporta **extensiones** de terceros (p. ej. Riverpod) que agregan vistas específicas según el paquete que uses.

2. Cómo iniciar DevTools

2.1 Desde VS Code

1. Durante debugging, abre la Command Palette (`Ctrl+Shift+P`)
2. Ejecuta `Dart: Open DevTools`

3. Selecciona la vista deseada

2.2 Desde la terminal

```
# Con la app corriendo en modo debug, desde la terminal de flutter run:
d

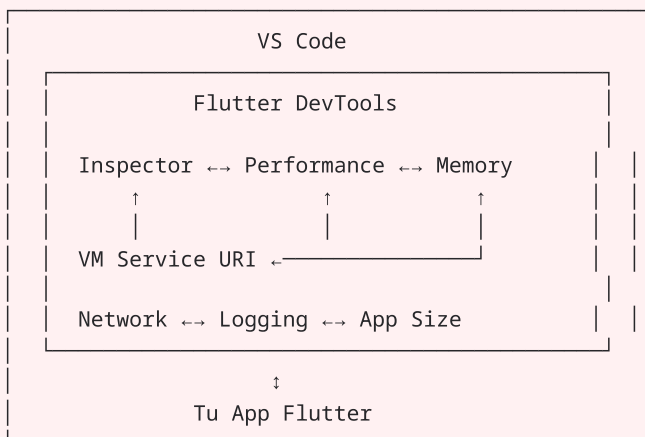
# Desde otra terminal, pasando la URI del VM Service (posicional):
dart devtools http://127.0.0.1:8181/XXXXXX/

# O simplemente abrir DevTools y conectar a la app descubierta:
dart devtools
```

2.3 Con la app corriendo

DevTools se conecta al **VM Service** que expone tu app en modo debug/profile. La forma más cómoda es presionar **d** en la terminal donde corre **flutter run**.

3. Conexión entre herramientas



4. Requisitos previos

4.1 Modo debug o profile obligatorio

```
# DevTools NO funciona en modo release
flutter run --debug

# O con profile para métricas de rendimiento reales
flutter run --profile
```

4.2 Versiones recomendadas

- Flutter SDK: 3.x reciente (DevTools viaja dentro del SDK)
- VS Code: versión estable actual
- Extensión Dart: versión estable actual

El "7 vistas de DevTools" clásico ya no aplica: algunas vistas se fusionaron (Timeline → Performance) y otras se volvieron extensiones (Provider → Riverpod/Bloc extensions).

5. Configuración de DevTools en VS Code

5.1 settings.json

```
{
  "dart.devToolsTheme": "dark",
  "dart.devToolsBrowser": "embedded"
}
```

- `dart.devToolsTheme` : "dark" o "light"
- `dart.devToolsBrowser` : "embedded" (panel de VS Code), "chrome" u otro navegador

5.2 Cómo abrir cada vista

Acción	Cómo
Abrir DevTools	<code>Ctrl+Shift+P</code> → <code>Dart: Open DevTools</code>
Panel Run and Debug	<code>Ctrl+Shift+D</code>
Abrir Inspector	DevTools → pestaña Flutter Inspector

6. Flujo típico de trabajo

1. flutter run --debug
↓
2. Abrir DevTools desde VS Code (o presionar "d")
↓
3. Seleccionar la vista según el problema:
 - ¿UI se ve mal? → Inspector
 - ¿App va lenta? → Performance
 - ¿Memory leak? → Memory
 - ¿API falla? → Network
 - ¿Logs no aparecen? → Logging
↓
4. Diagnóstico con datos visuales
↓
5. Aplicar fix en código
↓
6. Hot Reload → Verificar fix en DevTools
↓
7. Cerrar DevTools y continuar desarrollo

7. Cómo funciona internamente

DevTools se comunica con tu app a través del **VM Service** (expuesto sobre el Dart Development Service o DDS):

```
Tu App Flutter
  ↓
  VM Service (Dart VM)
    ↓
    WebSocket (ws://127.0.0.1:PORT/...)
      ↓
      DevTools (HTML/JS app)
        ↓
        VS Code (renderiza)
```

7.1 Información que expone el VM Service

- Árbol de widgets y sus propiedades
- Métricas de rendimiento (frame timing)
- Uso de memoria (heap, GC events)
- Tráfico de red (requests/responses desde Dart)
- Logs de stdout/stderr
- Estado de isolates

8. Limitaciones conocidas

Limitación	Causa
No funciona en release mode	DevTools requiere VM Service
Performance afectada en debug	El debug mode tiene overhead
Snapshots de memoria son aproximados	El debug mode usa más memoria
Network no ve requests de plataforma nativa	Solo capta requests hechos desde Dart
Algunos widgets no muestran propiedades	Limitación del framework

Resumen

1. DevTools es la suite visual de diagnóstico de Flutter
2. Se conecta al VM Service de la app en modo debug/profile
3. Se abre con `Dart: Open DevTools` o presionando `d` en `flutter run`
4. Cada problema tiene una vista: UI → Inspector, lentitud → Performance, memoria → Memory, red → Network
5. No funciona en modo release

Referencias

- [Flutter | DevTools](#) — Documentación oficial de DevTools
- [Flutter | DevTools CLI](#) — Cómo abrir DevTools desde la terminal
- [Flutter | DevTools VS Code](#) — Integración con VS Code

 **Siguiente:** [09-inspector-layout.md](#) — Flutter Inspector y debugging de layout

09 — Flutter Inspector y Debug de Layout

Explorar el árbol de widgets, entender constraints con el Layout Explorer y visualizar boundaries con Debug Paint.

1. Vista general del Flutter Inspector

1.1 Pestañas principales

Pestaña	Propósito
Widget Tree	Árbol jerárquico de widgets
Details Panel	Propiedades del widget seleccionado
Layout Explorer	Visualización de constraints y flex

2. Widget Tree

2.1 Navegación por el árbol

```
MaterialApp
├── Navigator
│   └── Scaffold
│       ├── AppBar
│       │   └── Text ("Mi App")
│       └── Body
│           └── Column
│               ├── Padding
│               │   └── Text ("Hello")
│               └── Expanded
│                   └── ListView
│                       └── ListTile
│                           └── Text ("Item 1")
```

2.2 Filtrar widgets

- 1. Usa la barra de búsqueda arriba del Widget Tree
- 2. Filtra por nombre de widget

3. `Ctrl+F` para buscar texto específico

2.3 Select widget

Haz click en cualquier widget del árbol para ver sus **propiedades** en el Details Panel y su layout en el Layout Explorer. También puedes usar el modo **Select Widget** para elegir el widget directamente en la app.

3. Layout Explorer — Entender constraints

3.1 ¿Qué muestra?

Visualiza los **constraints** (tamaños que el padre impone) y el **size** (tamaño real del widget).

3.2 Ejemplo visual: Column con hijos

Parent constraints: $0 \leq \text{width} \leq 411$, $0 \leq \text{height} \leq \infty$

Column {

Padding (0, 0, 0, 8)

Text "Title"

Expanded (flex: 1)

ListView

ListTile

ListTile

← Height: 48

← Height: ∞ (flex)

}

3.3 Herramientas del Layout Explorer

Herramienta	Descripción
Select Widget	Elegir un widget en el árbol o en la app
Toggle Debug Paint	Muestra borders y constraints
Refresh Tree	Actualiza después de hot reload

4. Debug Paint — Visualización de boundaries

4.1 Qué muestra cada color

Color	Significado
 Azul	Bounds del widget
 Verde	Overflow indicator
 Amarillo	Padding/margin
 Rojo	Error de layout

4.2 Activar Debug Paint

Desde DevTools:

1. Inspector → click en **Toggle Debug Paint** (icono 🎨)
2. Navegar por la app → ver boundaries en tiempo real
3. Desactivar al terminar

También puedes activarlo en código con `debugPaintSizeEnabled` (desde `package:flutter/rendering.dart`), pero desde DevTools no requieres tocar el código.

5. Propiedades de widgets comunes

5.1 Container

```
Container {  
  width: 300.0,  
  height: 200.0,  
  padding: EdgeInsets(16.0),  
  margin: EdgeInsets(8.0),  
  decoration: BoxDecoration {  
    color: Colors.blue,  
    borderRadius: BorderRadius.circular(8.0)  
  }  
}
```

5.2 Row/Column (Flex)

```
Row {  
  direction: horizontal,  
  mainAxisAlignment: start,  
  crossAxisAlignment: center,  
  mainAxisAlignment: max,  
  children: 3  
}
```

5.3 Expanded

```
Expanded {  
  flex: 2,  
  child: Text("Content")  
}
```

5.4 Padding

```
Padding {  
  padding: EdgeInsets.all(16.0),  
  child: Text("Padded content")  
}
```

6. Layout debugging paso a paso

6.1 Problema común: widget no aparece

Caso: un widget no se muestra en pantalla

Paso 1: Verificar en Inspector que el widget existe en el árbol

- Si no existe → problema de lógica (condicional, null check)

Paso 2: Verificar constraints

- `width: 0` o `height: 0` → el widget tiene tamaño cero
- Causa: padding incorrecto, o un padre con constraints restrictivos

Paso 3: Verificar overflow

- Layout Explorer muestra  verde → contenido desbordado
- Fix: usar `SingleChildScrollView`, `Wrap` o ajustar tamaños

6.2 Problema común: widget desbordado

Caso: layout overflow error

Paso 1: Activar Debug Paint en DevTools **Paso 2:** Identificar el widget que causa overflow **Paso 3:** Verificar si es un Flex (Row/Column) con hijos que exceden el espacio

Solución típica:

```
// Antes (overflow)
Row(
  children: [
    Text("Very long text that exceeds screen width"),
    Icon(Icons.star),
  ],
)

// Después (wrap)
Wrap(
  children: [
    Text("Very long text that exceeds screen width"),
    Icon(Icons.star),
  ],
)
```

7. Ejercicios prácticos

7.1 Ejercicio 1: Layout Explorer con ListView

1. Crear app con ListView que tiene 100 items
2. Abrir DevTools → Inspector
3. Seleccionar ListView en Widget Tree
4. En Layout Explorer ver:
 - Viewport height (cuánto espacio visible)
 - Content height (cuánto contenido total)
 - Scroll offset (dónde estamos scrolleando)
5. Scroll en la app → ver cómo cambia el offset en tiempo real

7.2 Ejercicio 2: Debug Paint con Stack

```
Stack(
  children: [
    Container(width: 200, height: 200, color: Colors.red),
    Positioned(
      top: 50,
      left: 50,
      child: Container(width: 100, height: 100, color: Colors.blue),
    ),
  ],
)
```

1. Activar Debug Paint

2. Ver cómo `Positioned` afecta el layout
3. Verificar que el Stack toma el tamaño del hijo más grande

7.3 Ejercicio 3: Performance Overlay

```
MaterialApp(  
  showPerformanceOverlay: true, // ← Activar  
  home: MyHome(),  
)
```

1. Ver el overlay con las dos gráficas:
 - **Shader compilation**: tiempo de compilar shaders
 - **Frame rendering**: tiempo de renderizar frames
2. Identificar frames que toman >16 ms (causan jank)
3. Usar la Performance view para más detalles

Resumen

Herramienta	Para qué
Widget Tree	Navegar la jerarquía de widgets
Details Panel	Ver propiedades del widget
Layout Explorer	Entender constraints y size
Debug Paint	Ver boundaries y overflows visualmente
Select Widget	Elegir widgets directamente en la app

Referencias

- [Flutter | Flutter Inspector](#) — Documentación oficial del Inspector
- [Flutter | Understanding constraints](#) — Cómo funcionan los constraints en Flutter
- [Flutter | Performance overlay](#) — Cómo leer el Performance Overlay

 **Siguiente:** 10-performance-view.md — La Performance view: FPS, frames y jank

10 — La Performance View: FPS, Frames y Jank

Entender cómo Flutter renderiza frames, leer el Frame Chart y el Frame Analysis, y diagnosticar el jank.

1. Frame rendering en Flutter

1.1 ¿Cómo funciona?

Flutter renderiza a 60 FPS (16.67 ms por frame) o 120 FPS (8.33 ms) en dispositivos de alta tasa de refresco.

Frame Timeline:
|---- 16ms ----|---- 16ms ----|---- 16ms ----|
Frame 1 Frame 2 Frame 3

1.2 Etapas de un frame

Build	Construir árbol de widgets (Dart)
Layout	Calcular positions y sizes
Paint	Dibujar pixels en pantalla
Composite	Combinar capas (GPU)

1.3 Frame budget

Dispositivo	Target FPS	Budget por frame
Mobile 60 Hz	60 FPS	16.67 ms
Mobile 120 Hz	120 FPS	8.33 ms
Desktop	60 FPS	16.67 ms
Web	Variable	Depende del browser

2. Performance View en DevTools

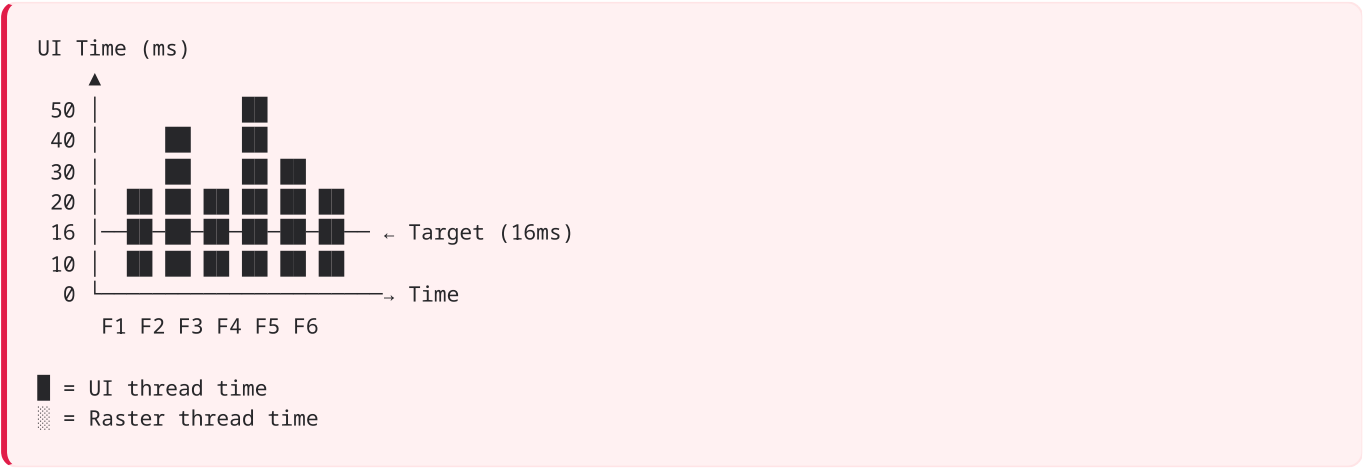
2.1 Pestañas disponibles

Pestaña	Propósito
Frame Chart	Gráfico de frames en tiempo real
Frame Analysis	Análisis detallado de frames seleccionados
Timeline	Grabación de timeline completa
Enhanced Tracing	Añadir tracing events específicos

La vista actual de DevTools fusiona lo que antes eran **Timeline** y **Performance** en una sola pantalla con pestañas. El **Performance Overlay** in-app (banda verde/roja arriba) es el primer filtro rápido.

3. Frame Chart

3.1 Qué muestra



3.2 Colores y su significado

Color	Significado
Azul claro	UI thread (build/layout/paint)
Azul oscuro	Raster thread (GPU rendering)
Rojo	Frame que supera el budget (jank)
Verde	Frame dentro del budget

3.3 Interacción con el gráfico

- 1. **Click en frame** → seleccionar para análisis detallado
- 2. **Drag** → hacer zoom en un rango de frames
- 3. **Double click** → resetear zoom
- 4. **Hover** → ver stats del frame (UI time, Raster time, Total)

4. Frame Analysis (detallado)

4.1 Seleccionar un frame

- 1. Click en un frame en el Frame Chart
- 2. Se abre el panel de análisis

4.2 Información mostrada

Frame #1234

└─ Build Phase

└─ Widget rebuilds: 47

└─ Stateful rebuilds: 12

└─ Time: 8.2ms

└─ Layout Phase

└─ Layout calls: 23

└─ Time: 2.1ms

└─ Paint Phase

└─ Layers painted: 15

└─ Time: 3.4ms

└─ Total: 13.7ms ✓

4.3 Métricas clave

Métrica	Valor ideal	Acción si no se cumple
Widget rebuilds	< 100	Reducir usando <code>const</code>
Stateful rebuilds	< 20	Usar reconstrucción selectiva
Layout time	< 5 ms	Simplificar el widget tree
Paint time	< 5 ms	Reducir complejidad visual
Total frame time	< 16 ms	Investigar el bottleneck

5. Identificando Jank

5.1 Causas comunes de jank

Causa	Síntoma en Performance	Solución
Build excesivo	UI time alto	Widgets <code>const</code> , <code>RepaintBoundary</code>
Layout complejo	Layout time alto	Simplificar el widget tree
Paint costoso	Paint time alto	Imágenes cacheadas, menos capas
Shaders	Picos aleatorios	Impeller / precaching
Isolate bloqueado	UI thread freeze	Mover a <code>compute</code> /isolate
GC pressure	Picos frecuentes	Reducir allocaciones

5.2 Ejemplo: build excesivo

Problema:

```
// × Mal: reconstruye en cada frame
class MyWidget extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Text('${DateTime.now()}'); // Se reconstruye constantemente
  }
}
```

Fix:

```
// ✅ Bien: usar Timer para actualizar solo el texto
class MyWidget extends StatefulWidget {
  @override
  State<MyWidget> createState() => _MyWidgetState();
}

class _MyWidgetState extends State<MyWidget> {
  late Timer _timer;
  String _time = '';

  @override
  void initState() {
    super.initState();
    _timer = Timer.periodic(Duration(seconds: 1), (_) {
      setState(() => _time = DateTime.now().toString());
    });
  }

  @override
  Widget build(BuildContext context) {
    return Text(_time); // Solo reconstruye este widget
  }

  @override
  void dispose() {
    _timer.cancel();
    super.dispose();
  }
}
```

6. Enhanced Tracing

6.1 Qué es

Añade eventos personalizados al timeline para debugging específico.

6.2 Tipos de tracing disponibles

Trace	Propósito
Widget Builds	Ver qué widgets se reconstruyen
Layout	Ver operaciones de layout
Paint	Ver operaciones de paint
Semantics	Ver cálculos de accesibilidad
Animations	Ver frames de animación

6.3 Cómo activar

1. Performance → Enhanced Tracing

2. Click en el trace que quieras activar
3. Grabar la profiling session
4. Ver eventos en el Timeline

6.4 Trazado desde código

```
import 'dart:developer';

// Marcar inicio de operación
Timeline.startSync('MyOperation');

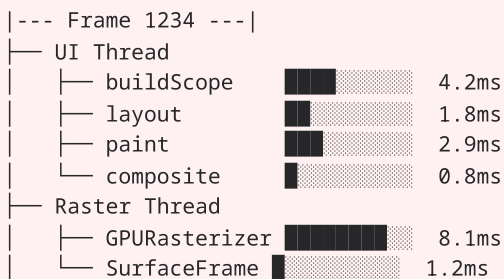
// ... código costoso ...

// Marcar fin
Timeline.finishSync();
```

7. Flame Chart

7.1 ¿Qué es?

Visualización de qué función/tarea está consumiendo tiempo en cada frame.



7.2 Interacción

- **Click** → seleccionar evento
- **Double click** → hacer zoom
- **Hover** → ver duración exacta
- **Scroll** → navegar entre frames

7.3 Eventos típicos en el Flame Chart

Evento	Thread	Qué significa
buildScope	UI	Construcción de widgets
layout	UI	Cálculo de posiciones

Evento	Thread	Qué significa
paint	UI	Dibujado
intrinsic	UI	Cálculo de tamaños intrínsecos
animate	UI	Animaciones en curso
GPURasterizer	Raster	Renderizado GPU
GrFragmentProcess	Raster	Procesamiento de fragmentos

8. Ejercicios prácticos

8.1 Ejercicio 1: ListView vs ListView.builder

```
// Versión 1: ListView (construye TODO)
ListView(
  children: List.generate(10000, (i) => ListTile(title: Text('Item $i'))),
)

// Versión 2: ListView.builder (construye solo lo visible)
ListView.builder(
  itemCount: 10000,
  itemBuilder: (context, i) => ListTile(title: Text('Item $i')),
)
```

1. Medir ambas versiones en Performance
2. Comparar frame times
3. Documentar la diferencia

8.2 Ejercicio 2: Impacto de RepaintBoundary

```
// Sin RepaintBoundary
Column(
  children: [
    AnimatedWidget(), // Se reconstruye frecuentemente
    StaticWidget(),   // No cambia pero se repinta igual
  ],
)

// Con RepaintBoundary
Column(
  children: [
    RepaintBoundary(
      child: AnimatedWidget(), // Solo repinta este subtree
    ),
    RepaintBoundary(
      child: StaticWidget(),   // No se repinta
    ),
  ],
)
```

1. Medir sin `RepaintBoundary`
2. Agregar `RepaintBoundary`
3. Comparar paint time

8.3 Ejercicio 3: Shader compilation jank

```
// Primera vez que se usa un efecto visual complejo
ClipRect(
  borderRadius: BorderRadius.circular(20),
  child: BackdropFilter(
    filter: ImageFilter.blur(sigmaX: 10, sigmaY: 10),
    child: Container(color: Colors.black26),
  ),
)
```

1. Ejecutar la app en profile mode
2. Navegar a la pantalla con el efecto blur
3. Observar el primer frame (shader compilation)
4. En Android/iOS con **Impeller** (default desde Flutter 3.10+) este jank se elimina casi por completo; en Web sigue siendo relevante precargar los shaders navegando por todas las pantallas antes del release.

Resumen

Concepto	Punto clave
Frame budget	16.67 ms @60 Hz, 8.33 ms @120 Hz
Frame Chart	Verde = bien, rojo = jank

Concepto	Punto clave
Frame Analysis	Desglosa build/layout/paint del frame
Jank	Causas: build, layout, paint, shaders, isolates, GC
Flame Chart	Muestra qué función consume cada frame

Referencias

- [Flutter | Performance view](#) — Documentación oficial de la Performance view
- [Flutter | Performance best practices](#) — Buenas prácticas de rendimiento
- [Flutter | Impeller](#) — Renderer por defecto y shader compilation

 **Siguiente:** 11-cpu-profiler.md — CPU Profiler: identificando funciones costosas

11 — CPU Profiler: Funciones Costosas

Grabar y leer un perfil de CPU para identificar qué funciones consumen más tiempo y causan lentitud o jank.

1. ¿Qué es el CPU Profiler?

Herramienta que muestra qué parte del código está consumiendo más tiempo de CPU. Permite identificar funciones costosas que causan jank o lentitud.

2. Pestañas del CPU Profiler

Pestaña	Propósito
Flame Chart	Call tree visual con tiempos
Call Tree	Árbol de llamadas con tiempos
Bottom Up	Funciones más costosas primero
Source Code	Código fuente con highlights

3. Grabar un perfil de CPU

3.1 Pasos

1. Ir a CPU Profiler en DevTools
2. Click en **Record** (●)
3. Interactuar con la app (reproducir el problema)
4. Click en **Stop** para detener la grabación
5. Analizar resultados

3.2 Configuración de grabación

Opción	Descripción
Include dartsdk	Incluir frames internos del Dart SDK
Include Flutter	Incluir frames del framework Flutter
Sample rate	Frecuencia de muestreo por defecto del VM

El profiler usa **sampling**: muestrea el stack periódicamente (p. ej. cada pocos ms) y estima el tiempo por función con esas muestras. No es una medición exacta de nanosegundos, sino una distribución aproximada.

4. Interpretando resultados

4.1 Profile View (resumen)

Total sample count: 1234

Top functions:

Function	Samples	% Total
buildScope	342	27.7%
paint	218	17.7%
layout	189	15.3%
_InkResponsePainter.paint	156	12.6%
TextPainter.layout	134	10.9%
ImageCache.putIfAbsent	98	7.9%
Other	97	7.9%

4.2 Call Tree (árbol de llamadas)

```
root (1234 samples)
├── WidgetBase.build (342)
│   ├── Column.build (120)
│   │   ├── Text.build (45)
│   │   ├── Icon.build (30)
│   │   └── Container.build (45)
│   └── ListView.builder.build (222)
│       └── _ListViewBuilderState.build (222)
├── RenderObject.layout (189)
│   ├── RenderFlex.performLayout (98)
│   └── RenderParagraph.performLayout (91)
└── Canvas.drawParagraph (218)
    └── [native] (218)
```

4.3 Bottom Up (funciones costosas primero)

Function	Self	Children	Total
TextPainter.layout	134	0	134
ImageCache.putIfAbsent	98	0	98
RenderFlex.performLayout	56	42	98
_ListViewBuilderState.build	45	177	222

- **Self time:** tiempo en la función misma (sin contar hijos)
- **Total time:** tiempo total (función + hijos)

5. Causas comunes de alto uso de CPU

5.1 Build methods costosos

Problema:

```
@override
Widget build(BuildContext context) {
  // × Cálculo costoso en cada build
  final items = List.generate(10000, (i) => computeExpensiveItem(i));
  return ListView.builder(
    itemCount: items.length,
    itemBuilder: (context, i) => items[i],
  );
}
```

El CPU Profiler muestra:

```
buildScope: 890 samples (72%)
└─ MyWidget.build: 890
   └─ List.generate: 650
      └─ computeExpensiveItem: 650
```

Fix:

```

class MyWidget extends StatefulWidget {
  @override
  State<MyWidget> createState() => _MyWidgetState();
}

class _MyWidgetState extends State<MyWidget> {
  late final List<Widget> _items;

  @override
  void initState() {
    super.initState();
    _items = List.generate(10000, (i) => computeExpensiveItem(i));
  }

  @override
  Widget build(BuildContext context) {
    return ListView.builder(
      itemCount: _items.length,
      itemBuilder: (context, i) => _items[i],
    );
  }
}

```

5.2 Imágenes sin cache

Problema:

```

// × Descarga la imagen en cada build
Image.network(
  'https://example.com/image.jpg',
  // Sin cache, se vuelve a descargar
)

```

Fix:

```

// ✅ Usa cache implícito de Flutter (ImageCache) y evita re-descargas
Image.network(
  'https://example.com/image.jpg',
  cacheWidth: 500, // Decodifica a menor resolución
)

```

Si necesitas control fino de cache, usa `cached_network_image`, que guarda también en disco.

5.3 Text rendering excesivo

El CPU Profiler muestra:

```

TextPainter.layout: 450 samples (36%)
├─ Text.computeMaxIntrinsicWidth: 200
└─ TextPainter: 250

```

Fix:

```
// × Textos largos sin límites
Text(
  'Very very very long text that needs to compute width...',
  style: TextStyle(fontSize: 16),
)

// ✔ Con límites explícitos
Text(
  'Very very very long text...',
  maxLines: 2,
  overflow: TextOverflow.ellipsis,
)

// ✔ 0 mejor aún: con constraints claros
ConstrainedBox(
  constraints: BoxConstraints(maxWidth: 300),
  child: Text('Long text...'),
)
```

5.4 Animaciones sin optimizar

Problema:

```
// × Animación que reconstruye todo el árbol
AnimatedBuilder(
  animation: _controller,
  builder: (context, child) {
    return Transform.rotate(
      angle: _controller.value * 2 * 3.14159,
      child: ExpensiveWidget(), // Se reconstruye cada frame
    );
  },
)
```

Fix:

```
// ✔ Solo animar lo que cambia
AnimatedBuilder(
  animation: _controller,
  builder: (context, child) {
    return Transform.rotate(
      angle: _controller.value * 2 * 3.14159,
      child: child, // ← Pasar child como parámetro
    );
  },
  child: ExpensiveWidget(), // No se reconstruye
)
```

6. Ejercicios prácticos

6.1 Ejercicio 1: identificar widget costoso

1. Crear lista con 1000 items
2. Cada item tiene imagen + texto + botón
3. Grabar CPU profile
4. Identificar qué parte consume más CPU
5. Optimizar y re-medir

6.2 Ejercicio 2: medir impacto de const

```
// Versión 1: Sin const
Column(
  children: [
    Text('Hello'), // Se reconstruye cada build
    Icon(Icons.star),
    Text('World'),
  ],
)

// Versión 2: Con const
Column(
  children: [
    const Text('Hello'), // Se reutiliza la instancia
    const Icon(Icons.star),
    const Text('World'),
  ],
)
```

1. Medir ambas versiones
2. Comparar samples de `buildScope`

6.3 Ejercicio 3: Flame Chart de animación

1. Crear animación compleja (múltiples animaciones simultáneas)
2. Grabar CPU profile durante 3 segundos de animación
3. Identificar cuánto tiempo consume cada animación
4. Verificar si alguna animación causa jank

Resumen

Concepto	Punto clave
Sampling	El profiler muestrea, no mide exacto
Call Tree	Muestra la jerarquía de llamadas

Concepto	Punto clave
Bottom Up	Funciones más costosas primero
Self vs Total	Self = solo la función; Total = función + hijos
Optimización típica	<code>const</code> , cache, límites en Text, <code>child</code> en AnimatedBuilder



Referencias

- Flutter | CPU Profiler — Documentación oficial del CPU Profiler
- Flutter | Performance best practices — Buenas prácticas de rendimiento
- Flutter | Const performance — Uso de `const` para rendimiento



Siguiente: 12-memory-profiler.md — Memory Profiler: leaks y uso de memoria

12 — Memory Profiler: Leaks y Uso de Memoria

Monitorear el uso de memoria en tiempo real, tomar heap snapshots y detectar memory leaks con diffs y tracing.

1. ¿Qué es el Memory Profiler?

Herramienta para monitorear el uso de memoria en tiempo real, identificar memory leaks y analizar heap dumps. Una gestión eficiente de memoria evita crashes por OOM (Out of Memory).

2. Conceptos básicos

2.1 Memoria en Dart/Flutter

Concepto	Descripción
Dart Heap	Memoria donde viven los objetos Dart
External	Memoria fuera del heap Dart (nativa, imágenes)
GPU	Memoria usada por la GPU para rendering
Total	Suma de todos los anteriores

2.2 Lifecycle de memoria

Allocation	→ Objeto creado, memoria asignada
Use	→ Objeto en uso activo
GC (Mark)	→ GC identifica objetos vivos
GC (Sweep)	→ GC libera objetos no referenciados

2.3 Tipos de memory leaks

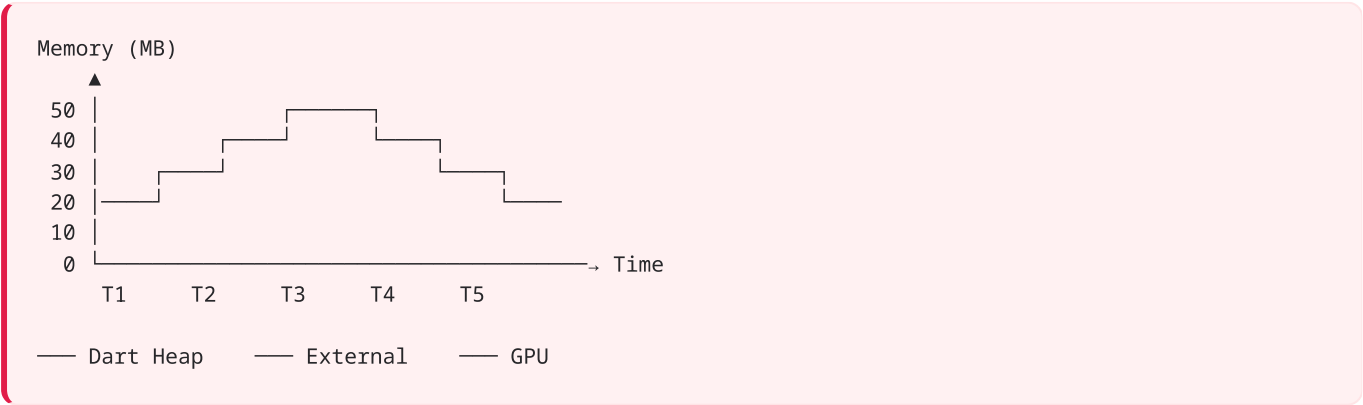
Tipo	Descripción	Ejemplo
Reference leak	Objeto referenciado innecesariamente	Controller sin dispose
Closure leak	Closure captura una variable que mantiene el objeto vivo	Timer sin cancel
Stream leak	Stream subscription sin cancel	<code>StreamSubscription</code> sin dispose
Image leak	Imágenes cargadas sin liberar	<code>ImageProvider</code> sin eviction

3. Memory View — Pestañas

Pestaña	Propósito
Chart	Gráfico de memoria en tiempo real
Snapshot	Heap snapshot para análisis
Diff	Comparar dos snapshots
Trace	Tracking de allocaciones específicas

4. Memory Chart

4.1 Qué muestra



4.2 Métricas en tiempo real

Métrica	Descripción
Dart Heap	Tamaño actual del heap Dart
Dart Heap Limit	Límite máximo del heap

Métrica	Descripción
External	Memoria nativa (imágenes, etc.)
GPU	Memoria de rendering
Allocation Rate	Velocidad de nuevas asignaciones
GC Frequency	Frecuencia de garbage collection

4.3 Interpretación del gráfico

Patrón	Significado
Subida gradual constante	Memory leak probable
Picos seguidos de bajada	GC funcionando normal
Plateau sin bajada	Memoria retenida sin liberar
Subida exponencial	Asignación excesiva, crítico

5. Heap Snapshots

5.1 ¿Qué es un heap snapshot?

Captura del estado completo del heap en un momento dado. Contiene todos los objetos vivos y sus referencias.

5.2 Cómo tomar un snapshot

1. Memory View → click en **Take Snapshot** 📷
2. Esperar a que se procese
3. Analizar resultados

5.3 Contenido del snapshot

Heap Snapshot: 12,345 objects

Retained Size: 2.3 MB

Classes:

Class	Instances	Retained	Shallow
_ListNode<int>	5,432	432 KB	216 KB
Picture	234	512 KB	117 KB
ImageStreamCompleter	89	340 KB	44 KB
_GradientTransform	156	187 KB	93 KB
Uint8List	67	156 KB	78 KB

5.4 Métricas clave

Métrica	Descripción
Retained Size	Memoria total retenida (objeto + todo lo que referencia)
Shallow Size	Memoria solo del objeto (sin referencias)
Instances	Número de instancias de esa clase

5.5 Tips para analizar

1. Ordenar por **Retained Size** para ver qué consume más
2. Expandir la clase para ver instancias individuales
3. Click en la instancia para ver referencias (qué la mantiene viva)
4. Buscar patrones: muchas instancias de una clase = posible leak

6. Memory Diff

6.1 ¿Qué es?

Compara dos snapshots para ver qué cambió: objetos nuevos, eliminados y cuánta memoria se incrementó/decrementó.

6.2 Cómo usar

1. Tomar snapshot A (baseline)
2. Interactuar con la app (navegar, abrir/cerrar pantallas)
3. Tomar snapshot B
4. Ver el diff: qué se agregó, qué se eliminó

6.3 Resultado del diff

Comparison: Snapshot A → Snapshot B

Added:

Class	Instances	Size	
_Route	3	12 KB	← Nuevas rutas
ProductModel	47	89 KB	← Carga de datos
ImageStreamCompleter	12	45 KB	← Imágenes cargadas

Deleted:

Class	Instances	Size	
ProductModel	23	44 KB	← Viejos datos

Net Change: +123 KB

6.4 Detectando memory leaks con Diff

Escenario: navegar a una pantalla 10 veces

Snapshot A: Baseline

Snapshot B: Después de 10 navegaciones

Resultado:

- _Route: +10 instances ← ¡Leak! Las rutas no se eliminan
- ProductModel: +100 instances ← ¡Leak! Datos cacheados sin límite

Fix:

```
// Con GoRouter, asegurar que los controllers/scopes se limpien al salir
GoRoute(
  path: '/products/:id',
  builder: (context, state) {
    return BlocProvider(
      create: (context) => getIt<ProductBloc>(),
      child: ProductScreen(), // Sus recursos se dispoen al salir
    );
  },
)
```

7. Trace (Allocation Tracing)

7.1 ¿Qué es?

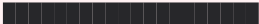
Tracking en tiempo real de cuándo se crean y destruyen objetos de una clase específica.

7.2 Cómo usar

1. Click en la pestaña **Trace**
2. Agregar la clase a monitorear (ej: `ProductModel`)
3. Interactuar con la app
4. Ver el gráfico de allocaciones vs deallocations

7.3 Resultado

ProductModel Allocation Trace

Allocations:  200
Deallocations:  160
Live:  40 ← ¡Posible leak!

Timeline:

T1: +10 allocated
T2: +5 allocated, -3 deallocated
T3: +15 allocated
T4: +8 allocated, -12 deallocated
...

7.4 Interpretación

- **Allocations > Deallocations constantemente** → Memory leak
 - **Allocations ≈ Deallocations** → Memoria sana
 - **Picos de allocations** → Momentos de alta carga (carga de lista, imágenes)
-

8. Problemas comunes de memoria en Flutter

8.1 Controllers sin dispose

```
// ✗ Memory leak
class MyScreen extends StatelessWidget {
  final _controller = TextEditingController(); // Nunca se dispone

  @override
  Widget build(BuildContext context) {
    return TextField(controller: _controller);
  }
}

// ✔ Correcto
class MyScreen extends StatefulWidget {
  @override
  State<MyScreen> createState() => _MyScreenState();
}

class _MyScreenState extends State<MyScreen> {
  late final TextEditingController _controller;

  @override
  void initState() {
    super.initState();
    _controller = TextEditingController();
  }

  @override
  void dispose() {
    _controller.dispose(); // ← Importante
    super.dispose();
  }
}
```

8.2 Streams sin cancel

```
// × Memory leak: subscription que nunca se cancela
class _MyScreenState extends State<MyScreen> {
  @override
  void initState() {
    super.initState();
    myStream().listen((event) {
      // Se ejecuta indefinidamente
    });
  }
}

// ✅ Correcto
class _MyScreenState extends State<MyScreen> {
  StreamSubscription? _subscription;

  @override
  void initState() {
    super.initState();
    _subscription = myStream().listen((event) {
      // Manejar el evento
    });
  }

  @override
  void dispose() {
    _subscription?.cancel(); // ← Importante
    super.dispose();
  }
}
```

8.3 Imágenes sin cache eviction

```
// × Acumula imágenes en memoria
ListView.builder(
  itemCount: 1000,
  itemBuilder: (context, i) {
    return Image.network('https://example.com/image_${i}.jpg');
    // Cada imagen se mantiene en memoria
  },
)

// ✅ Con límite de cache
// Usa el cacheWidth para decodificar más pequeño:
Image.network('https://example.com/image_${i}.jpg', cacheWidth: 300)

// O un cacheManager con límite de objetos y eviction:
CachedNetworkImage(
  imageUrl: 'https://example.com/image_${i}.jpg',
  cacheManager: CacheManager(
    Config(
      'uniqueCacheKey',
      stalePeriod: Duration(days: 7),
      maxNrOfCacheObjects: 100, // ← Límite + eviction automática
    ),
  ),
)
```


9. Ejercicios prácticos

9.1 Ejercicio 1: detectar memory leak

- 1. Crear pantalla con `TextEditingController`
- 2. Navegar a esa pantalla 10 veces
- 3. Tomar snapshot después de cada navegación
- 4. Verificar si las instancias de `TextEditingController` aumentan
- 5. Fix con `dispose()`

9.2 Ejercicio 2: memory diff de carga de datos

- 1. Tomar snapshot baseline
- 2. Cargar lista de 100 productos desde API
- 3. Tomar segundo snapshot
- 4. Analizar cuánta memoria consumen los productos
- 5. Evaluar si es aceptable

9.3 Ejercicio 3: trace de imágenes

- 1. Crear pantalla con 20 imágenes de red
- 2. Monitorear allocaciones de `ImageStreamCompleter`
- 3. Navegar fuera de la pantalla
- 4. Verificar si las imágenes se liberan
- 5. Implementar cache eviction si no se liberan

Resumen

Concepto	Punto clave
Dart Heap vs External	Objetos Dart vs memoria nativa (imágenes)
Snapshot	Estado completo del heap en un momento
Diff	Compara dos snapshots para detectar leaks
Trace	Sigue allocaciones/deallocations por clase
Fix típico	<code>dispose()</code> , <code>cancel()</code> , cache con límites

Referencias

- [Flutter | Memory view](#) — Documentación oficial de la Memory view
 - [Flutter | Memory, leaks y perfiles](#) — Conceptos de memoria en Flutter
 - [Flutter | Repainting y cachés de imágenes](#) — Cómo Flutter cachea imágenes
-

 **Siguiente:** 13-network-view.md — Network view: requests HTTP y WebSocket

13 — Network View: Requests HTTP y WebSocket

Inspeccionar todas las solicitudes HTTP y WebSocket de tu app: headers, bodies, tiempos y errores.

1. ¿Qué es la Network View?

Herramienta para inspeccionar todas las solicitudes HTTP y WebSocket realizadas por tu app Flutter. Permite ver headers, bodies, tiempos de respuesta y errores.

2. Network View — Pestañas

Pestaña	Propósito
HTTP Requests	Todas las solicitudes HTTP (GET, POST, etc.)
HTTP Events	WebSocket y Server-Sent Events
Filter	Filtrar por método, status, tipo

3. HTTP Requests

3.1 Lista de requests

Method	URL	Status	Size	Time
GET	/rest/v1/products	200	12 KB	145 ms
GET	/rest/v1/products?category=	200	4 KB	89 ms
POST	/auth/v1/token	200	0.5 KB	234 ms
GET	/storage/v1/object/public/	200	45 KB	567 ms
POST	/rest/v1/rpc/get_user_stats	200	1 KB	1234 ms
GET	/rest/v1/orders	401	0.1 KB	45 ms

3.2 Iconos de estado

Icono	Status	Significado
	200–299	Exitoso
	300–399	Redirect
	400–499	Error del cliente
	500–599	Error del servidor

4. Detalle de un request

4.1 Request Headers

```
GET /rest/v1/products HTTP/1.1
Host: xxx.supabase.co
apikey: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/json
User-Agent: Dart/3.2 (dart:io)
```

4.2 Response Headers

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Range: 0-49/150
X-Request-Id: abc123
Cache-Control: max-age=0
Content-Length: 12345
```

4.3 Response Body (pretty-printed)

```
[
  {
    "id": 1,
    "name": "Product 1",
    "price": 29.99,
    "category": "electronics",
    "created_at": "2024-01-15T10:30:00Z"
  },
  {
    "id": 2,
    "name": "Product 2",
    "price": 49.99,
    "category": "clothing",
    "created_at": "2024-01-15T11:45:00Z"
  }
]
```

5. Filtros

5.1 Filtros disponibles

Filtro	Descripción	Ejemplo
Method	Filtrar por método HTTP	method:GET , method:POST
Status	Filtrar por status code	status:200 , status:4xx
Type	Filtrar por tipo de recurso	type:document , type:script
URL	Buscar en la URL	products , auth
Content-Type	Filtrar por content type	content-type:json

5.2 Sintaxis de filtros

```
# Filtros básicos
method:GET
status:200
status:4xx

# Combinar filtros
method:POST status:200
method:GET type:document

# Excluir
method:OPTIONS
-status:200

# Buscar por texto
url:products
content-type:json
```

5.3 Ejemplos de uso

```
# Ver solo errores
status:4xx

# Ver solo llamadas a autenticación
url:auth

# Ver solo POST exitosos
method:POST status:200

# Ver requests de WebSocket
type:websocket
```

6. Análisis de tiempos

6.1 Timeline de un request

POST /auth/v1/token

DNS:		12ms
TCP:		15ms
TLS:		45ms
Request:		8ms
Response:		154ms
Total:		234ms

6.2 Métricas por request

Métrica	Descripción
DNS	Resolución del nombre de dominio
TCP	Conexión TCP
TLS	Handshake TLS/SSL
Request	Envío del request
Waiting	Esperando respuesta del servidor
Response	Descarga de la respuesta
Total	Tiempo total

6.3 Identificando cuellos de botella

Problema	Métrica afectada	Solución
DNS lento	DNS alto	Usar IP directa o DNS cache
Conexión lenta	TCP alto	Connection pooling
Handshake TLS	TLS alto	Reusar conexiones
Servidor lento	Waiting alto	Optimizar backend
Payload grande	Response alto	Paginación, compresión

7. HTTP Events (WebSocket)

7.1 Lista de eventos

Type	Event	Data
receive	postgres_changes	{...}
send	heartbeat	{}
receive	broadcast	{...}
error	connection_timeout	null

7.2 Detalle de evento WebSocket

WebSocket: Realtime Connection

Connected: true

Channel: public:messages

Topic: INSERT

Events:

Time	Payload
10:30:01	{"type": "INSERT", "record": {...}}
10:30:05	{"type": "UPDATE", "record": {...}}
10:30:12	{"type": "DELETE", "old_record": {...}}

8. Debugging con la Network View

8.1 Escenario 1: API retorna error 401

Observación en Network View:

POST /auth/v1/token → 401 Unauthorized

Response Body:

```
{
  "error": "invalid_grant",
  "message": "Invalid login credentials"
}
```

Diagnóstico:

1. Verificar el request body: ¿email/password correctos?
2. Verificar la API key en headers
3. Verificar si el usuario existe en Supabase

8.2 Escenario 2: request lento (>1 s)

Observación:

GET /rest/v1/orders?user_id=eq.123 → 3456ms

Diagnóstico:

1. Verificar el tamaño del response (payload)
2. Verificar si hay índices en la tabla
3. Verificar si RLS está causando overhead

4. Considerar paginación

8.3 Escenario 3: request falla intermitentemente

Observación:

```
GET /rest/v1/products → 200 (145ms)
GET /rest/v1/products → 503 (12ms)
GET /rest/v1/products → 200 (167ms)
```

Diagnóstico:

1. Verificar rate limiting de Supabase
2. Verificar la conexión del dispositivo
3. Verificar si el servidor está bajo carga

9. Network View vs Debug Console

Feature	Network View	Debug Console
Ver requests HTTP	✓	✗
Ver WebSocket events	✓	✗
Filtrar por status	✓	✗
Ver tiempos detallados	✓	✗
Ver headers completos	✓	✗
Evaluar expresiones	✗	✓
Imprimir valores	✗	✓
Breakpoints	✗	✓

Recomendación: usar Network View para debugging de API, Debug Console para debugging de lógica.

10. Ejercicios prácticos

10.1 Ejercicio 1: auditar llamadas de carga inicial

1. Iniciar la app con Network View abierto
2. Filtrar solo `type:document`
3. Identificar cuántas llamadas se hacen al iniciar
4. Optimizar: ¿se pueden reducir?

10.2 Ejercicio 2: debug de Supabase Realtime

1. Conectar a Supabase Realtime
2. Ver los eventos WebSocket en Network View
3. Modificar datos en otra pestaña
4. Verificar que los eventos llegan correctamente

10.3 Ejercicio 3: medir performance de la API

1. Hacer 10 llamadas a la misma API
2. Verificar los tiempos de respuesta
3. Identificar patrones (¿la primera es más lenta?)
4. Documentar los hallazgos

Resumen

Concepto	Punto clave
HTTP Requests	Método, URL, status, size, tiempo
Detalle	Headers + response body pretty-printed
Filtros	<code>method:</code> , <code>status:</code> , <code>url:</code> , <code>type:</code>
Timing	DNS/TCP/TLS/Request/Waiting/Response
WebSocket	Eventos Realtime de Supabase visibles

Referencias

- [Flutter | Network view](#) — Documentación oficial de la Network view
- [Supabase | Realtime](#) — Eventos WebSocket de Supabase
- [Dart | package:http](#) — Cliente HTTP estándar de Dart

 **Siguiente:** 14-debugger-view.md — Debugger view y control de ejecución

14 — Debugger View: Control de Ejecución

El debugger integrado en DevTools: breakpoints, variables, watch expressions y correlación con performance.

1. Debugger View en DevTools

El Debugger View es una alternativa al debugging de VS Code pero integrado en DevTools. Comparte el mismo VM Service, así que los breakpoints que pongas en VS Code se sincronizan automáticamente.

2. Pestañas del Debugger

Pestaña	Propósito
Scripts	Lista de scripts cargados
Variables	Variables del scope actual
Watch	Expresiones monitoreadas
Call Stack	Pila de llamadas
Breakpoints	Todos los breakpoints configurados

3. Diferencias entre el Debugger de VS Code y el de DevTools

Feature	VS Code	DevTools
Breakpoints de línea	✓	✓
Breakpoints condicionales	✓	✓
Logpoints	✓	✓
Watch expressions	✓	✓
Debug Console (REPL)	✓	✓
Hot reload integrado	✓	✓

Feature	VS Code	DevTools
Inspección de widgets	✗	✓
Correlación con performance	✗	✓

Recomendación: usar VS Code para el día a día y DevTools cuando necesites contexto visual o correlación con performance.

4. Cuándo usar el Debugger de DevTools

- 1. Cuando necesitas ver **contexto visual** (qué widgets están en pantalla)
- 2. Cuando quieres **correlacionar** breakpoints con performance
- 3. Para **debugging de layout** mientras inspeccionas código
- 4. Cuando el debugging remoto no pasa por VS Code

4.1 Cómo iniciar

- 1. Abrir DevTools
- 2. Ir a la pestaña **Debugger**
- 3. Los breakpoints de VS Code se sincronizan automáticamente (mismo VM Service)

5. Breakpoints en DevTools

5.1 Tipos soportados

Tipo	Soportado	Cómo
Línea	✓	Click en el gutter del código
Condicional	✓	Right-click → Expression
Hit count	✓	Right-click → Hit Count
Logpoint	✓	Right-click → Log Message

5.2 Gestión de breakpoints

```
Breakpoints Panel:  
├─ auth_cubit.dart:45 ✓  
├─ product_screen.dart:78 ✓  
├─ api_client.dart:123 ✗ (deshabilitado)  
└─ router.dart:15 ✓
```

- **Click** en el checkbox → habilitar/deshabilitar
 - **Right-click** → condición, hit count o eliminar
-

6. Variables en DevTools

6.1 Panel de Variables

```
Variables:  
├─ this: AuthCubit  
│  ├── _state: AuthSuccess {user: UserModel}  
│  ├── _loginUseCase: LoginUseCase  
│  └── isClosed: false  
├─ email: "user@example.com"  
├─ password: "....."  
└─ result: UserModel {id: 1, name: "John"}
```

6.2 Watch expressions

```
Watch:  
├─ state.runtimeType → "AuthSuccess"  
├─ state.props → [UserModel]  
├─ email.isEmpty → false  
├─ user.isValid() → true  
└─ products.length → 25
```

7. Call Stack

7.1 Estructura típica

```
Call Stack:  
├─ #0: AuthCubit.login (auth_cubit.dart:45)  
├─ #1: AuthBloc._onLogin (auth_bloc.dart:23)  
├─ #2: AuthBloc.onEvent (auth_bloc.dart:15)  
├─ #3: Bloc._onTransition (bloc.dart:456)  
└─ #4: Bloc._bindEventsToStates (bloc.dart:312)
```

7.2 Navegación

- **Click** en un frame → ver código y variables de ese frame
 - **Double-click** → saltar al archivo
-

8. Correlación con Performance

8.1 Flujo de trabajo

1. Performance View → identificar frame lento
2. Debugger → poner breakpoint en la función sospechosa
3. Ejecutar → llegar al breakpoint
4. Variables → ver el estado actual
5. Step through → identificar la línea problemática

8.2 Ejemplo

```
Performance:
  Frame #1234: UI time 45ms (JANK!)
  → Flame Chart muestra: _buildProductsList: 38ms

Debugger:
  Breakpoint en _buildProductsList
  Step through:
    Line 23: for (var product in products) ← Products tiene 10,000 items
    Line 24: widgets.add(ProductCard(product)) ← Se construye en cada iteración

Diagnóstico: construir 10,000 widgets en cada frame
Fix: usar ListView.builder para lazy loading
```

9. Scripts Panel

9.1 Lista de scripts cargados

```
Scripts:
├─ dart:core
├─ dart:async
├─ package:flutter/widgets.dart
├─ package:flutter/material.dart
├─ package:bloc/bloc.dart
├─ lib/main.dart
├─ lib/features/auth/presentation/cubit/auth_cubit.dart
├─ lib/features/auth/data/repositories/auth_repository.dart
└─ ...
```

9.2 Filtrar

- **Filter:** buscar por nombre de archivo
 - **Source:** filtrar por paquete o librería
-

10. Tips para usar el Debugger View

1. **Sincronizar con VS Code:** los breakpoints se comparten (mismo VM Service)
2. **Usar para layout debugging:** correlacionar código con el widget tree
3. **No usarlo para debugging extensivo:** VS Code es más rápido para el día a día
4. **Usar para presentaciones:** mostrar debugging en equipo con la vista web
5. **Hot reload preserva breakpoints:** si hot reload funciona, los breakpoints se mantienen

Resumen

Concepto	Punto clave
Debugger View	Debugger web integrado en DevTools
Sincronización	Comparte breakpoints con VS Code
Variables/Watch	Inspeccionar estado y expresiones
Call Stack	Navegar la pila de llamadas
Uso ideal	Correlación de código con performance/layout

Referencias

- [Flutter | Debugger view](#) — Documentación oficial del Debugger view
- [Dart | Observatory vs DevTools](#) — Visión general de DevTools

 **Siguiente:** 15-logging-view.md — Logging view y estrategias de logging

15 — Logging View y Estrategias de Logging

Ver logs de la app en tiempo real, filtrarlos y escribir logs útiles y estructurados.

1. ¿Qué es la Logging View?

Panel que muestra todos los logs de stdout/stderr de tu app Flutter. Útil para ver `print`, `debugPrint` y errores de consola sin usar la terminal.

2. Logging View — Pestañas

Pestaña	Propósito
Logs	Lista cronológica de todos los logs
Details	Detalle del log seleccionado
Filter	Filtrar por nivel o contenido

3. Lista de Logs

Time	Level	Message
10:30:01	INFO	App started
10:30:02	INFO	User authenticated: john@mail.com
10:30:05	WARNING	Cache miss for product:123
10:30:08	ERROR	Failed to load image: timeout
10:30:12	INFO	Navigated to /products
10:30:15	DEBUG	BLoC state: AuthSuccess
10:30:18	ERROR	API Error: 500 Internal Server

3.1 Niveles de log

Nivel	Color	Usado para
FINE	Gris claro	Verbose debugging
INFO	Azul	Información general
WARNING	Amarillo	Advertencias
SEVERE	Rojo	Errores críticos
SHOUT	Rojo oscuro	Errores fatales

La vista Logging interpreta los niveles del paquete `package:logging` de Dart (`LogRecord.level`).

4. Filtrar logs

4.1 Filtros disponibles

Filtro	Descripción
Nivel mínimo	Mostrar solo WARNING o superior, etc.
Texto	Buscar en el mensaje: <code>error</code> , <code>auth</code> , <code>product</code>
Origen	Filtrar por el logger/isolate de origen

4.2 Ejemplos de uso

```
# Buscar solo errores
error

# Buscar logs del flujo de auth
auth

# Buscar logs de un tag propio
[AUTH]

# Combinar: errores de API
error api
```

5. Detalle de log

```
Log Detail:
├─ Time: 10:30:08.123
├─ Level: SEVERE
├─ Logger: ImageLoader
├─ Message: Failed to load image: Connection timeout
├─ Stack Trace:
│   └─ #0 ImageLoader.load (image_loader.dart:45)
│   └─ #1 CachedNetworkImageProvider._loadAsync (cached_network_image.dart:234)
│   └─ #2 ImageStreamCompleter._handleImageLoad (image_stream.dart:156)
│   └─ ...
└─ Error: SocketException: Connection timed out
```

6. Logging desde código

6.1 `print()` y `debugPrint()`

```
// Básico
print('User logged in: $email');

// Con debugPrint (mejor para Flutter: hace throttling en modo debug)
debugPrint('Widget rebuild: ${runtimeType}');
debugPrint('State changed: $newState');
```

6.2 Logger package (recomendado)

```
import 'package:logger/logger.dart';

final logger = Logger(
  printer: PrettyPrinter(
    methodCount: 2,
    errorMethodCount: 5,
    lineLength: 50,
    colors: true,
    printEmojis: true,
  ),
);

// Uso
logger.d('Debug message'); // 🐛 Debug message
logger.i('Info message');  // 💡 Info message
logger.w('Warning message'); // ⚠️ Warning message
logger.e('Error message');  // ✖️ Error message
```

6.3 Logging estructurado con tags

```
// Con tags para filtrar
logger.d('[AUTH] User logged in: $email');
logger.d('[API] GET /products - Status: 200');
logger.d('[BLOC] State changed: AuthSuccess');
logger.e('[ERROR] Failed to load data: $error');

// En Logging View, filtrar por tag:
// message:AUTH
// message:API
// message:BLOC
```

6.4 Logging condicional

```
// Solo en debug mode
if (kDebugMode) {
  debugPrint('[DEBUG] Widget tree rebuilt');
  debugPrint('[DEBUG] API response: ${response.body}');
}

// Con nivel de logging configurable
enum LogLevel { debug, info, warning, error }

class AppConfig {
  static const LogLevel logLevel = kDebugMode
    ? LogLevel.debug
    : LogLevel.warning;
}

void log(String tag, String message, LogLevel level) {
  if (level.index >= AppConfig.logLevel.index) {
    debugPrint('[$tag] $message');
  }
}
```

7. Tips de logging

7.1 Usar tags consistentes

```
// Tags para cada feature
debugPrint('[AUTH] Login attempt');
debugPrint('[PRODUCTS] Loading products');
debugPrint('[CART] Item added');

// Tags para cada capa
debugPrint('[DATASOURCE] API call');
debugPrint('[REPOSITORY] Cache hit');
debugPrint('[BLOC] State emitted');
```

7.2 Loggear errores con contexto

```
// ✗ Mal
print('Error');

// ✔ Bien
print('Error loading products: $error');
print('Stack trace: $stackTrace');
print('User: ${currentUser?.id}');
print('URL: $apiUrl');
```

7.3 Loggear timing

```
final stopwatch = Stopwatch()..start();
final products = await repository.getProducts();
stopwatch.stop();
debugPrint('[PERF] getProducts: ${stopwatch.elapsedMilliseconds}ms');
```

7.4 Loggear el estado del BLoC

```
// En el BLoC
on<LoadProducts>((event, emit) async {
  debugPrint('[BLOC] LoadProducts event received');
  emit(ProductsLoading());
  try {
    final products = await getProducts();
    debugPrint('[BLOC] Products loaded: ${products.length} items');
    emit(ProductsLoaded(products));
  } catch (e) {
    debugPrint('[BLOC] Error loading products: $e');
    emit(ProductsError(e.toString()));
  }
});
```

8. Ejercicios prácticos

8.1 Ejercicio 1: logging de flujo completo

1. Implementar login completo con logging
2. Loggear cada paso: attempt, success, failure
3. Ver los logs en Logging View
4. Filtrar por tag `[AUTH]`

8.2 Ejercicio 2: logging de performance

1. Loggear el tiempo de cada llamada API

2. Loggear el tiempo de construcción de widgets
3. Identificar operaciones lentas vía logs

Resumen

Concepto	Punto clave
Logging View	stdout/stderr en tiempo real
Niveles	FINE, INFO, WARNING, SEVERE, SHOUT
Tags	<code>[AUTH]</code> , <code>[API]</code> , <code>[BLOC]</code> para filtrar
<code>debugPrint</code>	Throttling automático en debug
Buen logging	Contexto: error + stack + usuario + URL

Referencias

- [Flutter | Logging view](#) — Documentación oficial de la Logging view
- [pub.dev | logger](#) — Paquete de logging popular para Flutter
- [Dart | package:logging](#) — Logging estándar de Dart

 **Siguiente:** 16-app-size.md — App Size: analizando el tamaño del bundle

16 — App Size: Analizando el Tamaño del Bundle

Medir el tamaño de tu app por plataforma, cargar el reporte en DevTools y reducir los principales contribuyentes.

1. ¿Qué es App Size Analysis?

Herramienta para analizar el tamaño de tu app Flutter en diferentes plataformas. Ayuda a identificar qué paquetes, assets y código contribuyen al tamaño final.

2. ¿Por qué importa el tamaño?

Impacto	Descripción
Google Play	Límite de 200 MB por AAB
App Store	Binario hasta 2 GB; >200 MB no se descarga por datos móviles
Primera impresión	Los usuarios borran apps grandes
Velocidad de descarga	Apps grandes tardan más en descargar
Almacenamiento	Usuarios con poco espacio prefieren apps ligeras

3. Cómo generar el análisis de tamaño

3.1 Android (AAB)

```
# Build app bundle
flutter build appbundle --analyze-size

# El reporte se genera en:
# build/app/outputs/bundle/release/app-release.aab
# build/app/outputs/flutter-size/snapshot-size.json
```

3.2 iOS

```
# Build IPA
flutter build ipa --analyze-size

# El reporte se genera en:
# build/ios/ipa/Runner.ipa
# build/app/outputs/flutter-size/snapshot-size.json
```

3.3 Web / Desktop

```
# Build web
flutter build web --analyze-size

# Build desktop
flutter build linux --analyze-size
flutter build windows --analyze-size
flutter build macos --analyze-size
```

4. App Size View en DevTools

4.1 Cargar el reporte

1. DevTools → App Size
2. Click en **Load Report**
3. Seleccionar `snapshot-size.json`
4. Ver el análisis visual

4.2 Estructura del reporte

Total App Size: 18.5 MB

```
├─ Dart AOT (compiled code)
│   ├── lib (tu código): 3.2 MB (17.3%)
│   ├── packages (dependencias): 4.5 MB (24.3%)
│   │   ├── flutter: 1.8 MB
│   │   ├── supabase_flutter: 1.2 MB
│   │   ├── bloc: 0.3 MB
│   │   ├── go_router: 0.4 MB
│   │   ├── cached_network_image: 0.6 MB
│   │   └─ other packages: 0.2 MB
│   └─ dart SDK: 2.1 MB (11.4%)
├─ Assets
│   ├── Images: 5.2 MB (28.1%)
│   │   ├── logo.png: 1.2 MB
│   │   ├── splash.png: 0.8 MB
│   │   └─ product_images/: 3.2 MB
│   ├── Fonts: 1.8 MB (9.7%)
│   └─ Other assets: 0.5 MB
└─ Native libraries
    ├── libflutter.so: 1.2 MB (6.5%)
    └─ libapp.so: 0.0 MB
```

5. Análisis por categorías

5.1 Código Dart (AOT)

Qué incluye:

- Tu código (`lib/`)
- Dependencias (`packages/`)
- Dart SDK

Cómo optimizar:

```
# pubspec.yaml - Solo incluir lo que usas
dependencies:
  # ✗ No importar paquetes completos si no los usas
  # ✔ Usar packages ligeros y específicos
```

5.2 Assets (imágenes)

Qué incluye:

- Imágenes en `assets/`
- Imágenes de paquetes
- Iconos

Cómo optimizar:


```
# pubspec.yaml
flutter:
  assets:
    # ✗ No incluir directorios enteros
    # - assets/images/

    # ✔ Incluir solo las necesarias
    - assets/images/logo.png
    - assets/images/splash.png
```

```
// ✔ Usar imágenes de red en lugar de locales cuando no son críticas
Image.network('https://cdn.example.com/product.jpg')

// ✔ Para imágenes críticas, usar formatos modernos
// Convertir PNGs a WebP (mejor compresión)
// Reducir resolución al tamaño real necesario
```

5.3 Fonts (fuentes)

Qué incluye:

- Fuentes en `assets/fonts/`
- Fuentes de paquetes

Cómo optimizar:

```
# pubspec.yaml
flutter:
  fonts:
    # ✗ No incluir todas las variantes
    - family: Roboto
      fonts:
        - asset: assets/fonts/Roboto-Regular.ttf
        - asset: assets/fonts/Roboto-Medium.ttf
        - asset: assets/fonts/Roboto-Bold.ttf
        # 3 variantes × 200KB = 600KB

    # ✔ Solo incluir las que usas
    - family: Roboto
      fonts:
        - asset: assets/fonts/Roboto-Regular.ttf
        # 1 variante = 200KB
```

5.4 Native Libraries

Qué incluye:

- `libflutter.so` (Flutter engine)
- `libapp.so` (tu app compilada)
- Librerías nativas de plugins

Optimización:

- Poco que optimizar: es el runtime de Flutter

- Usar release mode para tamaño mínimo
- No usar plugins innecesarios

6. Estrategias de reducción de tamaño

6.1 Tree shaking (eliminación de código no usado)

Flutter elimina automáticamente el código no usado en release. Para maximizarlo:

```
// ✗ Importar todo el paquete
import 'package:flutter/material.dart';

// ✔ Importar solo lo necesario
import 'package:flutter/widgets.dart';
import 'package:flutter/painting.dart';
```

6.2 Deferred loading (carga diferida)

```
// Cargar librerías pesadas solo cuando se necesitan (soporte en web)
import 'package:heavy_feature/heavy_feature.dart' deferred as heavy;

ElevatedButton(
  onPressed: () async {
    await heavy.loadLibrary(); // Carga bajo demanda
    Navigator.push(context, MaterialPageRoute(
      builder: (_) => heavy.HeavyScreen(),
    ));
  },
  child: const Text('Open Heavy Feature'),
)
```

Los `deferred` imports solo aplican para web; en Android/iOS el código se compila completo (AOT).

6.3 Shrinking (Android)

```
// android/app/build.gradle
android {
  buildTypes {
    release {
      minifyEnabled true    // Shrink Java/Kotlin code
      shrinkResources true  // Shrink unused resources
      proguardFiles getDefaultProguardFile(
        'proguard-android-optimize.txt'
      ), 'proguard-rules.pro'
    }
  }
}
```

7. Comparar versiones

7.1 Antes vs Después

Version 1.0 (18.5 MB):

- └─ Dart AOT: 9.8 MB
- └─ Assets: 7.5 MB
- └─ Native: 1.2 MB

Version 1.1 (12.3 MB) - Optimizada:

- └─ Dart AOT: 6.2 MB (-3.6 MB)
 - └─ Eliminados paquetes no usados
- └─ Assets: 4.9 MB (-2.6 MB)
 - └─ Imágenes convertidas a WebP
 - └─ Fuentes reducidas
- └─ Native: 1.2 MB

Reducción: 33%

7.2 Cómo comparar

1. Generar `snapshot-size.json` de la versión A
 2. Generar `snapshot-size.json` de la versión B
 3. Cargar ambos en DevTools
 4. Comparar visualmente qué cambió
-

8. Monitoreo continuo

8.1 CI/CD pipeline

```
# .github/workflows/size-check.yml
name: App Size Check

on:
  pull_request:
    branches: [main]

jobs:
  size-check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: subosito/flutter-action@v2
      - name: Build and analyze size
        run: |
          flutter pub get
          flutter build appbundle --analyze-size
      - name: Check size limit
        run: |
          SIZE=$(stat -c%s build/app/outputs/flutter-size/snapshot-size.json)
          if [ $SIZE -gt 20000000 ]; then
            echo "⚠️ App size increased! Current: $SIZE bytes"
            exit 1
          fi
```

8.2 Métricas a monitorear

Métrica	Objetivo	Alerta si
Total size	< 15 MB	> 20 MB
Dart AOT	< 8 MB	> 10 MB
Assets	< 5 MB	> 8 MB
Imágenes	< 3 MB	> 5 MB
Paquetes	< 3 MB	> 5 MB

9. Ejercicios prácticos

9.1 Ejercicio 1: análisis de tu app

1. Build con `--analyze-size`
2. Cargar en DevTools
3. Identificar el top 3 de contribuyentes al tamaño

4. Proponer estrategias de reducción

9.2 Ejercicio 2: optimización de imágenes

1. Tomar todas las imágenes PNG del proyecto
2. Convertir a WebP con `cwebp`
3. Medir la reducción de tamaño
4. Reemplazar en el proyecto

9.3 Ejercicio 3: análisis de dependencias

1. Ejecutar `dart pub deps`
2. Identificar los paquetes más pesados
3. Evaluar si todos son necesarios
4. Eliminar los no esenciales

Resumen

Concepto	Punto clave
<code>--analyze-size</code>	Genera <code>snapshot-size.json</code>
Reporte	Código AOT, assets, fuentes, librerías nativas
Estrategias	Tree shaking, WebP, fuentes mínimas, plugins justos
Deferred	Solo web (no Android/iOS)
Monitoreo	CI con umbral de tamaño

Referencias

- [Flutter | App size view](#) — Documentación oficial de App Size
- [Flutter | Building your app](#) — Comandos de build por plataforma
- [Flutter | Optimizing performance](#) — Reducción de tamaño y rendimiento

 **Siguiente:** 17-cheatsheet-devtools.md — Cheatsheet de DevTools

17 — Cheatsheet de DevTools

Referencia rápida: herramientas, filtros, métricas, atajos y comandos de terminal para el día a día.

1. Herramientas de DevTools

Herramienta	Propósito
Inspector	Widget tree y layout
Performance	Frame timing y jank
CPU Profiler	Uso de CPU por función
Memory	Uso de memoria y leaks
Network	Requests HTTP y WebSocket
Debugger	Breakpoints y control de ejecución
Logging	Logs de stdout/stderr
App Size	Tamaño del bundle

2. Atajos y cómo abrir cada vista

Acción	Cómo
Abrir DevTools	<code>Ctrl+Shift+P</code> → <code>Dart: Open DevTools</code>
Abrir la vista deseada	<code>Dart: Open DevTools</code> → elegir pestaña
Panel Run and Debug	<code>Ctrl+Shift+D</code>
Buscar en panel actual	<code>Ctrl+F</code>
Grabar profiling session	Botón Record en CPU Profiler / Performance
Recargar vista	<code>Ctrl+R</code> en la ventana de DevTools

3. Sintaxis de filtros

3.1 Network View

Filtro	Ejemplo	Descripción
method:	method:GET	Filtrar por método HTTP
status:	status:200	Filtrar por status code
status:	status:4xx	Filtrar por rango
type:	type:document	Filtrar por tipo de recurso
url:	url:products	Buscar en la URL
content-type:	content-type:json	Filtrar por content type
-	-status:200	Excluir filtro

3.2 Logging View

Filtro	Ejemplo	Descripción
Nivel mínimo	WARNING	Solo warnings o superiores
Texto	auth	Buscar en el mensaje
Tag propio	[API]	Filtrar por tag

4. Métricas clave

4.1 Performance

Métrica	Valor ideal	Significado
Frame time	< 16 ms	Sin jank
UI time	< 10 ms	Build/layout/paint eficiente
Raster time	< 6 ms	GPU rendering eficiente
Shader compilation	< 5 ms	Sin stuttering inicial

4.2 Memory

Métrica	Valor ideal	Significado
Dart Heap	< 50 MB	Uso razonable
Allocation rate	Estable	Sin leaks
GC frequency	Baja	Sin pressure
External	< 30 MB	Memoria nativa controlada

4.3 CPU

Métrica	Valor ideal	Significado
buildScope	< 5% CPU	Builds eficientes
layout	< 3% CPU	Layouts simples
paint	< 3% CPU	Pintura eficiente
GC	< 2% CPU	Sin pressure de memoria

4.4 Network

Métrica	Valor ideal	Significado
Response time	< 500 ms	APIs rápidas
Payload size	< 100 KB	Respuestas ligeras
Error rate	< 1%	APIs estables

5. Iconos y colores

5.1 Performance View

Color	Significado
Azul claro	UI thread
Azul oscuro	Raster thread
Verde	Frame dentro de budget
Rojo	Frame jank (fuera de budget)

5.2 Memory View

Color	Significado
Azul	Dart Heap
Verde	External
Amarillo	GPU
Gris	Reserved

5.3 Network View

Color	Status
Verde	200-299 (Exitoso)
Amarillo	300-399 (Redirect)
Rojo	400-599 (Error)

6. Uso típico por escenario de debugging

Problema	Herramienta principal	Herramienta secundaria
UI no se ve bien	Inspector	Performance
App va lenta	Performance	CPU Profiler
Memory leak	Memory	CPU Profiler
API falla	Network	Logging
Crash	Logging	CPU Profiler
Tamaño grande	App Size	-
Layout overflow	Inspector	Performance
Animación jank	Performance	CPU Profiler

7. Plantillas de código

7.1 Logging estructurado

```
// Tag consistente por feature
debugPrint('[AUTH] Login attempt');
debugPrint('[PRODUCTS] Loaded ${products.length} items');
debugPrint('[CART] Added item: ${item.id}');
debugPrint('[PAYMENT] Processing: $amount $currency');
```

7.2 Performance tracking

```
final stopwatch = Stopwatch()..start();
// ... operación ...
stopwatch.stop();
debugPrint('[PERF] operation: ${stopwatch.elapsedMilliseconds}ms');
```

7.3 Memory tracking

```
debugPrint('[MEM] Before: ${ProcessInfo.currentRssBytes}');
// ... operación que puede hacer leak ...
debugPrint('[MEM] After: ${ProcessInfo.currentRssBytes}');
```

7.4 Error logging

```
try {
  // ... código ...
} catch (e, stackTrace) {
  debugPrint('[ERROR] $runtimeType: $e');
  debugPrint('[STACK] $stackTrace');
  rethrow;
}
```

8. Comandos útiles de terminal

```
# Build con análisis de tamaño
flutter build apk --analyze-size
flutter build appbundle --analyze-size
flutter build ios --analyze-size

# Run con profiling
flutter run --profile
flutter run --profile --trace-skia

# DevTools desde la terminal (URI posicional del VM Service)
dart devtools http://127.0.0.1:8181/XXXXXX/

# Ver logs de Android
adb logcat -s flutter

# Ver logs de iOS
xcrun simctl spawn booted log stream --level=debug
```

9. Solución de problemas comunes

Problema	Causa	Solución
DevTools no conecta	App en release mode	Usar <code>--debug</code> o <code>--profile</code>
Performance muestra 0 frames	No hay interacción	Interactuar con la app
Memory snapshot vacío	App recién iniciada	Esperar a que haya datos
Network no muestra requests	Requests nativos	Solo capta requests Dart
Logging no aparece	<code>print()</code> en release	Usar modo <code>--debug</code>

Resumen

1. DevTools se abre con `Dart: Open DevTools`
2. Cada vista tiene filtros: `method:`, `status:`, `url:`
3. Métricas de referencia: frame < 16 ms, heap < 50 MB
4. Colores: verde = bien, rojo = problema
5. `--analyze-size` para el reporte de tamaño

Referencias

- [Flutter | DevTools overview](#) — Visión general de DevTools
 - [Flutter | DevTools CLI](#) — Comandos de terminal
 - [Flutter | Performance debugging](#) — Guía de rendimiento
-

 **Siguiente:** [18-practicas-devtools.md](#) — Buenas prácticas con DevTools

18 — Prácticas y Ejercicios con DevTools

Cinco ejercicios guiados + un ejercicio integrador para practicar el debugging completo con DevTools.

1. Ejercicio 1: Inspector — Debug de Layout Overflow

1.1 Contexto

Un `Row` con tres `Expanded` containers causa overflow en pantallas pequeñas.

1.2 Setup del proyecto

```
lib/features/home/presentation/widgets/product_row.dart
```

```

class ProductRow extends StatelessWidget {
  final Product product;

  const ProductRow({super.key, required this.product});

  @override
  Widget build(BuildContext context) {
    return Row(
      children: [
        Expanded(
          flex: 2,
          child: Container(
            height: 80,
            color: Colors.blue.shade100,
            child: Image.network(product.imageUrl),
          ),
        ),
        Expanded(
          flex: 3,
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(product.name, style: TextStyle(fontWeight: FontWeight.bold)),
              Text(product.description, maxLines: 2, overflow: TextOverflow.ellipsis),
            ],
          ),
        ),
        Expanded(
          flex: 1,
          child: Column(
            children: [
              Text('\${product.price}'),
              IconButton(
                icon: Icon(Icons.add_shopping_cart),
                onPressed: () {},
              ),
            ],
          ),
        ),
      ],
    );
  }
}

```

1.3 Ejercicio paso a paso

Paso 1: identificar el overflow

1. Ejecutar la app en modo debug
2. Abrir DevTools → Inspector
3. Activar **Toggle Debug Paint**
4. Rotar el dispositivo o cambiar el tamaño de ventana
5. Observar el banner amarillo de overflow

Paso 2: analizar en Layout Explorer

1. Seleccionar el **Row** en el Widget Tree

2. Ver en Layout Explorer:

- Parent constraints: $0 \leq \text{width} \leq 411$
- Children flex: $2 + 3 + 1 = 6$
- Cada child intenta usar más espacio del disponible

Paso 3: diagnóstico

En la Debug Console, durante un breakpoint en el build:

```
MediaQuery.sizeOf(context).width // 411px
// Expanded calcula: 411 / 6 * flex
// Container 1: 411/6*2 = 137px ✓
// Column 1: 411/6*3 = 205.5px ✓
// Column 2: 411/6*1 = 68.5px ← ¡Muy poco para el contenido!
```

Paso 4: fix con Wrap

```
@override
Widget build(BuildContext context) {
  return Wrap(
    spacing: 8,
    runSpacing: 8,
    children: [
      SizedBox(
        width: 80,
        height: 80,
        child: Image.network(product.imageUrl),
      ),
      SizedBox(
        width: 200,
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(product.name, style: TextStyle(fontWeight: FontWeight.bold)),
            Text(product.description, maxLines: 2, overflow: TextOverflow.ellipsis),
          ],
        ),
      ),
      Column(
        children: [
          Text('\${product.price}'),
          IconButton(
            icon: Icon(Icons.add_shopping_cart),
            onPressed: () {},
          ),
        ],
      ),
    ],
  );
}
```

Paso 5: verificar el fix

1. Hot Restart
2. Verificar en Inspector que no hay overflow
3. Verificar en diferentes tamaños de pantalla

2. Ejercicio 2: Performance — Optimización de ListView

2.1 Contexto

Un `ListView` con 10,000 items causa jank al hacer scroll.

2.2 Setup

`lib/features/products/presentation/screens/products_list_screen.dart`

```
class ProductsListScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Products')),
      body: ListView(
        children: List.generate(
          10000,
          (i) => ListTile(
            leading: CircleAvatar(child: Text('${i + 1}')),
            title: Text('Product ${i + 1}'),
            subtitle: Text('Description for product ${i + 1}'),
            trailing: Icon(Icons.chevron_right),
          ),
        ),
      ),
    );
  }
}
```

2.3 Ejercicio paso a paso

Paso 1: medir Performance

1. Abrir DevTools → Performance
2. Empezar la grabación
3. Hacer scroll rápido por la lista
4. Detener la grabación
5. Analizar frames: ¿cuántos son > 16 ms?

Paso 2: identificar la causa

En Frame Analysis:

```
Frame #1234:
├─ Build Phase: 120ms ← ¡Muy alto!
│   └─ ListView.build: 118ms
│       └─ List.generate: 115ms
└─ Total: 125ms (JANK!)
```


Paso 3: fix con ListView.builder

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Products')),
    body: ListView.builder(
      itemCount: 10000,
      itemBuilder: (context, i) {
        return ListTile(
          leading: CircleAvatar(child: Text('${i + 1}')),
          title: Text('Product ${i + 1}'),
          subtitle: Text('Description for product ${i + 1}'),
          trailing: const Icon(Icons.chevron_right),
        );
      },
    ),
  );
}
```

Paso 4: re-medir Performance

- 1. Hot Restart
- 2. Grabar Performance de nuevo
- 3. Hacer scroll
- 4. Comparar: ¿mejoró el frame time?

Paso 5: documentar resultados

Métrica	Antes	Después
Promedio frame time	125 ms	12 ms
Max frame time	340 ms	18 ms
Frames > 16 ms	89%	2%
Construcción inicial	2300 ms	45 ms

3. Ejercicio 3: Memory — Detectar y arreglar un leak

3.1 Contexto

Navegar a una pantalla 10 veces incrementa la memoria en 50 MB sin liberarse.

3.2 Setup

```
lib/features/chat/presentation/screens/chat_screen.dart
```

```

class ChatScreen extends StatefulWidget {
  @override
  State<ChatScreen> createState() => _ChatScreenState();
}

class _ChatScreenState extends State<ChatScreen> {
  final List<StreamSubscription> _subscriptions = [];
  final List<AnimationController> _controllers = [];
  final List<Timer> _timers = [];
  final TextEditingController _controller = TextEditingController();

  @override
  void initState() {
    super.initState();

    // Suscripción a mensajes en tiempo real
    _subscriptions.add(
      Supabase.instance.client
        .from('messages')
        .stream(primaryKey: ['id'])
        .listen((data) {
          // Manejar mensajes
        }),
    );

    // Animación
    final controller = AnimationController(
      vsync: this,
      duration: const Duration(seconds: 2),
    )..repeat();
    _controllers.add(controller);

    // Timer para actualizar online status
    _timers.add(
      Timer.periodic(const Duration(seconds: 30), (_) {
        // Update online status
      }),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Chat')),
      body: TextField(controller: _controller),
    );
  }

  // x NO tiene dispose() - ¡MEMORY LEAK!
}

```

3.3 Ejercicio paso a paso

Paso 1: confirmar el leak con Memory

1. Abrir DevTools → Memory
2. Tomar snapshot baseline
3. Navegar a ChatScreen 10 veces
4. Tomar segundo snapshot

5. Comparar: `_ChatScreenState` tiene 10 instancias (debería ser 1)

Paso 2: identificar referencias

En el snapshot, expandir `_ChatScreenState` :

```
_ChatScreenState (10 instances)
├── _subscriptions: List<StreamSubscription> (10 items)
├── _controllers: List<AnimationController> (10 items)
├── _timers: List<Timer> (10 items)
└── _controller: TextEditingController (10 items)
```

Paso 3: fix — agregar dispose

```
@override
void dispose() {
  // Cancelar suscripciones
  for (final sub in _subscriptions) {
    sub.cancel();
  }

  // Cancelar timers
  for (final timer in _timers) {
    timer.cancel();
  }

  // Dispose controllers
  for (final controller in _controllers) {
    controller.dispose();
  }

  // Dispose text controller
  _controller.dispose();

  super.dispose();
}
```

Paso 4: verificar el fix

1. Hot Restart
2. Repetir el paso 1
3. Verificar que las instancias de `_ChatScreenState` se eliminan

Paso 5: documentar resultados

Métrica	Antes (sin dispose)	Después (con dispose)
Instancias después de 10 navegaciones	10	1
Memoria después de 10 navegaciones	+50 MB	+2 MB
Stream subscriptions	10 (leaked)	0 (disposed)
Animation controllers	10 (leaked)	0 (disposed)

4. Ejercicio 4: Network — Debug de API lenta

4.1 Contexto

La pantalla de perfil tarda 3 segundos en cargar. Necesitas identificar qué llamada es lenta.

4.2 Setup

```
lib/features/profile/presentation/screens/profile_screen.dart
```

```

class ProfileScreen extends StatefulWidget {
  @override
  State<ProfileScreen> createState() => _ProfileScreenState();
}

class _ProfileScreenState extends State<ProfileScreen> {
  UserModel? _user;
  List<OrderModel> _orders = [];
  Map<String, dynamic>? _stats;
  bool _isLoading = true;

  @override
  void initState() {
    super.initState();
    _loadData();
  }

  Future<void> _loadData() async {
    final user = await Supabase.instance.client.auth.currentUser;

    // Llamadas secuenciales (!INEFICIENTE!)
    final userData = await Supabase.instance.client
      .from('users')
      .select()
      .eq('id', user!.id)
      .single();

    final orders = await Supabase.instance.client
      .from('orders')
      .select()
      .eq('user_id', user.id)
      .order('created_at', ascending: false);

    final stats = await Supabase.instance.client
      .rpc('get_user_stats', params: {'user_id': user.id});

    setState(() {
      _user = UserModel.fromJson(userData);
      _orders = (orders as List).map((o) => OrderModel.fromJson(o)).toList();
      _stats = stats;
      _isLoading = false;
    });
  }

  @override
  Widget build(BuildContext context) {
    if (_isLoading) return const CircularProgressIndicator();
    // ... build UI
  }
}

```

4.3 Ejercicio paso a paso

Paso 1: medir con Network View

1. Abrir DevTools → Network
2. Navegar a ProfileScreen
3. Ver las 3 llamadas secuenciales:

```
GET /rest/v1/users?id=eq.123      → 800ms
GET /rest/v1/orders?user_id=eq.123 → 1200ms
POST /rest/v1/rpc/get_user_stats   → 900ms
Total secuencial: 2900ms
```

Paso 2: identificar el problema

Las llamadas son secuenciales. Se podrían hacer en paralelo.

Paso 3: fix con Future.wait

```
Future<void> _loadData() async {
  final user = await Supabase.instance.client.auth.currentUser;

  // Llamadas en paralelo (¡EFICIENTE!)
  final results = await Future.wait([
    Supabase.instance.client
      .from('users')
      .select()
      .eq('id', user!.id)
      .single(),
    Supabase.instance.client
      .from('orders')
      .select()
      .eq('user_id', user.id)
      .order('created_at', ascending: false),
    Supabase.instance.client
      .rpc('get_user_stats', params: {'user_id': user.id}),
  ]);

  setState(() {
    _user = UserModel.fromJson(results[0] as Map<String, dynamic>);
    _orders = (results[1] as List).map((o) => OrderModel.fromJson(o)).toList();
    _stats = results[2] as Map<String, dynamic>;
    _isLoading = false;
  });
}
```

Paso 4: re-medir

```
GET /rest/v1/users?id=eq.123      → 800ms ┐
GET /rest/v1/orders?user_id=eq.123 → 1200ms ┤ En paralelo
POST /rest/v1/rpc/get_user_stats   → 900ms ┘
Total paralelo: 1200ms (el más lento de los 3)
```

Paso 5: documentar

Métrica	Antes (secuencial)	Después (paralelo)
Total time	2900 ms	1200 ms
Mejora	-	59% más rápido
UX	Lento	Aceptable

5. Ejercicio 5: CPU Profiler — Optimización de widget

5.1 Contexto

Un widget de lista se reconstruye completamente cada segundo por un timer.

5.2 Setup

`lib/features/dashboard/presentation/widgets/live_stats_widget.dart`

```
class LiveStatsWidget extends StatefulWidget {
  @override
  State<LiveStatsWidget> createState() => _LiveStatsWidgetState();
}

class _LiveStatsWidgetState extends State<LiveStatsWidget> {
  int _activeUsers = 0;
  int _totalOrders = 0;
  double _revenue = 0;

  @override
  void initState() {
    super.initState();
    _startUpdates();
  }

  void _startUpdates() {
    Timer.periodic(const Duration(seconds: 1), (_) async {
      final stats = await Supabase.instance.client
        .rpc('get_live_stats');

      setState(() {
        _activeUsers = stats['active_users'];
        _totalOrders = stats['total_orders'];
        _revenue = stats['revenue'].toDouble();
      });
    });
  }

  @override
  Widget build(BuildContext context) {
    return Card(
      child: Column(
        children: [
          Text('Active Users: $_activeUsers'),
          Text('Total Orders: $_totalOrders'),
          Text('Revenue: \${_revenue.toStringAsFixed(2)}'),
          // Widget pesado que NO debería reconstruirse
          ExpensiveChart(data: [_activeUsers, _totalOrders, _revenue]),
        ],
      ),
    );
  }
}
```

5.3 Ejercicio paso a paso

Paso 1: medir con CPU Profiler

1. Abrir DevTools → CPU Profiler
2. Grabar 10 segundos
3. Ver el Flame Chart:

```
buildScope: 340 samples (34%)  
├─ _LiveStatsWidgetState.build: 280  
│   └─ ExpensiveChart.build: 260   ← ¡Cuello de botella!  
└─ Other widgets: 60
```

Paso 2: identificar el problema

`ExpensiveChart` se reconstruye cada segundo porque está dentro del `setState`.

Paso 3: fix con ValueNotifier + reconstrucción selectiva


```

class _LiveStatsWidgetState extends State<LiveStatsWidget> {
  int _activeUsers = 0;
  int _totalOrders = 0;
  double _revenue = 0;

  // Fuente de datos para el chart, actualizada por separado
  final ValueNotifier<List<double>> _chartData = ValueNotifier([]);

  @override
  void initState() {
    super.initState();
    _startUpdates();
  }

  void _startUpdates() {
    Timer.periodic(const Duration(seconds: 1), (_) async {
      final stats = await Supabase.instance.client.rpc('get_live_stats');

      final active = stats['active_users'];
      final total = stats['total_orders'];
      final revenue = stats['revenue'].toDouble();

      // El chart solo escucha el ValueNotifier
      _chartData.value = [active, total, revenue];

      if (mounted) {
        setState(() {
          _activeUsers = active;
          _totalOrders = total;
          _revenue = revenue;
        });
      }
    });
  }

  @override
  Widget build(BuildContext context) {
    return Card(
      child: Column(
        children: [
          Text('Active Users: $_activeUsers'),
          Text('Total Orders: $_totalOrders'),
          Text('Revenue: \${_revenue.toStringAsFixed(2)}'),
          // Solo se reconstruye cuando cambian los datos del chart
          ValueListenableBuilder<List<double>>(
            valueListenable: _chartData,
            builder: (context, data, child) {
              return ExpensiveChart(data: data);
            },
          ),
        ],
      ),
    );
  }
}

```

Paso 4: re-medir

```
buildScope: 85 samples (8.5%) ← ¡Reducción de 75%!
├─ _LiveStatsWidgetState.build: 60
│   └─ Text widgets: 30 (ligeros)
└─ ExpensiveChart: 25 (solo cuando cambian datos)
```

Paso 5: documentar

Métrica	Antes	Después
CPU samples (build)	340 (34%)	85 (8.5%)
ExpensiveChart rebuilds/s	1 (junto al resto)	1 (aislado)
Frame time promedio	28 ms	11 ms
Jank frames	45%	3%

6. Ejercicio Integrador: Debug completo de una feature

6.1 Contexto

App de e-commerce con una pantalla de checkout que tiene múltiples problemas:

1. Layout overflow en el botón de pago
2. Llamada API lenta
3. Memory leak en la validación de tarjeta
4. Jank al mostrar la animación de confirmación

6.2 Archivos del ejercicio

```
lib/features/checkout/presentation/screens/checkout_screen.dart
```

```

class CheckoutScreen extends StatefulWidget {
  @override
  State<CheckoutScreen> createState() => _CheckoutScreenState();
}

class _CheckoutScreenState extends State<CheckoutScreen> {
  final _cardController = TextEditingController();
  final _expiryController = TextEditingController();
  final _cvvController = TextEditingController();
  StreamSubscription? _validationSubscription;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Checkout')),
      body: Column(
        children: [
          // Resumen del pedido
          const OrderSummary(),

          // Formulario de pago (¡overflow aquí!)
          Row(
            children: [
              Expanded(child: TextField(controller: _cardController)),
              Expanded(child: TextField(controller: _expiryController)),
              Expanded(child: TextField(controller: _cvvController)),
            ],
          ),

          // Botón de pago
          ElevatedButton(
            onPressed: _processPayment,
            child: const Text('Pay \$99.99'),
          ),
        ],
      ),
    );
  }

  Future<void> _processPayment() async {
    // Llamada secuencial (¡lenta!)
    final token = await Supabase.instance.client.functions.invoke('create-payment');
    final result = await Supabase.instance.client.functions.invoke('process-payment');

    if (result.status == 200) {
      Navigator.push(context, MaterialPageRoute(
        builder: (_) => const PaymentSuccessScreen(),
      ));
    }
  }

  // x No tiene dispose() - ¡MEMORY LEAK!
}

```

lib/features/checkout/presentation/widgets/order_summary.dart

```
class OrderSummary extends StatelessWidget {
  const OrderSummary({super.key});

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.all(16),
      child: Column(
        children: [
          const Text('Order #1234'),
          const Text('Total: \$99.99'),
          const Text('Items: 3'),
          // Expensive widget que se reconstruye
          const LiveExchangeRate(),
        ],
      ),
    );
  }
}
```

lib/features/checkout/presentation/screens/payment_success_screen.dart

```

class PaymentSuccessScreen extends StatefulWidget {
  const PaymentSuccessScreen({super.key});

  @override
  State<PaymentSuccessScreen> createState() => _PaymentSuccessScreenState();
}

class _PaymentSuccessScreenState extends State<PaymentSuccessScreen>
  with SingleTickerProviderStateMixin {
  late AnimationController _controller;
  late Animation<double> _scaleAnimation;

  @override
  void initState() {
    super.initState();
    _controller = AnimationController(
      vsync: this,
      duration: const Duration(milliseconds: 600),
    );
    _scaleAnimation = Tween<double>(begin: 0, end: 1).animate(
      CurvedAnimation(parent: _controller, curve: Curves.elasticOut),
    );
    _controller.forward();
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return ScaleTransition(
      scale: _scaleAnimation,
      child: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Icon(Icons.check_circle, size: 100, color: Colors.green),
            const Text('Payment Successful!'),
          ],
        ),
      ),
    );
  }
}

```

6.3 Flujo de debugging paso a paso

Paso 1: diagnosticar overflow

1. Ejecutar la app
2. Ir a CheckoutScreen
3. Ver el overflow en el Row de text fields
4. **Fix:** usar Wrap o Column en lugar de Row

Paso 2: medir Performance

1. Abrir Performance
2. Grabar al hacer click en "Pay"
3. Identificar el frame jank
4. **Fix:** usar Future.wait para llamadas paralelas

Paso 3: detectar el memory leak

1. Abrir Memory
2. Tomar snapshot antes y después del checkout
3. Verificar las instancias de CheckoutScreen
4. **Fix:** agregar `dispose()` con cancelación de streams y controllers

Paso 4: optimizar la animación

1. Abrir CPU Profiler
2. Grabar durante la animación
3. Verificar si ExpensiveChart se reconstruye
4. **Fix:** aislar con RepaintBoundary / ValueListenableBuilder

Paso 5: verificar con Network

1. Abrir Network View
2. Verificar los tiempos de las llamadas API
3. Confirmar que las paralelas son más rápidas

6.4 Checklist final

Problema	Herramienta	Fix	Verificado
Layout overflow	Inspector	Wrap	☐
API lenta	Network	Future.wait	☐
Memory leak	Memory	dispose()	☐
Jank animation	CPU Profiler	RepaintBoundary	☐

Resumen

Ejercicio	Herramienta	Lección clave
Layout overflow	Inspector	Constraints y flex
ListView 10k items	Performance	<code>ListView.builder</code>
Leak en ChatScreen	Memory	<code>dispose()</code> completo
API lenta	Network	<code>Future.wait</code> en paralelo

Ejercicio	Herramienta	Lección clave
Rebuild cada segundo	CPU Profiler	<code>ValueListenableBuilder</code>

Referencias

- [Flutter | DevTools](#) — Documentación oficial de DevTools
- [Flutter | Performance best practices](#) — Buenas prácticas de rendimiento
- [Flutter | Memory](#) — Gestión de memoria en Flutter

 **Siguiente:** [19-fundamentos-rendimiento.md](#) — Fundamentos de rendimiento: frames, jank y pipelines

19 — Fundamentos de Rendimiento

Qué es el rendimiento en Flutter, por qué ocurre el jank y cómo medirlo correctamente (modo profile).

1. ¿Qué es el rendimiento en Flutter?

Es la capacidad de tu app de **responder a la interacción del usuario sin interrupciones visibles**. En Flutter, esto se mide fundamentalmente por la suavidad de las animaciones y la rapidez de respuesta a los eventos.

1.1 FPS (Frames Per Second)

Flutter dibuja la pantalla a **60 frames por segundo** en la mayoría de los dispositivos (120 en dispositivos ProMotion). Cada frame debe renderizarse dentro de un **frame budget**:

60 FPS → 16.67 ms por frame
120 FPS → 8.33 ms por frame

Si un frame toma más tiempo, el usuario percibe **jank** (tartamudeo visual).

1.2 Jank

El **jank** ocurre cuando Flutter no puede completar un frame dentro del budget:



Hay dos tipos de jank:

Tipo	Causa	Síntoma
UI Jank	Build/layout/paint lento en el frame principal	Frame rojo en DevTools
Raster Jank	Rasterizer (GPU) lento	Frame morado en DevTools

2. Métricas clave de rendimiento

Métrica	Qué mide	Rango saludable	DevTools View
FPS	Frames por segundo	≥ 55 FPS	Performance
Build time	Tiempo para construir el widget tree	< 8 ms	Performance > Enhanced Tracing
Raster time	Tiempo para rasterizar en GPU	< 8 ms	Performance > Enhanced Tracing
Frame budget	Tiempo máximo por frame	16.67 ms (60 Hz)	Performance Overlay
Memory usage	RAM utilizada por la app	Variable, sin crecimiento continuo	Memory
Shader compilation	Compilación de shaders en tiempo real	Sin spikes visibles	Performance
GC pauses	Pausas por garbage collection	< 5 ms, poco frecuentes	Memory

3. Build modes: debug vs profile vs release

Flutter tiene tres modos de compilación. **Nunca midas rendimiento en modo debug.**

Característica	Debug	Profile	Release
Compilador	Dart VM (JIT)	Dart AOT	Dart AOT
Optimizaciones del compilador	Ninguna	Mayoría	Todas
<code>assert()</code> habilitado	Sí	No	No
DevTools disponible	Sí	Sí	No
Performance Overlay	Disponible	Disponible	Disponible
asserts del framework	Sí	No	No
Velocidad real	~10-100x más lento	~Idéntica a release	Referencia
Para medir performance	NO	SÍ	SÍ (pero sin DevTools)
Para distribuir	No	No	SÍ

3.1 Cómo ejecutar en profile

```
# Ejecutar en modo profile
flutter run --profile

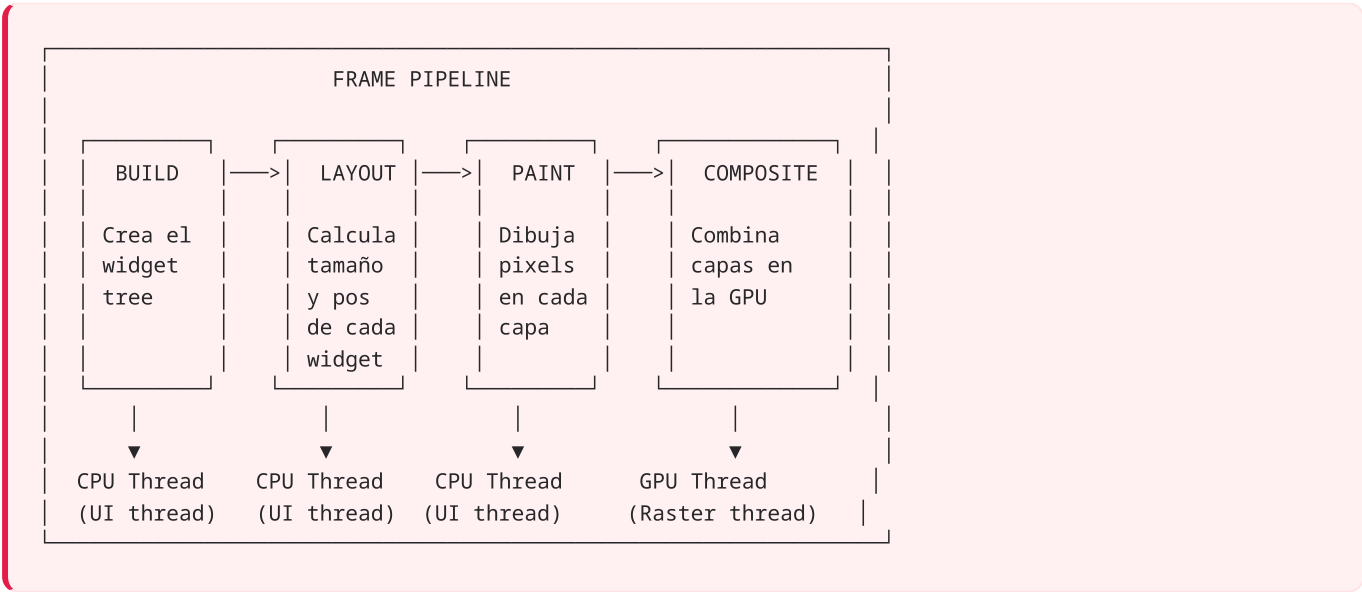
# Build de release para testear
flutter build apk --release

# Ver el tamaño del build
flutter build apk --analyze-size
```

Regla de oro: usa siempre `flutter run --profile` para mediciones. El modo debug agrega overhead del JIT compiler que distorsiona todos los resultados.

4. El pipeline de rendering de frames

Cada frame en Flutter pasa por este pipeline:



4.1 Qué afecta cada fase

Fase	Qué hace	Dónde se optimiza
Build	Llama a <code>build()</code> en cada widget	<code>const</code> , evitar <code>setState</code> innecesario
Layout	Calcula constraints y posición	Evitar widgets con constraints costosos
Paint	Dibuja en el canvas	<code>RepaintBoundary</code> , <code>shouldRepaint</code>
Composite	Une capas en la GPU	Menos capas = menos trabajo GPU

4.2 UI Thread vs Raster Thread

UI Thread

Framework (build, layout, paint)

Tu código Dart

BLoC/Cubit state changes

Raster Thread

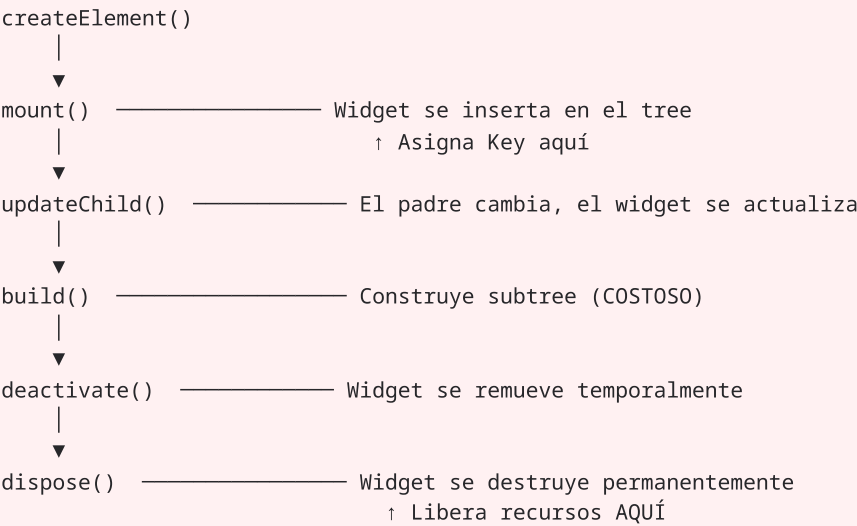
Skia/Impeller rendering

Shader compilation (puede causar jank!)

Texture uploads

5. El ciclo de vida del widget y su impacto

Cada widget tiene un ciclo de vida que afecta el rendimiento:



5.1 Impacto en rendimiento

Evento	Impacto	Cada vez que ocurre
<code>build()</code> completo	Alto	Cada rebuild del widget
<code>setState()</code>	Medio-Alto	Marca subtree para rebuild
<code>initState()</code>	Medio	Solo una vez por widget
<code>dispose()</code>	Bajo	Solo una vez (pero fallar aquí = leak)

6. Cuándo te importa el rendimiento

6.1 Advertencia: optimización prematura

"Premature optimization is the root of all evil" — Donald Knuth

No optimices hasta que tengas un problema medible. Usa esta guía:

Tu app tiene problemas de rendimiento?

- NO puedo hacer scroll fluido → Optimizar
- NO puedo cargar datos rápido → Optimizar
- La app se siente "ok" → NO optimizar aún
- No se sabe → MEDIR PRIMERO antes de tocar código

6.2 Señales de que necesitas optimizar

Síntoma	Probable causa	Urgencia
Scroll se congela momentáneamente	Build costoso en items visibles	Alta
Animaciones se ven "cortadas"	Frame budget excedido	Alta
Memoria crece sin parar	Memory leak	Alta
App tarda en iniciar	Trabajo pesado en <code>initState</code>	Media
Transiciones entre pantallas lentas	Demasiados widgets o rebuilds	Media
App consume mucha batería	Animaciones o streams innecesarios	Baja

7. Referencia rápida: métricas saludables

Métrica	Excelente	Aceptable	Necesita optimización
FPS	60 (120 en ProMotion)	55-59	< 55
Build time	< 4 ms	4-8 ms	> 8 ms
Raster time	< 4 ms	4-8 ms	> 8 ms
Memory (idle)	< 100 MB	100-200 MB	> 200 MB
Memory growth	0 MB/min	< 1 MB/min	> 1 MB/min
GC frequency	< 1/min	1-5/min	> 5/min
App startup	< 1 s	1-3 s	> 3 s

8. Comandos esenciales

```
# Ejecutar en profile para mediciones reales
flutter run --profile

# Alternar el Performance Overlay en la app
# (botón en la barra de DevTools o:
# Performance Overlay con M en la consola de flutter run)

# Analizar el tamaño del build
flutter build apk --analyze-size
flutter build ios --analyze-size

# Ver dependencias del proyecto
flutter pub deps
```

Resumen

Concepto	Clave
FPS	60 frames/segundo, 16.67 ms por frame
Jank	Frame que excede el budget (UI o Raster)
Build modes	Nunca medir en debug, siempre profile
Pipeline	Build → Layout → Paint → Composite
Optimización prematura	Mide primero, optimiza después
DevTools	Performance view para frames, Memory para memoria

Referencias

- Flutter | Performance best practices — Buenas prácticas de rendimiento
- Flutter | Profiling — Cómo medir el rendimiento de UI
- Flutter | Build modes — Diferencias entre debug, profile y release

 **Siguiente:** 20-optimizar-rebuilds.md — Optimizando rebuilds innecesarios

20 — Optimizando Rebuilds Innecesarios

`const`, Keys, reconstrucción selectiva y `RepaintBoundary` para evitar que tu widget tree se reconstruya sin necesidad.

1. Por qué los rebuilds innecesarios son el problema #1

En Flutter, **cada vez que un widget se reconstruye, todos sus hijos también se reconstruyen**. Si tu widget tree es profundo y un widget padre llama a `setState()` frecuentemente, puedes estar reconstruyendo cientos de widgets sin necesidad.

Widget Tree con rebuild innecesario:

```
HomePage (setState)
├── Header ────────── Reconstruido (innecesario)
├── Sidebar ───────── Reconstruido (innecesario)
├── Content
│   ├── ListWidget ── Reconstruido (innecesario)
│   │   ├── Item 1
│   │   ├── Item 2
│   │   └── Item 3
│   └── Footer ────── Reconstruido (innecesario)
└── BottomBar ────── Reconstruido (innecesario)
```

Solo ListWidget cambió, pero TODO se reconstruyó.

2. El keyword `const`: la optimización más simple

2.1 Qué hace `const`

Cuando marcas un widget como `const`, Flutter **reutiliza la misma instancia** en lugar de crear una nueva. No se ejecuta `build()` para ese widget ni sus hijos.

2.2 Sin const

```
@override
Widget build(BuildContext context) {
  return Column(
    children: [
      Text('Hola'),           // ← Nuevo widget cada rebuild
      Icon(Icons.star),       // ← Nuevo widget cada rebuild
      Text('Mundo'),          // ← Nuevo widget cada rebuild
      ElevatedButton(         // ← Nuevo widget cada rebuild
        onPressed: () {},
        child: Text('Click'),
      ),
    ],
  );
}
```

2.3 Con const

```
@override
Widget build(BuildContext context) {
  return Column(
    children: const [
      Text('Hola'),           // ← Reutilizado, sin rebuild
      Icon(Icons.star),       // ← Reutilizado, sin rebuild
      Text('Mundo'),          // ← Reutilizado, sin rebuild
      ElevatedButton(         // ← Reutilizado, sin rebuild
        onPressed: null,
        child: Text('Click'),
      ),
    ],
  );
}
```

2.4 Reglas de const

Regla	Ejemplo	Resultado
Todos los parámetros deben ser compile-time	<code>Text('literal')</code>	OK
Parámetros dinámicos	<code>Text(variable)</code>	NO puedes usar const
Sub-widgets const	<code>const [Text('a'), Text('b')]</code>	OK
Solo algunos hijos son const	Individuales: <code>const Text('a')</code>	OK
Widget con <code>key</code>	<code>const Text('a', key: ValueKey(1))</code>	OK

2.5 Cuándo NO usar const

```
// × NO puedes usar const aquí
Text(someVariable)
Image.network('$imageUrl')
Text('${user.name}')
```



```
// ✔ Usa const cuando todos los valores son literales
const Text('static text')
const Icon(Icons.home)
const SizedBox(height: 16)
const Padding(
  padding: EdgeInsets.all(8.0),
  child: Text('static'),
)
```

3. Keys: controlar el matching de widgets

Las **Keys** le dicen a Flutter cómo emparejar widgets entre rebuilds. Sin la key correcta, Flutter puede reutilizar el widget equivocado.

3.1 Tipos de Keys

Key	Qué hace	Cuándo usar
ValueKey	Identifica por un valor (String, int, etc.)	Items de lista con ID único
ObjectKey	Identifica por referencia a objeto	Objetos que pueden no ser <code>==</code>
UniqueKey	Identifica por identidad única	Forzar recreación completa
GlobalKey	Acceso al estado del widget desde cualquier lugar	Formularios, navegación

3.2 Ejemplo: ValueKey en listas

```
// × Sin key: Flutter puede mezclar items al reordenar
ListView(
  children: items.map((item) => ItemWidget(item)).toList(),
)
```



```
// ✔ Con ValueKey: Flutter sabe qué item es cada widget
ListView(
  children: items.map((item) => ItemWidget(
    key: ValueKey(item.id),
    item: item,
  )).toList(),
)
```


3.3 Cuándo usar cada Key

Necesitas Key?

- Lista de items con ID único → `ValueKey(item.id)`
- Lista de objetos sin ID estable → `UniqueKey()`
- Necesitas acceder al estado de un widget → `GlobalKey()`
- Widget se mueve entre padres → `GlobalKey()`
- No hay problemas de matching → Sin key (default)

4. BlocSelector vs BlocBuilder: reconstrucción selectiva

4.1 El problema con BlocBuilder

```
// × BlocBuilder reconstruye TODO cuando el estado cambia
BlocBuilder<AuthBloc, AuthState>(  
  builder: (context, state) {  
    return Column(  
      children: [  
        Text('User: ${state.user.name}'), // ← Solo esto cambia  
        Text('Email: ${state.user.email}'), // ← Pero todo se reconstruye  
        // ... 50 widgets más ...  
        BigExpensiveWidget(), // ← Incluido este  
      ],  
    );  
  },  
)
```

4.2 La solución: BlocSelector

```
// ✓ BlocSelector solo reconstruye cuando el valor seleccionado cambia
BlocSelector<AuthBloc, AuthState, String>(  
  selector: (state) => state.user.name, // ← Solo monitorea esto  
  builder: (context, name) {  
    return Text('User: $name'); // ← Solo este widget se rebuild  
  },  
)
```

4.3 Comparación BlocBuilder vs BlocSelector

Característica	BlocBuilder	BlocSelector
Reconstruye cuando...	Cualquier cambio de estado	Solo cuando el selector cambia
Granularidad	Todo el subtree	Widget específico
Uso de memoria	Más alto	Más bajo
Complejidad	Simple	Requiere definir selector

Característica	BlocBuilder	BlocSelector
Mejor para...	Widgets pequeños, estados simples	Estados grandes, widgets críticos

4.4 Ejemplo completo: Login form

```
// × BlocBuilder reconstruye todo el form en cada tecla
BlocBuilder<LoginBloc, LoginState>(
  builder: (context, state) {
    return Form(
      child: Column(
        children: [
          TextField(
            onChanged: (v) => context.read<LoginBloc>().add(
              EmailChanged(v),
            ),
          ),
          TextField(
            onChanged: (v) => context.read<LoginBloc>().add(
              PasswordChanged(v),
            ),
          ),
          // ... más campos
          ElevatedButton(
            onPressed: state.isValid ? () { /* login */ } : null,
            child: const Text('Entrar'),
          ),
        ],
      ),
    );
  },
)

// ✔ Cada TextField usa context.read y solo el botón escucha el estado
Column(
  children: [
    TextField(
      onChanged: (v) => context.read<LoginBloc>().add(EmailChanged(v)),
    ),
    TextField(
      onChanged: (v) => context.read<LoginBloc>().add(PasswordChanged(v)),
    ),
    // Solo el botón usa BlocSelector
    BlocSelector<LoginBloc, LoginState, bool>(
      selector: (state) => state.isValid,
      builder: (context, isValid) {
        return ElevatedButton(
          onPressed: isValid ? () { /* login */ } : null,
          child: const Text('Entrar'),
        );
      },
    ),
  ],
)
```

5. buildWhen en BlocBuilder

Si necesitas más control que `BlocSelector`, usa `buildWhen`:

```
BlocBuilder<LoginBloc, LoginState>(  
  // Solo reconstruye cuando isValid o isLoading cambian  
  buildWhen: (previous, current) {  
    return previous.isValid != current.isValid ||  
      previous.isLoading != current.isLoading;  
  },  
  builder: (context, state) {  
    return Column(  
      children: [  
        if (state.isLoading) const CircularProgressIndicator(),  
        ElevatedButton(  
          onPressed: state.isValid ? () {} : null,  
          child: const Text('Entrar'),  
        ),  
      ],  
    );  
  },  
)
```

5.1 buildWhen vs BlocSelector

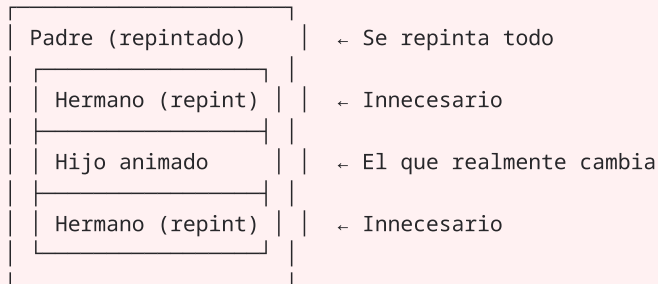
Cuándo usar	buildWhen	BlocSelector
Filtrar por múltiples campos del estado	✅ Mejor	Requiere combinar en un solo valor
Seleccionar un solo campo simple	Puede	✅ Mejor
Control granular con lógica compleja	✅ Mejor	Limitado
Código conciso y declarativo	Más verbose	✅ Más limpio

6. RepaintBoundary: limitar el área de repintado

Un `RepaintBoundary` le dice a Flutter: "si este widget no cambió, no repintes nada dentro de él".

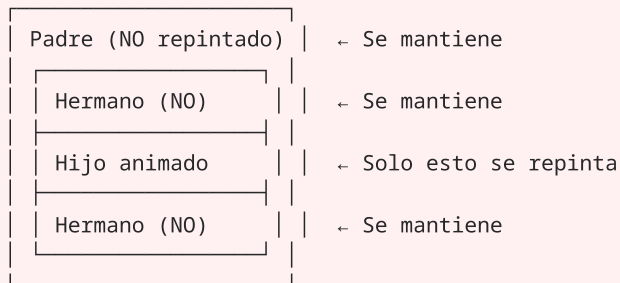
6.1 Sin RepaintBoundary

Animación en un widget hijo:



6.2 Con RepaintBoundary

Animación en un widget hijo:



6.3 Ejemplo

```
ListView.builder(  
  itemCount: items.length,  
  itemBuilder: (context, index) {  
    return RepaintBoundary( // ← Envuelve cada item  
      child: ItemWidget(  
        item: items[index],  
        onAnimate: () { /* animación */ },  
      ),  
    );  
  },  
);
```

6.4 Cuándo usar RepaintBoundary

Escenario	¿Usar?
Lista con items que tienen animaciones independientes	Sí
Widget con CustomPainter que se redibuja frecuentemente	Sí
Texto que cambia raramente dentro de un área animada	Sí
Widget simple sin animaciones	No necesario

Escenario	¿Usar?
Toda la pantalla como RepaintBoundary	Raramente útil

7. Anti-patrones de setState

7.1 Anti-patrón 1: setState en loop

```
// × MAL: setState se llama múltiples veces
for (var item in items) {
  setState(() {
    item.selected = !item.selected;
  });
}
// Resultado: N rebuilds, uno por cada item

// ✅ BIEN: Un solo setState para todos los cambios
setState(() {
  for (var item in items) {
    item.selected = !item.selected;
  }
});
// Resultado: 1 solo rebuild
```

7.2 Anti-patrón 2: setState en didChangeDependencies

```
// × MAL: didChangeDependencies puede llamarse muchas veces
@override
void didChangeDependencies() {
  super.didChangeDependencies();
  setState(() { // ← Causa rebuilds potencialmente infinitos
    _data = fetchData(context);
  });
}

// ✅ BIEN: Inicializar en initState con una carga async
@override
void initState() {
  super.initState();
  _loadData();
}

Future<void> _loadData() async {
  final data = await fetchData(context);
  if (mounted) {
    setState(() {
      _data = data;
    });
  }
}
```

7.3 Anti-patrón 3: setState sin cambios reales

```
// × MAL: setState aunque nada haya cambiado
void onPressed() {
  setState(() {
    // No hay cambios en el estado
  });
}

// ✔ BIEN: Solo setState si hay cambios
void onPressed() {
  if (_counter < _max) {
    setState(() {
      _counter++;
    });
  }
}
```

7.4 Anti-patrón 4: reasignar objetos mutables

```
// × MAL: Flutter no detecta cambios en objetos mutables
final List<String> _items = [];

void addItem(String item) {
  _items.add(item);
  setState(() {}); // Flutter no sabe que _items cambió
}

// ✔ BIEN: Crear una nueva referencia
final List<String> _items = [];

void addItem(String item) {
  setState(() {
    _items = [..._items, item]; // Nueva referencia
  });
}
```

8. Reglas de oro para la optimización de rebuilds

#	Regla	Herramienta
1	Usa const en todos los widgets que puedan serlo	<code>const</code> keyword
2	Mueve widgets costosos a su propio StatelessWidget	Extracción de widgets
3	Usa BlocSelector para estados grandes	<code>BlocSelector</code>
4	Agrega buildWhen cuando BlocSelector no baste	<code>buildWhen</code> param
5	Usa RepaintBoundary en listas con animaciones	<code>RepaintBoundary</code>
6	Usa Keys en listas dinámicas	<code>ValueKey</code> , <code>UniqueKey</code>

#	Regla	Herramienta
7	Un solo setState por ciclo de actualización	Agrupar cambios
8	Nunca mutar el estado sin crear nueva referencia	Inmutabilidad
9	Extrae widgets que no dependen del estado	<code>StatelessWidget</code>
10	Mide antes de optimizar	DevTools Performance

9. Ejemplos: antes vs después

9.1 Ejemplo 1: lista con items constantes

Antes:

```
@override
Widget build(BuildContext context) {
  return Column(
    children: [
      Text('Titulo de seccion'),      // No const
      Icon(Icons.settings),           // No const
      const Divider(),                // No const
      ...items.map((i) => ItemWidget(i)),
    ],
  );
}
```

Después:

```
@override
Widget build(BuildContext context) {
  return Column(
    children: [
      const Text('Titulo de seccion'), // Const
      const Icon(Icons.settings),       // Const
      const Divider(),                 // Const
      ...items.map((i) => ItemWidget(
        key: ValueKey(i.id),           // Key
        item: i,
      )),
    ],
  );
}
```

9.2 Ejemplo 2: extraer el subtree que cambia

Antes:

```
class ParentWidget extends StatefulWidget {  
  @override  
  State<ParentWidget> createState() => _ParentWidgetState();  
}  
  
class _ParentWidgetState extends State<ParentWidget> {  
  int _counter = 0;  
  
  @override  
  Widget build(BuildContext context) {  
    return Column(  
      children: [  
        const ExpensiveChild(), // Se reconstruye cada vez  
        Text('Counter: $_counter'),  
        ElevatedButton(  
          onPressed: () => setState(() => _counter++),  
          child: const Text('Increment'),  
        ),  
      ],  
    );  
  }  
}
```

Después: extraer el bloque que cambia a su propio widget con estado:


```

class ParentWidget extends StatefulWidget {
  @override
  State<ParentWidget> createState() => _ParentWidgetState();
}

class _ParentWidgetState extends State<ParentWidget> {
  int _counter = 0;

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        const ExpensiveChild(), // const → no se reconstruye
        _CounterSection(
          counter: _counter,
          onIncrement: () => setState(() => _counter++),
        ),
      ],
    );
  }
}

class _CounterSection extends StatelessWidget {
  const _CounterSection({required this.counter, required this.onIncrement});

  final int counter;
  final VoidCallback onIncrement;

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text('Counter: $counter'),
        ElevatedButton(
          onPressed: onIncrement,
          child: const Text('Increment'),
        ),
      ],
    );
  }
}

```

10. Matriz de decisión: qué técnica para qué escenario

Escenario	Técnica principal	Técnica secundaria
Textos, iconos, espaciados estáticos	<code>const</code>	–
Lista de items con ID	<code>ValueKey</code>	<code>ListView.builder</code>
Estado BLoC con 1 campo relevante	<code>BlocSelector</code>	–
Estado BLoC con múltiples campos	<code>buildWhen</code>	Múltiples <code>BlocSelector</code>
Animación en un item de lista	<code>RepaintBoundary</code>	<code>AnimatedBuilder</code>

Escenario	Técnica principal	Técnica secundaria
Widget complejo que rara vez cambia	Extracción <code>StatelessWidget</code>	<code>const</code>
setState con múltiples cambios	Agrupar en un solo <code>setState</code>	-
Objeto mutable que cambia	Nueva referencia (<code>[...]</code>)	Inmutabilidad
Widget con painter custom	<code>shouldRepaint: false</code>	<code>RepaintBoundary</code>
Texto que cambia en área animada	<code>RepaintBoundary</code> + <code>const</code>	Extracción

Resumen

Concepto	Clave
<code>const</code>	Reutiliza la instancia, sin <code>build()</code>
Keys	Emparejamiento correcto de widgets
BlocSelector	Rebuild solo cuando el selector cambia
buildWhen	Control fino sobre cuándo reconstruir
RepaintBoundary	Limita el área de repintado
setState	Un solo call por ciclo, sin mutar el estado

Referencias

- Flutter | [const performance](#) — Uso de `const` para rendimiento
- Bloc | [BlocSelector y buildWhen](#) — Documentación oficial de bloc
- Flutter | [RepaintBoundary](#) — API de RepaintBoundary

 **Siguiente:** [21-memory-leak-detection.md](#) — Detección y prevención de memory leaks

21 — Detección y Prevención de Memory Leaks

Qué es un memory leak, cómo detectarlo con DevTools Memory y qué patrones aplicar para prevenirlo.

1. ¿Qué es un memory leak en Flutter?

Un **memory leak** ocurre cuando tu app asigna memoria pero nunca la libera, aunque ya no la necesita. En Flutter, esto sucede cuando los recursos (streams, controllers, suscripciones) no se cierran correctamente en `dispose()`.

Memoria normal:

Asignada

Liberada

← La memoria se recicla

Memory leak:

Asignada

Liberada: (nunca)

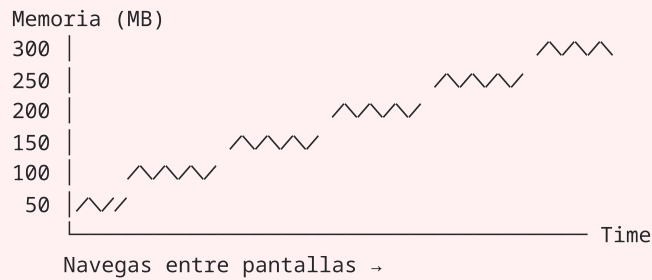
↑ Nunca se libera

2. Tipos de memory leaks en Flutter

Tipo	Causa	Síntoma
Stream abierto	<code>StreamSubscription</code> no cancelada	Memoria crece con cada evento
Controller no disposed	<code>TextEditingController</code> sin dispose	Memoria no liberada al salir
Timer/Animation sin cancel	<code>Timer.periodic</code> o <code>AnimationController</code> sin detener	CPU + memoria creciente
Listener no removido	<code>WidgetsBindingObserver</code> sin remove	Referencia circular
Closure que captura context	Lambda que mantiene referencia al widget	Widget no se destruye
GlobalKey no removida	GlobalKey en widget temporal	Estado se mantiene en memoria

2.1 Cómo se manifiestan

Memory leak creciente:



La memoria sube pero NUNCA baja → LEAK

3. Detección con DevTools Memory

3.1 Paso 1: tomar un heap snapshot

1. Abre DevTools → **Memory** view
2. Navega a la pantalla que sospechas
3. Presiona "**Take heap snapshot**" (icono de cámara 📷)
4. Este es tu snapshot base

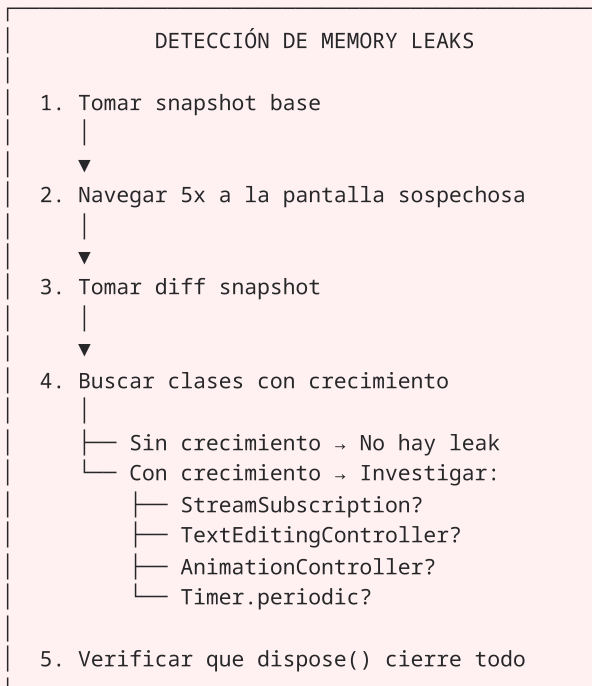
3.2 Paso 2: tomar un diff snapshot

1. Navega fuera de la pantalla y de vuelta (repite 3-5 veces)
2. Presiona "**Take heap snapshot**" de nuevo
3. Los objetos que **aumentan** entre snapshots son candidatos a leak

3.3 Paso 3: allocation sampling

1. Activa **Allocation sampling** (botón de grabar)
2. Interactúa con la app normalmente
3. Detén el sampling
4. Revisa las clases con **mayor crecimiento** de instancias

3.4 Flujo de detección



4. Patrón correcto de dispose() en StatefulWidget

```
class _MyWidgetState extends State<MyWidget> {
  // Controllers que necesitan dispose
  late TextEditingController _textController;
  late ScrollController _scrollController;
  late AnimationController _animController;
  late StreamSubscription _subscription;

  @override
  void initState() {
    super.initState();

    // Inicializar controllers
    _textController = TextEditingController();
    _scrollController = ScrollController();
    _animController = AnimationController(
      vsync: this,
      duration: const Duration(seconds: 1),
    );

    // Suscribirse a streams
    _subscription = myStream.listen((data) {
      // Manejar datos
    });
  }

  @override
  void dispose() {
    // ! Cerrar TODOS los recursos en orden inverso
    _subscription.cancel();
    _animController.dispose();
    _scrollController.dispose();
    _textController.dispose();

    super.dispose();
  }
}
```

4.1 Regla de oro

Si creaste un controller o te suscribiste a un stream, DEBES cerrarlo en dispose().

5. Gestión de StreamSubscription

5.1 Patrón básico

```
class _ChatState extends State<Chat> {
  StreamSubscription? _messagesSubscription;

  @override
  void initState() {
    super.initState();
    _messagesSubscription = Supabase.instance.client
      .from('messages')
      .stream(primaryKey: ['id'])
      .order('created_at')
      .listen((messages) {
        setState(() {
          _messages = messages;
        });
      });
  }

  @override
  void dispose() {
    _messagesSubscription?.cancel(); // ← Cancelar suscripción
    super.dispose();
  }
}
```

5.2 Patrón con BloC

```
class ChatBloc extends Bloc<ChatEvent, ChatState> {
  final ChatRepository _repository;
  StreamSubscription? _messagesSubscription;

  ChatBloc(this._repository) : super(ChatInitial()) {
    on<StartListening>(_onStartListening);
    on<StopListening>(_onStopListening);
    on<MessageReceived>(_onMessageReceived);
  }

  void _onStartListening(StartListening event, Emitter<ChatState> emit) {
    _messagesSubscription = _repository.getMessages().listen(
      (messages) => add(MessageReceived(messages)),
    );
  }

  @override
  Future<void> close() {
    _messagesSubscription?.cancel(); // ← Cerrar en close()
    return super.close();
  }
}
```

6. Ciclo de vida de controllers comunes

6.1 TextEditingController

```
// ✅ Patrón correcto
late TextEditingController _controller;

@override
void initState() {
  super.initState();
  _controller = TextEditingController(text: 'initial');
}

@override
void dispose() {
  _controller.dispose(); // ← Siempre dispose
  super.dispose();
}
```

6.2 ScrollController

```
// ✅ Patrón correcto
late ScrollController _scrollController;

@override
void initState() {
  super.initState();
  _scrollController = ScrollController();
  _scrollController.addListener(_onScroll);
}

void _onScroll() {
  if (_scrollController.position.pixels ==
      _scrollController.position.maxScrollExtent) {
    _loadMore();
  }
}

@override
void dispose() {
  _scrollController.removeListener(_onScroll); // ← Primero remover listener
  _scrollController.dispose();                // ← Luego dispose
  super.dispose();
}
```


6.3 AnimationController

```
// ✅ Patrón correcto
late AnimationController _controller;

@override
void initState() {
  super.initState();
  _controller = AnimationController(
    vsync: this,
    duration: const Duration(milliseconds: 500),
  );
}

@override
void dispose() {
  _controller.dispose(); // ← Siempre dispose
  super.dispose();
}
```

6.4 Timer

```
// ✗ MAL: Timer.periodic sin cancelar
Timer.periodic(const Duration(seconds: 5), (timer) {
  _fetchData();
});

// ✅ BIEN: Guardar referencia y cancelar
late Timer _timer;

@override
void initState() {
  super.initState();
  _timer = Timer.periodic(const Duration(seconds: 5), (timer) {
    _fetchData();
  });
}

@override
void dispose() {
  _timer.cancel(); // ← Cancelar timer
  super.dispose();
}
```

7. Patrón de cancelación segura

```
// Patrón para streams con posibles errores
StreamSubscription? _subscription;
bool _isDisposed = false;

@override
void initState() {
  super.initState();
  _subscription = myStream.listen(
    (data) {
      if (!_isDisposed) {
        setState(() { _data = data; });
      }
    },
    onError: (error) {
      if (!_isDisposed) {
        handleError(error);
      }
    },
  );
}

@override
void dispose() {
  _isDisposed = true;
  _subscription?.cancel();
  super.dispose();
}
```

8. Patrones comunes de leak y sus soluciones

8.1 Patrón 1: stream en initState sin dispose

```
// × LEAK: Stream nunca se cancela
@override
void initState() {
  super.initState();
  Supabase.instance.client
    .from('posts')
    .stream(primaryKey: ['id'])
    .listen((posts) {
      setState(() { _posts = posts; });
    });
  // ← No hay cancelación
}

// ✅ FIX: Guardar y cancelar
StreamSubscription? _postsSubscription;

@override
void initState() {
  super.initState();
  _postsSubscription = Supabase.instance.client
    .from('posts')
    .stream(primaryKey: ['id'])
    .listen((posts) {
      if (mounted) {
        setState(() { _posts = posts; });
      }
    });
}

@override
void dispose() {
  _postsSubscription?.cancel();
  super.dispose();
}
```

8.2 Patrón 2: closure con referencia al contexto

```
// × LEAK: Closure mantiene referencia al widget
Timer.periodic(const Duration(seconds: 1), (timer) {
  Navigator.of(context).push(...); // context capturado
});

// ✅ FIX: Verificar mounted antes de usar context
Timer.periodic(const Duration(seconds: 1), (timer) {
  if (mounted) {
    Navigator.of(context).push(...);
  }
});
```

8.3 Patrón 3: listener sin remover

```
// × LEAK: Listener nunca se remueve
@override
void initState() {
  super.initState();
  WidgetsBinding.instance.addObserver(this);
  // ← No se remueve en dispose
}

// ✅ FIX: Agregar y remover simétricamente
@override
void initState() {
  super.initState();
  WidgetsBinding.instance.addObserver(this);
}

@override
void dispose() {
  WidgetsBinding.instance.removeObserver(this); // ← Remover
  super.dispose();
}
```

9. Checklist de detección de memory leaks

- Paso 1: Identificar la pantalla sospechosa
 - └─ ¿Qué pantalla muestra crecimiento de memoria?
- Paso 2: Auditar initState()
 - └─ ¿Qué controllers/streams/timers se crean aquí?
 - └─ Por cada uno: ¿tiene su correspondiente dispose()?
- Paso 3: Auditar dispose()
 - └─ ¿Todos los recursos tienen close/cancel/dispose?
 - └─ ¿Se llama super.dispose() al final?
- Paso 4: Verificar patrones de suscripción
 - └─ StreamSubscription: ¿se cancela en dispose?
 - └─ Timer: ¿se cancela en dispose?
 - └─ WidgetsBindingObserver: ¿se remueve en dispose?
- Paso 5: Probar con DevTools Memory
 - └─ Tomar snapshot base
 - └─ Navegar 5x a la pantalla
 - └─ Tomar diff snapshot
 - └─ Buscar clases con crecimiento de instancias
- Paso 6: Verificar BLoC/Cubit
 - └─ ¿close() cancela todas las suscripciones?
 - └─ ¿BlocObserver reporta correctamente?

10. BLoC/Cubit: cuándo cerrar streams

10.1 En Cubit

```
class ChatCubit extends Cubit<ChatState> {
  final ChatRepository _repository;
  StreamSubscription? _messagesSub;

  ChatCubit(this._repository) : super(ChatInitial());

  void startListening() {
    _messagesSub = _repository.getMessages().listen(
      (messages) => emit(ChatLoaded(messages)),
    );
  }

  @override
  void close() {
    _messagesSub?.cancel(); // ← Cerrar suscripción
    return super.close();
  }
}
```

10.2 BlocObserver para tracking de memoria

```
class AppBlocObserver extends BlocObserver {
  @override
  void onCreate(BlocBase bloc) {
    super.onCreate(bloc);
    debugPrint('BLoC created: ${bloc.runtimeType}');
  }

  @override
  void onClose(BlocBase bloc) {
    debugPrint('BLoC closed: ${bloc.runtimeType}');
    super.onClose(bloc);
  }

  @override
  void onError(BlocBase bloc, Object error, StackTrace stackTrace) {
    debugPrint('BLoC error: ${bloc.runtimeType} - $error');
    super.onError(bloc, error, stackTrace);
  }
}

// En main.dart
void main() {
  Bloc.observer = AppBlocObserver();
  runApp(MyApp());
}
```

11. Comandos para investigación de memoria

```
# Ejecutar en profile para mediciones reales
flutter run --profile

# Ver memoria en tiempo real con DevTools
# Abrir DevTools > Memory view

# Analizar el uso de memoria del build
flutter build apk --analyze-size
```

Resumen

Concepto	Acción
Stream sin cancelar	Guardar <code>StreamSubscription</code> y cancelar en <code>dispose()</code>
Controller sin dispose	Siempre llamar <code>.dispose()</code> en <code>dispose()</code>
Timer sin cancel	Guardar referencia y <code>.cancel()</code> en <code>dispose()</code>
Listener sin remove	<code>removeObserver</code> simétrico a <code>addObserver</code>
Closure con context	Verificar <code>mounted</code> antes de usar
BLoC sin close	Cancelar suscripciones en <code>close()</code>
Detección	DevTools Memory → heap snapshots → diff

Referencias

- Flutter | Memory view — Documentación oficial de la Memory view
- Flutter | Memory — Conceptos de memoria en Flutter
- Dart | Streams — Manejo de streams en Dart

 **Siguiente:** 22-rendering-complejo.md — Rendering complejo: shaders y rasterización

22 — Rendering Complejo: Listas, Slivers e Imágenes

Optimizar listas largas, scrolls complejos con Slivers, imágenes y rendering pesado.

1. ListView.builder vs ListView

La diferencia más crítica en listas Flutter: **ListView** **construye todos los items de una vez**, mientras que **ListView.builder** **solo construye los visibles**.

1.1 ListView (sin builder)

```
// × CONSTRUYE TODOS LOS ITEMS de una vez
ListView(
  children: items.map((item) => ItemWidget(
    key: ValueKey(item.id),
    item: item,
  )).toList(),
)
```

```
Items totales: 1000
Items contruidos: 1000 ← TODOS
Items visibles: 15
Memoria usada: MUCHA
```

1.2 ListView.builder (lazy loading)

```
// ✓ SOLO CONSTRUYE los items visibles
ListView.builder(
  itemCount: items.length,
  itemBuilder: (context, index) {
    return ItemWidget(
      key: ValueKey(items[index].id),
      item: items[index],
    );
  },
)
```

```
Items totales: 1000
Items contruidos: ~20 ← Solo visibles + buffer
Items visibles: 15
Memoria usada: MÍNIMA
```

1.3 Comparación

Característica	ListView	ListView.builder
Items contruidos	Todos	Solo visibles
Memoria	Lineal con items	Constante
Scroll inicial	Instantáneo	Puede tener micro-lag
Ideal para	< 20 items	> 20 items
Performance	O(n)	O(1)

1.4 Cuándo usar cada uno

```
¿Cuántos items tiene tu lista?
├─ < 10 items → ListView (simple, no hay diferencia)
├─ 10-50 items → ListView.builder (precaución)
├─ > 50 items → ListView.builder (obligatorio)
└─ Items de tamaño variable → ListView.builder + SliverList
```

2. Slivers: layouts de scroll complejos

Los **Slivers** permiten crear layouts de scroll donde diferentes secciones tienen diferentes comportamientos.

2.1 CustomScrollView

```
CustomScrollView(  
  slivers: [  
    // Header fijo  
    SliverAppBar(  
      expandedHeight: 200,  
      floating: true,  
      pinned: true,  
      flexibleSpace: FlexibleSpaceBar(  
        title: const Text('Mi App'),  
        background: Image.network(url, fit: BoxFit.cover),  
      ),  
    ),  
  
    // Lista de items  
    SliverList(  
      delegate: SliverChildBuilderDelegate(  
        (context, index) => ListTile(  
          title: Text('Item $index'),  
        ),  
        childCount: 100,  
      ),  
    ),  
  
    // Grid de productos  
    SliverGrid(  
      delegate: SliverChildBuilderDelegate(  
        (context, index) => ProductCard(product: products[index]),  
        childCount: products.length,  
      ),  
      gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(  
        crossAxisCount: 2,  
        childAspectRatio: 0.75,  
      ),  
    ),  
  
    // Footer  
    const SliverToBoxAdapter(  
      child: FooterWidget(),  
    ),  
  ],  
)
```

2.2 Slivers disponibles

Sliver	Uso	Performance
SliverList	Lista vertical lazy	✓ Excelente
SliverGrid	Grid lazy	✓ Excelente
SliverAppBar	AppBar con scroll	✓ Excelente
SliverFixedExtentList	Lista con items de alto fijo	✓✓ Mejor que SliverList
SliverPrototypeExtentList	Lista con prototype de alto	✓✓✓ Muy rápido
SliverFillRemaining	Contenido que llena el espacio restante	✓

Sliver	Uso	Performance
SliverToBoxAdapter	Widget normal dentro de CustomScrollView	✓

2.3 SliverFixedExtentList: la versión rápida

```
// SliverList: calcula constraints por cada item
SliverList(
  delegate: SliverChildBuilderDelegate(
    (context, index) => ListTile(title: Text('Item $index')),
    childCount: 1000,
  ),
)

// SliverFixedExtentList: sabe el alto exacto, más rápido
SliverFixedExtentList(
  delegate: SliverChildBuilderDelegate(
    (context, index) => ListTile(title: Text('Item $index')),
    childCount: 1000,
  ),
  itemExtent: 56.0, // ← Alto fijo en pixels
)
```

3. Optimización de imágenes

3.1 cacheWidth y cacheHeight

Cuando cargas una imagen de red, Flutter la decodifica a su tamaño original. Si la imagen es de 4000x3000 pero solo necesitas 200x150, estás desperdiciando memoria y CPU.

```
// × IMAGEN COMPLETA decodificada (4000x3000 = 12M pixels)
Image.network(
  'https://example.com/photo.jpg',
)

// ✓ IMAGEN REDIMENSIONADA al tamaño real de display (200x150 = 30K pixels)
Image.network(
  'https://example.com/photo.jpg',
  width: 200,
  height: 150,
  cacheWidth: 200, // ← Decodifica a este tamaño
  cacheHeight: 150, // ← Ahorra ~99% de memoria
)
```

Memoria de imagen:

Sin cacheWidth/Height:

$4000 \times 3000 \times 4 \text{ bytes} = 48 \text{ MB}$ ← Desperdicio

Con cacheWidth/Height (display: 200x150):

$200 \times 150 \times 4 \text{ bytes} = 120 \text{ KB}$ ← 400x menos memoria

3.2 AssetImage vs NetworkImage

Tipo	Carga	Cache	Uso ideal
AssetImage	Local, rápida	En memoria	Iconos, logos, backgrounds
NetworkImage	Remota, lenta	Disco + memoria	Fotos de usuario, contenido
FileImage	Local, rápida	En memoria	Fotos tomadas con cámara

3.3 precacheImage: cargar imágenes antes de necesitarlas

```
@override
void initState() {
  super.initState();

  // Precargar la imagen para que aparezca instantáneamente
  precacheImage(
    NetworkImage('https://example.com/hero.jpg'),
    context,
  );
}

@override
Widget build(BuildContext context) {
  return Image.network('https://example.com/hero.jpg');
  // ↑ La imagen ya está en cache, carga instantánea
}
```

3.4 Patrón completo de imagen optimizada

```
class OptimizedImage extends StatelessWidget {
  final String url;
  final double width;
  final double height;

  const OptimizedImage({
    super.key,
    required this.url,
    required this.width,
    required this.height,
  });

  @override
  Widget build(BuildContext context) {
    final dpr = MediaQuery.devicePixelRatioOf(context);
    return ClipRRect(
      borderRadius: BorderRadius.circular(8),
      child: Image.network(
        url,
        width: width,
        height: height,
        cacheWidth: (width * dpr).toInt(),
        cacheHeight: (height * dpr).toInt(),
        fit: BoxFit.cover,
        errorBuilder: (context, error, stackTrace) {
          return Container(
            width: width,
            height: height,
            color: Colors.grey[300],
            child: const Icon(Icons.error),
          );
        },
      ),
    );
  }
}
```

4. ShaderMask y ClipRect: consideraciones de performance

4.1 ShaderMask

```
// ShaderMask aplica un gradiente sobre el contenido
// Cada frame: lee el widget hijo + aplica el shader
ShaderMask(
  shaderCallback: (Rect bounds) {
    return LinearGradient(
      colors: [Colors.white, Colors.transparent],
    ).createShader(bounds);
  },
  blendMode: BlendMode.dstIn,
  child: Image.network(url),
)
```

Impacto	Causa
Medio	Calcula shader por cada frame
Alto si hay animación	Shader se recalcula continuamente
Bajo si es estático	Una sola vez al renderizar

4.2 ClipRect

```
// ClipRect recorta el contenido a los límites del padre
// Impide que los hijos se dibujen fuera del área
ClipRect(
  child: SizedBox(
    width: 100,
    height: 100,
    child: LargeWidget(), // ← Solo se dibuja lo visible
  ),
)
```

Impacto	Causa
Bajo-Medio	Crea una capa de composición
Alto si anidados	Múltiples clip layers = GPU work

4.3 Reglas de uso

Widget	Usar cuando...	Evitar cuando...
<code>ShaderMask</code>	Efecto visual necesario	Decoración sin valor real
<code>ClipRect</code>	Overflow controlado	Se puede evitar con constraints en el hijo
<code>ClipRRect</code>	Bordes redondeados	Usar <code>Decoration</code> con <code>borderRadius</code>

5. RepaintBoundary en listas con animaciones

```
class AnimatedListItem extends StatefulWidget {
  final Item item;

  const AnimatedListItem({super.key, required this.item});

  @override
  State<AnimatedListItem> createState() => _AnimatedListItemState();
}

class _AnimatedListItemState extends State<AnimatedListItem>
  with SingleTickerProviderStateMixin {
  late AnimationController _controller;

  @override
  void initState() {
    super.initState();
    _controller = AnimationController(
      vsync: this,
      duration: const Duration(milliseconds: 300),
    );
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return RepaintBoundary( // ← Limita el repintado a este item
      child: AnimatedBuilder(
        animation: _controller,
        builder: (context, child) {
          return Opacity(
            opacity: _controller.value,
            child: child,
          );
        },
        child: ListTile(
          title: Text(widget.item.name),
          leading: CircleAvatar(child: Text(widget.item.name[0])),
        ),
      ),
    );
  }
}
```

6. shouldRepaint en CustomPainter

6.1 Patrón básico

```
class MyPainter extends CustomPainter {
  final double progress;

  MyPainter(this.progress);

  @override
  void paint(Canvas canvas, Size size) {
    final paint = Paint()
      ..color = Colors.blue
      ..strokeWidth = 2;

    canvas.drawLine(
      Offset(0, size.height * progress),
      Offset(size.width, size.height * progress),
      paint,
    );
  }

  @override
  bool shouldRepaint(MyPainter oldDelegate) {
    // ✅ SOLO repintar si el progress cambió
    return oldDelegate.progress != progress;
  }
}
```

6.2 shouldRepaint: qué retornar

Escenario	Retorno	Razón
Widget completamente estático	<code>return false</code>	Nunca cambia
Depende de un solo campo	<code>return old.field != field</code>	Solo repinta si cambia
Siempre cambia	<code>return true</code>	No hay forma de evitarlo
Depende de múltiples campos	Comparar todos los campos	Granularidad máxima

6.3 Ejemplo estático

```
class BackgroundPainter extends CustomPainter {
  @override
  void paint(Canvas canvas, Size size) {
    // Dibuja un patrón estático
    final paint = Paint()..color = Colors.grey[200]!;
    for (double i = 0; i < size.width; i += 20) {
      canvas.drawLine(Offset(i, 0), Offset(i, size.height), paint);
    }
  }

  @override
  bool shouldRepaint(BackgroundPainter oldDelegate) {
    return false; // ← NUNCA repinta, es estático
  }
}
```

7. Animaciones: AnimatedContainer vs AnimatedBuilder

7.1 AnimatedContainer

```
// ✅ SIMPLE: animación implícita de propiedades
AnimatedContainer(
  duration: const Duration(milliseconds: 300),
  width: isExpanded ? 200 : 100,
  color: isExpanded ? Colors.blue : Colors.red,
  child: const Text('Animated'),
)
```

7.2 AnimatedBuilder

```
// ✅ COMPLEJO: animación personalizada
AnimatedBuilder(
  animation: _controller,
  builder: (context, child) {
    return Transform.rotate(
      angle: _controller.value * 2 * math.pi,
      child: child, // ← El child NO se reconstruye
    );
  },
  child: Icon(Icons.refresh, size: 48), // ← Construido una sola vez
)
```

7.3 Comparación

Característica	AnimatedContainer	AnimatedBuilder
Complejidad	Baja	Media-Alta

Característica	AnimatedContainer	AnimatedBuilder
Propiedades	Limitadas (width, height, color, etc.)	Cualquier propiedad
Performance	Buena	Excelente (con child)
Uso ideal	Animaciones de layout	Animaciones custom

8. Widgets pesados: splitting build methods

8.1 Patrón: extracción de widgets

```
// ✗ MAL: Todo en un solo build method gigante
class DashboardScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        // Header (100 líneas de código)
        // Chart (200 líneas de código)
        // Table (150 líneas de código)
        // Footer (50 líneas de código)
      ],
    );
  }
}

// ✔ BIEN: Widgets extraídos
class DashboardScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return const Column(
      children: [
        DashboardHeader(),
        DashboardChart(),
        DashboardTable(),
        DashboardFooter(),
      ],
    );
  }
}

class DashboardHeader extends StatelessWidget {
  const DashboardHeader({super.key});

  @override
  Widget build(BuildContext context) {
    // Header logic aquí
  }
}

class DashboardChart extends StatelessWidget {
  const DashboardChart({super.key});

  @override
  Widget build(BuildContext context) {
    // Chart logic aquí
  }
}
```

8.2 Beneficios

Beneficio	Descripción
Rebuild selectivo	Cada widget se reconstruye independientemente

Beneficio	Descripción
Reutilización	Puedes reusar widgets en otras partes
Testeabilidad	Tests más fáciles por widget aislado
Legibilidad	Cada widget tiene una responsabilidad

9. Costo de performance de widgets comunes

Widget	Costo	Alternativa más liviana
Stack	Medio	Column / Row si es posible
Positioned (en Stack)	Medio	Align en un Container
Opacity	Medio-Alto	AnimatedOpacity si cambia
ClipRRect	Medio	Decoration con borderRadius
ShaderMask	Alto	Pre-renderizar el efecto
BackdropFilter	Muy alto	Evitar o usar imagen pre-renderizada
Table	Medio	Column con Row custom
RichText	Bajo	Mejor que múltiples widgets Text
Wrap	Medio	Row con Expanded si es posible
LayoutBuilder	Medio	Constraints conocidos = sin LayoutBuilder

9.1 Opacity: el villano silencioso

`Opacity` crea una capa de composición separada en la GPU. No hay problema si es estático, pero **evita aplicarlo sobre subtrees grandes o animarlo con `setState`**.

```
// × MAL: Opacity sobre un subtree grande
Opacity(
  opacity: 0.5,
  child: ExpensiveWidget(),
)

// ✓ BIEN: Solo aplica opacidad cuando el valor cambia
AnimatedOpacity(
  opacity: isVisible ? 1 : 0,
  duration: const Duration(milliseconds: 300),
  child: ExpensiveWidget(),
)

// ✓ Si es para deshabilitar interacción, no uses opacidad:
IgnorePointer(
  child: ExpensiveWidget(), // Sin interacción
)
```

Resumen

Tema	Técnica clave
Listas largas	<code>ListView.builder</code> obligatorio
Scrolls complejos	<code>CustomScrollView</code> + Slivers
Listas con alto fijo	<code>SliverFixedExtentList</code>
Imágenes	<code>cacheWidth</code> + <code>cacheHeight</code> siempre
Carga de imágenes	<code>precacheImage</code> en initState
Animaciones en listas	<code>RepaintBoundary</code> por item
CustomPainter	<code>shouldRepaint</code> con comparación
Widgets pesados	Extracción a StatelessWidget
Opacity	Evitar sobre subtrees grandes

Referencias

- Flutter | Slivers — Scrolls avanzados con Slivers
- Flutter | Imágenes y memoria — `cacheWidth` / `cacheHeight` y cache de imágenes
- Flutter | CustomPainter — API de CustomPainter y shouldRepaint

 **Siguiente:** 23-cheatsheet-optimizacion.md — Cheatsheet de optimización de rendimiento

23 — Cheatsheet de Optimización de Rendimiento

Referencia rápida: problemas comunes → causa → solución → vista de DevTools, métricas saludables, anti-patrones, comandos y mitos.

1. Problema → Causa → Solución → DevTools View

Problema	Causa probable	Solución	DevTools View
Jank en scroll	Build time > 8ms por item	<code>ListView.builder</code> + <code>const</code>	Performance
Jank en animación	Muchos widgets en el subtree	<code>RepaintBoundary</code> + <code>AnimatedBuilder</code>	Performance
Memoria crece sin parar	Stream/Controller sin dispose	Agregar <code>dispose()</code> correcto	Memory
Frame rojo en DevTools	UI thread lento	Reducir trabajo en <code>build()</code>	Performance
Frame morado en DevTools	Raster thread lento	Menos capas, menos opacidad	Performance
App consume mucha RAM	Imágenes sin <code>cacheWidth/Height</code>	Agregar <code>cacheWidth</code> / <code>cacheHeight</code>	Memory
Transición lenta	Demasiados widgets en pantalla	Extraer widgets, usar <code>const</code>	Performance
Rebuilds excesivos	BlocBuilder sin filtro	<code>BlocSelector</code> o <code>buildWhen</code>	Performance > Enhance Tracing
Texto parpadea	Rebuild completo del subtree	<code>const</code> + extracción de widgets	Performance
App lenta en inicio	Work pesado en <code>initState</code>	Lazy loading, <code>Isolate</code>	CPU Profiler

2. Rangos saludables de métricas

Métrica	Excelente	Aceptable	Necesita acción
FPS	60 (120 ProMotion)	55-59	< 55
Build time	< 4ms	4-8ms	> 8ms
Raster time	< 4ms	4-8ms	> 8ms
Memory (idle)	< 100MB	100-200MB	> 200MB
Memory growth	0 MB/min	< 1 MB/min	> 1 MB/min
GC frequency	< 1/min	1-5/min	> 5/min
App startup	< 1s	1-3s	> 3s
Widget tree depth	< 15 niveles	15-25	> 25

3. Checklist pre-optimización

- Ejecutar en profile
 - `flutter run --profile`
- Habilitar Performance Overlay
 - Ver frames rojos/morados en pantalla
- Abrir DevTools Performance
 - Grabar frame timeline (5-10 segundos de interacción)
- Identificar el frame más lento
 - Buscar el frame con mayor tiempo total
- Analizar el flame chart
 - ¿Qué función consume más tiempo?
- Verificar memory baseline
 - Tomar heap snapshot base
- Repetir interacción 5x
 - Tomar diff snapshot
- Clasificar el problema
 - Build lento → Optimizar rebuilds
 - Raster lento → Optimizar rendering
 - Memory leak → Buscar dispose faltante
 - Todo bien → No optimizar aún

4. Anti-patrones

Patrón Incorrecto	Por qué es malo	Mejor enfoque
<code>ListView</code> con 100+ items	Construye todo de una vez	<code>ListView.builder</code>
<code>BlocBuilder</code> sin filtro	Reconstruye todo el subtree	<code>BlocSelector</code>
<code>setState</code> en loop	N rebuilds por N iteraciones	Un solo <code>setState</code> agrupado
<code>Opacity</code> widget	Crea capa GPU separada	<code>AnimatedOpacity</code> , <code>IgnorePointer</code> o <code>painter</code>
<code>Image.network</code> sin sizes	Decodifica imagen completa	<code>cacheWidth</code> + <code>cacheHeight</code>
<code>TextEditingController</code> sin dispose	Memory leak	Cerrar en <code>dispose()</code>
<code>Timer.periodic</code> sin cancel	CPU + memoria leak	Cerrar en <code>dispose()</code>
Widget sin <code>const</code> constante	Reconstrucción innecesaria	Agregar <code>const</code>
<code>setState</code> en <code>didChangeDependencies</code>	Rebuilds potencialmente infinitos	<code>initState</code> + carga async
<code>Key</code> sin razón	Overhead innecesario	Solo usar Key cuando es necesario

5. Atajos y comandos de profiling

5.1 CLI

```
# Ejecutar en profile (OBLIGATORIO para mediciones)
flutter run --profile

# Analizar tamaño del build
flutter build apk --analyze-size
flutter build ios --analyze-size

# Ver dependencias
flutter pub deps

# Clean y rebuild
flutter clean && flutter pub get

# Ver información del dispositivo
flutter devices
flutter emulators
```

5.2 DevTools Views

Vista	Acceso rápido	Para qué
Performance	<code>Ctrl+Shift+P</code> → "Performance"	Frames, jank, timeline
Memory	<code>Ctrl+Shift+P</code> → "Memory"	Leaks, heap, GC
CPU Profiler	<code>Ctrl+Shift+P</code> → "CPU Profiler"	Flame chart, sampling
Inspector	<code>Ctrl+Shift+P</code> → "Inspector"	Widget tree, layout
Network	<code>Ctrl+Shift+P</code> → "Network"	HTTP requests

6. Flujo de investigación de memoria (ASCII)

```
App consume mucha memoria?  
├─ 1. DevTools > Memory > Tomar snapshot base  
├─ 2. Navegar a pantalla sospechosa (5x)  
├─ 3. Tomar diff snapshot  
├─ 4. Buscar clases con crecimiento  
│   ├── StreamSubscription con crecimiento  
│   │   └─ ✓ Fix: Guardar y cancelar en dispose()  
│   ├── TextEditingController con crecimiento  
│   │   └─ ✓ Fix: Agregar dispose()  
│   ├── AnimationController con crecimiento  
│   │   └─ ✓ Fix: Agregar dispose()  
│   ├── Timer con crecimiento  
│   │   └─ ✓ Fix: Guardar referencia y cancel()  
│   └─ Clase desconocida con crecimiento  
│       ├── 🔍 Investigar: ¿qué la instancia?  
│       └─ Buscar en código: new ClassName()  
└─ 5. Repetir hasta que no haya crecimiento
```

7. Mitos comunes de rendimiento desmentidos

Mito	Realidad
"Flutter es lento porque usa Dart"	Dart AOT compila a código nativo, rendimiento similar a Kotlin/Swift

Mito	Realidad
"Debo usar keys en todos los widgets"	Keys solo son necesarios cuando Flutter necesita distinguir widgets
"setState es malo, siempre usa BLoC"	setState es perfecto para estado local simple
"Más widgets = más lento"	El número de widgets no importa, importa el trabajo en build/paint
"DevTools en debug sirve para medir"	FALSO: siempre medir en profile, debug tiene overhead JIT
"Las imágenes siempre son lentas"	Con <code>cacheWidth</code> / <code>cacheHeight</code> y precache, son eficientes
"Debo optimizar todo el código"	Solo optimiza donde tengas un problema medible
"const no hace diferencia"	const puede reducir rebuilds un 50-80% en UIs estáticas
"RepaintBoundary siempre ayuda"	Agregar RepaintBoundary incorrectamente puede empeorar performance
"Los Slivers son siempre mejores"	Para listas simples, ListView.builder es suficiente

8. Comandos rápidos de referencia

```
# === EJECUCIÓN ===
flutter run --profile           # Profile mode (mediciones)
flutter run --release          # Release mode (producción)
flutter run -d chrome          # Web
flutter run -d <device-id>     # Dispositivo específico

# === ANÁLISIS ===
flutter analyze                # Análisis estático
flutter build apk --analyze-size # Tamaño del APK
flutter build ios --analyze-size # Tamaño del IPA

# === DEPENDENCIAS ===
flutter pub get                # Instalar dependencias
flutter pub upgrade            # Actualizar dependencias
flutter pub outdated           # Ver dependencias desactualizadas

# === LIMPIEZA ===
flutter clean                  # Limpiar build cache
flutter pub cache repair       # Reparar cache global

# === PERFORMANCE ===
flutter drive --target=test_driver/perf_test.dart # Performance test
```

9. Referencia rápida de const

```
// ✅ PUEDES usar const
const Text('literal')
const Icon(Icons.star)
const SizedBox(height: 16)
const Padding(
  padding: EdgeInsets.all(8.0),
  child: Text('estático'),
)
const [Text('a'), Text('b'), Text('c')]
```



```
// ❌ NO puedes usar const
Text(dynamicVariable)
Text('${user.name}')
Image.network('$url')
ElevatedButton(
  onPressed: () {}, // Closure no es const
  child: Text('Click'),
)
```

Resumen

Sección	Clave
Diagnóstico	Asocia cada síntoma a su causa y vista de DevTools
Métricas	Build/raster < 4ms, sin crecimiento de memoria
Pre-optimización	Medir siempre en profile mode
Anti-patrones	builder + const + dispose cubren el 90% de los casos
Mitos	Optimiza solo donde haya un problema medible

Referencias

- Flutter | Performance best practices — Guía oficial de buenas prácticas
- Flutter | Profiling modes — Por qué medir en profile
- Flutter | Performance debugging — Depuración de rendimiento de UI

 **Siguiente:** 24-practicas-optimizacion.md — Prácticas de optimización de rendimiento

24 — Prácticas de Optimización de Rendimiento

Escenarios reales de optimización: lista congelada, login con rebuilds excesivos, memory leaks, dashboards pesados, imágenes lentas y apps que crecen en memoria. Cada escenario incluye diagnóstico con DevTools y solución paso a paso.

1. Estructura de cada escenario

Cada escenario sigue el mismo patrón:

1. **Contexto:** el problema real que enfrentas
 2. **Problema:** código con el issue
 3. **Pasos:** cómo diagnosticar con DevTools
 4. **Solución:** código optimizado
 5. **Resultado esperado:** qué cambia en la app
-

2. Escenario 1: "Lista que se congela"

Contexto

Tienes una pantalla de contactos con 500 entradas. Cuando el usuario hace scroll, la lista se "congela" momentáneamente cada cierto tiempo. El Performance Overlay muestra frames rojos.

Problema

```
class ContactsScreen extends StatelessWidget {
  final List<Contact> contacts;

  const ContactsScreen({super.key, required this.contacts});

  @override
  Widget build(BuildContext context) {
    return ListView(
      children: contacts.map((contact) => ContactTile(
        contact: contact,
      )).toList(),
    );
  }
}

class ContactTile extends StatelessWidget {
  final Contact contact;

  const ContactTile({super.key, required this.contact});

  @override
  Widget build(BuildContext context) {
    return ListTile(
      leading: CircleAvatar(
        child: Text(contact.name[0]),
      ),
      title: Text(contact.name),
      subtitle: Text(contact.email),
      trailing: Icon(
        contact.isFavorite ? Icons.star : Icons.star_border,
      ),
    );
  }
}

class Contact {
  final String id;
  final String name;
  final String email;
  final bool isFavorite;

  Contact({
    required this.id,
    required this.name,
    required this.email,
    required this.isFavorite,
  });
}
```

Pasos de diagnóstico

1. Ejecutar `flutter run --profile`
2. Abrir DevTools > Performance
3. Grabar mientras se hace scroll (5 segundos)
4. Observar frames con build time > 8ms
5. Flame chart muestra que `ContactTile.build()` se ejecuta 500 veces al inicio

Solución

```
class ContactsScreen extends StatelessWidget {
  final List<Contact> contacts;

  const ContactsScreen({super.key, required this.contacts});

  @override
  Widget build(BuildContext context) {
    return ListView.builder( // <- Cambio #1: builder
      itemCount: contacts.length,
      itemBuilder: (context, index) {
        return ContactTile( // <- Ahora solo construye ~20 items
          contact: contacts[index],
        );
      },
    );
  }
}

class ContactTile extends StatelessWidget {
  final Contact contact;

  const ContactTile({super.key, required this.contact}); // <- const

  @override
  Widget build(BuildContext context) {
    return ListTile(
      leading: CircleAvatar(
        child: Text(contact.name[0]),
      ),
      title: Text(contact.name),
      subtitle: Text(contact.email),
      trailing: Icon(
        contact.isFavorite ? Icons.star : Icons.star_border,
      ),
    );
  }
}
```

Resultado esperado

Métrica	Antes	Después
Items contruidos	500	~20
Build time promedio	45ms	3ms
Memoria usada	~80MB	~15MB
Scroll fluidez	Congelada	Suave

3. Escenario 2: "Login con rebuilds excesivos"

Contexto

Un form de login donde cada tecla que escribe el usuario reconstruye toda la pantalla. El Performance Overlay muestra micro-jank continuo. El usuario escribe el email y el form entero parpadea.

Problema

```
class LoginScreen extends StatelessWidget {
  const LoginScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Padding(
        padding: const EdgeInsets.all(24),
        child: BlocBuilder<LoginBloc, LoginState>(
          builder: (context, state) {
            return Form(
              child: Column(
                mainAxisAlignment: MainAxisAlignment.center,
                children: [
                  const Text(
                    'Bienvenido',
                    style: TextStyle(fontSize: 32, fontWeight: FontWeight.bold),
                  ),
                  const SizedBox(height: 48),
                  TextFormField(
                    decoration: const InputDecoration(
                      labelText: 'Email',
                      border: OutlineInputBorder(),
                    ),
                    onChanged: (value) {
                      context.read<LoginBloc>().add(LoginEmailChanged(value));
                    },
                  ),
                  const SizedBox(height: 16),
                  TextFormField(
                    obscureText: true,
                    decoration: const InputDecoration(
                      labelText: 'Password',
                      border: OutlineInputBorder(),
                    ),
                    onChanged: (value) {
                      context.read<LoginBloc>().add(LoginPasswordChanged(value));
                    },
                  ),
                  const SizedBox(height: 24),
                  if (state.isLoading)
                    const CircularProgressIndicator(),
                  const SizedBox(height: 16),
                  ElevatedButton(
                    onPressed: state.isValid
                      ? () {
                          context.read<LoginBloc>().add(LoginSubmitted());
                        }
                      : null,
                    child: const Text('Iniciar Sesión'),
                  ),
                  const SizedBox(height: 16),
                  Text(
                    state.errorMessage ?? '',
                    style: const TextStyle(color: Colors.red),
                  ),
                ],
              ),
            );
          },
        ),
      );
  }
}
```

```
    ),  
    );  
  }  
}
```

Pasos de diagnóstico

1. Ejecutar `flutter run --profile`
2. DevTools > Performance > Grabar mientras se escribe
3. Observar que `LoginScreen.build()` se ejecuta en cada tecla
4. Enhance Tracing > Widget build rebuilds: todas las widgets se reconstruyen

Solución

```
class LoginScreen extends StatelessWidget {
  const LoginScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Padding(
        padding: const EdgeInsets.all(24),
        child: Form(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              const Text(
                'Bienvenido',
                style: TextStyle(fontSize: 32, fontWeight: FontWeight.bold),
              ),
              const SizedBox(height: 48),
              TextFormField(
                decoration: const InputDecoration(
                  labelText: 'Email',
                  border: OutlineInputBorder(),
                ),
                onChanged: (value) {
                  context.read<LoginBloc>().add(LoginEmailChanged(value));
                },
              ),
              const SizedBox(height: 16),
              TextFormField(
                obscureText: true,
                decoration: const InputDecoration(
                  labelText: 'Password',
                  border: OutlineInputBorder(),
                ),
                onChanged: (value) {
                  context.read<LoginBloc>().add(LoginPasswordChanged(value));
                },
              ),
              const SizedBox(height: 24),
              // Solo el indicador y el botón usan el estado
              const _LoginActions(),
            ],
          ),
        ),
      ),
    );
  }
}

class _LoginActions extends StatelessWidget {
  const _LoginActions();

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        BlocSelector<LoginBloc, LoginState, bool>(
          selector: (state) => state.isLoading,
          builder: (context, isLoading) {
            return isLoading
              ? const CircularProgressIndicator()
              : const SizedBox.shrink();
          },
        ),
      ],
    );
  }
}
```

```

    },
  ),
  const SizedBox(height: 16),
  BlocSelector<LoginBloc, LoginState, bool>({
    selector: (state) => state.isValid,
    builder: (context, isValid) {
      return ElevatedButton(
        onPressed: isValid
          ? () {
              context.read<LoginBloc>().add(LoginSubmitted());
            }
          : null,
        child: const Text('Iniciar Sesión'),
      );
    },
  ),
  const SizedBox(height: 16),
  BlocSelector<LoginBloc, LoginState, String?>({
    selector: (state) => state.errorMessage,
    builder: (context, errorMessage) {
      return Text(
        errorMessage ?? '',
        style: const TextStyle(color: Colors.red),
      );
    },
  ),
),
],
);
}
}

```

Resultado esperado

Métrica	Antes	Después
Widgets rebuild por tecla	~15	3
Build time promedio	12ms	1ms
UI response time	45ms	8ms
Parpadeo visible	Sí	No

4. Escenario 3: "Chat con memory leak"

Contexto

Un app de chat que escucha mensajes en tiempo real vía Supabase. Después de navegar entre pantallas 10-15 veces, la app se vuelve lenta y eventualmente crashea. DevTools Memory muestra memoria creciente sin límites.

Problema

```
class ChatScreen extends StatefulWidget {
  final String roomId;

  const ChatScreen({super.key, required this.roomId});

  @override
  State<ChatScreen> createState() => _ChatScreenState();
}

class _ChatScreenState extends State<ChatScreen> {
  final List<Message> _messages = [];
  final TextEditingController _controller = TextEditingController();

  @override
  void initState() {
    super.initState();
    // LEAK: StreamSubscription nunca se cancela
    Supabase.instance.client
      .from('messages')
      .eq('room_id', widget.roomId)
      .stream(primaryKey: ['id'])
      .order('created_at')
      .listen((messages) {
        setState(() {
          _messages.clear();
          _messages.addAll(
            messages.map((m) => Message.fromJson(m)),
          );
        });
      });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Chat')),
      body: Column(
        children: [
          Expanded(
            child: ListView.builder(
              itemCount: _messages.length,
              itemBuilder: (context, index) {
                return MessageBubble(message: _messages[index]);
              },
            ),
          ),
          _buildInput(),
        ],
      ),
    );
  }

  Widget _buildInput() {
    return Padding(
      padding: const EdgeInsets.all(8),
      child: Row(
        children: [
          Expanded(
            child: TextField(
              controller: _controller,
              decoration: const InputDecoration(
```

```

        hintText: 'Escribe un mensaje...',
        border: OutlineInputBorder(),
      ),
    ),
  ),
  IconButton(
    icon: const Icon(Icons.send),
    onPressed: () {
      _sendMessage(_controller.text);
      _controller.clear();
    },
  ),
],
),
);
}

void _sendMessage(String text) {
  Supabase.instance.client.from('messages').insert({
    'room_id': widget.roomId,
    'content': text,
  });
}

// LEAK: No hay dispose(), controllers no se cierran
}

class Message {
  final String id;
  final String content;
  final DateTime createdAt;

  Message({required this.id, required this.content, required this.createdAt});

  factory Message.fromJson(Map<String, dynamic> json) {
    return Message(
      id: json['id'],
      content: json['content'],
      createdAt: DateTime.parse(json['created_at']),
    );
  }
}

class MessageBubble extends StatelessWidget {
  final Message message;
  const MessageBubble({super.key, required this.message});

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 4),
      child: Align(
        alignment: Alignment.centerLeft,
        child: Container(
          padding: const EdgeInsets.all(12),
          decoration: BoxDecoration(
            color: Colors.blue[100],
            borderRadius: BorderRadius.circular(12),
          ),
          child: Text(message.content),
        ),
      ),
    );
  }
}

```

```
}  
}
```

Pasos de diagnóstico

1. Ejecutar `flutter run --profile`
2. DevTools > Memory > Tomar snapshot base
3. Navegar al chat y de vuelta 10 veces
4. Tomar diff snapshot
5. Buscar `ChatScreen` o `Message` en las clases con crecimiento
6. Cada navegación crea una nueva instancia de `ChatScreen` con stream abierto

Solución

```
class ChatScreen extends StatefulWidget {
  final String roomId;

  const ChatScreen({super.key, required this.roomId});

  @override
  State<ChatScreen> createState() => _ChatScreenState();
}

class _ChatScreenState extends State<ChatScreen> {
  final List<Message> _messages = [];
  final TextEditingController _controller = TextEditingController();
  StreamSubscription<List<Map<String, dynamic>>>? _messagesSubscription;

  @override
  void initState() {
    super.initState();
    _messagesSubscription = Supabase.instance.client
      .from('messages')
      .eq('room_id', widget.roomId)
      .stream(primaryKey: ['id'])
      .order('created_at')
      .listen((messages) {
        if (mounted) { // <- Verificar que el widget sigue vivo
          setState(() {
            _messages.clear();
            _messages.addAll(
              messages.map((m) => Message.fromJson(m)),
            );
          });
        }
      });
  }

  @override
  void dispose() {
    _messagesSubscription?.cancel(); // <- Cancelar stream
    _controller.dispose();           // <- Cerrar controller
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Chat')),
      body: Column(
        children: [
          Expanded(
            child: ListView.builder(
              itemCount: _messages.length,
              itemBuilder: (context, index) {
                return MessageBubble(message: _messages[index]);
              },
            ),
          ),
          _buildInput(),
        ],
      ),
    );
  }
}
```

```
Widget _buildInput() {
  return Padding(
    padding: const EdgeInsets.all(8),
    child: Row(
      children: [
        Expanded(
          child: TextField(
            controller: _controller,
            decoration: const InputDecoration(
              hintText: 'Escribe un mensaje...',
              border: OutlineInputBorder(),
            ),
          ),
        ),
        IconButton(
          icon: const Icon(Icons.send),
          onPressed: () {
            _sendMessage(_controller.text);
            _controller.clear();
          },
        ),
      ],
    ),
  );
}

void _sendMessage(String text) {
  Supabase.instance.client.from('messages').insert({
    'room_id': widget.roomId,
    'content': text,
  });
}
```

Resultado esperado

Métrica	Antes	Después
Memory después de 15 nav	Creciente sin límite	Estable
StreamSubscription activas	Múltiples (leak)	1 por pantalla
Crash después de uso prolongado	Sí	No
Memoria al salir de pantalla	No se libera	Se libera

5. Escenario 4: "Dashboard con widgets pesados"

Contexto

Un dashboard con múltiples secciones: gráficos, tablas, métricas en tiempo real. Cada actualización de datos reconstruye toda la pantalla. El Performance Overlay muestra jank constante.

Problema

```
class DashboardScreen extends StatelessWidget {
  const DashboardScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: SingleChildScrollView(
        child: BlocBuilder<DashboardBloc, DashboardState>(
          builder: (context, state) {
            return Column(
              children: [
                _buildMetricsRow(state), // Widget pesado
                _buildSalesChart(state), // Widget muy pesado
                _buildRecentOrders(state), // Widget pesado
                _buildInventoryTable(state), // Widget muy pesado
                _buildActivityFeed(state), // Widget mediano
              ],
            );
          },
        ),
      ),
    );
  }

  Widget _buildMetricsRow(DashboardState state) {
    return Padding(
      padding: const EdgeInsets.all(16),
      child: Row(
        children: [
          MetricCard(title: 'Ventas', value: state.totalSales),
          MetricCard(title: 'Usuarios', value: state.totalUsers),
          MetricCard(title: 'Pedidos', value: state.totalOrders),
        ],
      ),
    );
  }

  Widget _buildSalesChart(DashboardState state) {
    return Container(
      height: 300,
      padding: const EdgeInsets.all(16),
      child: CustomPaint(
        painter: SalesChartPainter(state.salesData),
        size: Size.infinite,
      ),
    );
  }

  Widget _buildRecentOrders(DashboardState state) {
    return ListView.builder(
      shrinkWrap: true,
      physics: const NeverScrollableScrollPhysics(),
      itemCount: state.recentOrders.length,
      itemBuilder: (context, index) {
        return OrderTile(order: state.recentOrders[index]);
      },
    );
  }

  Widget _buildInventoryTable(DashboardState state) {
    return DataTable(
```



```

        columns: const [
          DataColumn(label: Text('Producto')),
          DataColumn(label: Text('Stock')),
          DataColumn(label: Text('Precio')),
        ],
        rows: state.inventory.map((item) {
          return DataRow(cells: [
            DataCell(Text(item.name)),
            DataCell(Text('${item.stock}')),
            DataCell(Text('\${item.price}')),
          ]);
        }).toList(),
      );
    }

    Widget _buildActivityFeed(DashboardState state) {
      return ListView.builder(
        shrinkWrap: true,
        physics: const NeverScrollableScrollPhysics(),
        itemCount: state.activities.length,
        itemBuilder: (context, index) {
          return ActivityTile(activity: state.activities[index]);
        },
      );
    }
  }
}

```

Pasos de diagnóstico

1. Ejecutar `flutter run --profile`
2. DevTools > Performance > Grabar actualizaciones del dashboard
3. Enhance Tracing > Ver que `DashboardScreen.build()` se ejecuta completo
4. Flame chart: `SalesChartPainter.paint()` domina el tiempo

Solución

```
class DashboardScreen extends StatelessWidget {
  const DashboardScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: SingleChildScrollView(
        child: Column(
          children: const [
            _MetricsSection(),
            _ChartSection(),
            _OrdersSection(),
            _InventorySection(),
            _ActivitySection(),
          ],
        ),
      ),
    );
  }
}

class _MetricsSection extends StatelessWidget {
  const _MetricsSection();

  @override
  Widget build(BuildContext context) {
    return BlocSelector<DashboardBloc, DashboardState, DashboardMetrics>(
      selector: (state) => DashboardMetrics(
        sales: state.totalSales,
        users: state.totalUsers,
        orders: state.totalOrders,
      ),
      builder: (context, metrics) {
        return Padding(
          padding: const EdgeInsets.all(16),
          child: Row(
            children: [
              MetricCard(title: 'Ventas', value: metrics.sales),
              MetricCard(title: 'Usuarios', value: metrics.users),
              MetricCard(title: 'Pedidos', value: metrics.orders),
            ],
          ),
        );
      },
    );
  }
}

class _ChartSection extends StatelessWidget {
  const _ChartSection();

  @override
  Widget build(BuildContext context) {
    return RepaintBoundary(
      child: BlocSelector<DashboardBloc, DashboardState, List<SalesData>>(
        selector: (state) => state.salesData,
        builder: (context, salesData) {
          return Container(
            height: 300,
            padding: const EdgeInsets.all(16),
            child: CustomPaint(
```

```

        painter: SalesChartPainter(salesData),
        size: Size.infinite,
      ),
    );
  },
),
);
}
}

class _OrdersSection extends StatelessWidget {
  const _OrdersSection();

  @override
  Widget build(BuildContext context) {
    return BlocSelector<DashboardBloc, DashboardState, List<Order>>>(
      selector: (state) => state.recentOrders,
      builder: (context, orders) {
        return ListView.builder(
          shrinkWrap: true,
          physics: const NeverScrollableScrollPhysics(),
          itemCount: orders.length,
          itemBuilder: (context, index) {
            return OrderTile(order: orders[index]);
          },
        );
      },
    );
  }
}

class _InventorySection extends StatelessWidget {
  const _InventorySection();

  @override
  Widget build(BuildContext context) {
    return BlocSelector<DashboardBloc, DashboardState, List<InventoryItem>>>(
      selector: (state) => state.inventory,
      builder: (context, inventory) {
        return DataTable(
          columns: const [
            DataColumn(label: Text('Producto')),
            DataColumn(label: Text('Stock')),
            DataColumn(label: Text('Precio')),
          ],
          rows: inventory.map((item) {
            return DataRow(cells: [
              DataCell(Text(item.name)),
              DataCell(Text('${item.stock}')),
              DataCell(Text('\${item.price}')),
            ]);
          }).toList(),
        );
      },
    );
  }
}

class _ActivitySection extends StatelessWidget {
  const _ActivitySection();

  @override
  Widget build(BuildContext context) {
    return BlocSelector<DashboardBloc, DashboardState, List<Activity>>>(

```

```
selector: (state) => state.activities,
builder: (context, activities) {
  return ListView.builder(
    shrinkWrap: true,
    physics: const NeverScrollableScrollPhysics(),
    itemCount: activities.length,
    itemBuilder: (context, index) {
      return ActivityTile(activity: activities[index]);
    },
  );
},
);
}
```

Resultado esperado

Métrica	Antes	Después
Widgets rebuild por update	Todo el árbol	Solo la sección que cambió
Build time promedio	35ms	6ms
Chart repaint	Completo	Solo chart (RepaintBoundary)
UI response time	50ms	10ms

6. Escenario 5: "Imagen que congela al cargar"

Contexto

Una pantalla de perfil con 20 fotos de usuario que se cargan de Supabase Storage. Al hacer scroll, la app se congela. Las imágenes tardan mucho en aparecer.

Problema

```
class GalleryScreen extends StatelessWidget {
  final List<String> imageUrls;

  const GalleryScreen({super.key, required this.imageUrls});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Galería')),
      body: GridView.builder(
        gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
          crossAxisCount: 3,
          crossAxisSpacing: 4,
          mainAxisSpacing: 4,
        ),
        itemCount: imageUrls.length,
        itemBuilder: (context, index) {
          return Image.network(
            imageUrls[index],
            fit: BoxFit.cover,
            // Sin cacheWidth/cacheHeight: decodifica imagen completa
          );
        },
      ),
    );
  }
}
```

Pasos de diagnóstico

1. Ejecutar `flutter run --profile`
2. DevTools > Memory > Ver picos de memoria al hacer scroll
3. Performance > Raster time elevado cuando cargan imágenes
4. Flame chart: `decodeImageFromList` domina el tiempo

Solución

```
class GalleryScreen extends StatelessWidget {
  final List<String> imageUrls;

  const GalleryScreen({super.key, required this.imageUrls});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Galería')),
      body: GridView.builder(
        gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
          crossAxisCount: 3,
          crossAxisSpacing: 4,
          mainAxisSpacing: 4,
        ),
        itemCount: imageUrls.length,
        itemBuilder: (context, index) {
          return _GalleryImage(url: imageUrls[index]);
        },
      ),
    );
  }
}

class _GalleryImage extends StatelessWidget {
  final String url;

  const _GalleryImage({required this.url});

  @override
  Widget build(BuildContext context) {
    // Calcular tamaño real en píxeles basado en el devicePixelRatio
    final pixelRatio = MediaQuery.devicePixelRatioOf(context);
    final displaySize = 120.0; // Tamaño en layout points

    return Image.network(
      url,
      fit: BoxFit.cover,
      width: displaySize,
      height: displaySize,
      cacheWidth: (displaySize * pixelRatio).toInt(),
      cacheHeight: (displaySize * pixelRatio).toInt(),
      errorBuilder: (context, error, stackTrace) {
        return Container(
          color: Colors.grey[300],
          child: const Icon(Icons.broken_image),
        );
      },
      loadingBuilder: (context, child, loadingProgress) {
        if (loadingProgress == null) return child;
        return const Center(child: CircularProgressIndicator());
      },
    );
  }
}
```

Resultado esperado

Métrica	Antes	Después
Memoria por imagen	~12MB	~120KB
Raster time por imagen	25ms	2ms
Total memoria galería	~240MB	~2.4MB
Scroll fluidez	Congelada	Suave

7. Escenario 6: "App que crece en memoria"

Contexto

Una app con múltiples pantallas que usan formularios. Después de 20 minutos de uso, la app se vuelve lenta y consume mucha RAM. El usuario reporta que "la app se pone lenta con el tiempo".

Problema

```
class ProfileScreen extends StatefulWidget {
  const ProfileScreen({super.key});

  @override
  State<ProfileScreen> createState() => _ProfileScreenState();
}

class _ProfileScreenState extends State<ProfileScreen> {
  final _nameController = TextEditingController();
  final _emailController = TextEditingController();
  final _bioController = TextEditingController();
  final _phoneController = TextEditingController();
  final _addressController = TextEditingController();
  final _scrollController = ScrollController();

  @override
  void initState() {
    super.initState();
    _loadProfile();
    _scrollController.addListener(_onScroll);
  }

  void _loadProfile() {
    // Cargar datos del perfil
    _nameController.text = 'Juan';
    _emailController.text = 'juan@email.com';
  }

  void _onScroll() {
    // Lógica de scroll
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Perfil')),
      body: SingleChildScrollView(
        controller: _scrollController,
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(
              controller: _nameController,
              decoration: const InputDecoration(labelText: 'Nombre'),
            ),
            TextField(
              controller: _emailController,
              decoration: const InputDecoration(labelText: 'Email'),
            ),
            TextField(
              controller: _bioController,
              decoration: const InputDecoration(labelText: 'Bio'),
            ),
            TextField(
              controller: _phoneController,
              decoration: const InputDecoration(labelText: 'Teléfono'),
            ),
            TextField(
              controller: _addressController,
              decoration: const InputDecoration(labelText: 'Dirección'),
            ),
          ],
        ),
      ),
    );
  }
}
```



```

        ],
      ),
    ),
  );
}

// No hay dispose() - TODOS los controllers se filtran
}

class SettingsScreen extends StatefulWidget {
  const SettingsScreen({super.key});

  @override
  State<SettingsScreen> createState() => _SettingsScreenState();
}

class _SettingsScreenState extends State<SettingsScreen> {
  late Timer _refreshTimer;

  @override
  void initState() {
    super.initState();
    // Timer que nunca se cancela
    _refreshTimer = Timer.periodic(
      const Duration(seconds: 30),
      (_) => _refreshSettings(),
    );
  }

  void _refreshSettings() {
    // Refrescar configuración
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Configuración')),
      body: const Center(child: Text('Settings')),
    );
  }

  // No hay dispose() - Timer sin cancelar + listener leak
}

```

Pasos de diagnóstico

1. Ejecutar `flutter run --profile`
2. DevTools > Memory > Tomar snapshot base
3. Navegar entre ProfileScreen y SettingsScreen 15 veces
4. Tomar diff snapshot
5. Clases con crecimiento: `TextEditingController`, `Timer`, `ScrollController`
6. Verificar: `ProfileScreen` y `SettingsScreen` no tienen `dispose()`

Solución

```
class ProfileScreen extends StatefulWidget {
  const ProfileScreen({super.key});

  @override
  State<ProfileScreen> createState() => _ProfileScreenState();
}

class _ProfileScreenState extends State<ProfileScreen> {
  final _nameController = TextEditingController();
  final _emailController = TextEditingController();
  final _bioController = TextEditingController();
  final _phoneController = TextEditingController();
  final _addressController = TextEditingController();
  final _scrollController = ScrollController();

  @override
  void initState() {
    super.initState();
    _loadProfile();
    _scrollController.addListener(_onScroll);
  }

  void _loadProfile() {
    _nameController.text = 'Juan';
    _emailController.text = 'juan@email.com';
  }

  void _onScroll() {
    // Lógica de scroll
  }

  @override
  void dispose() {
    // Remover listener antes de dispose
    _scrollController.removeListener(_onScroll);

    // Cerrar todos los controllers
    _scrollController.dispose();
    _nameController.dispose();
    _emailController.dispose();
    _bioController.dispose();
    _phoneController.dispose();
    _addressController.dispose();

    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Perfil')),
      body: SingleChildScrollView(
        controller: _scrollController,
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(
              controller: _nameController,
              decoration: const InputDecoration(labelText: 'Nombre'),
            ),
            TextField(
```

```

        controller: _emailController,
        decoration: const InputDecoration(labelText: 'Email'),
      ),
      TextField(
        controller: _bioController,
        decoration: const InputDecoration(labelText: 'Bio'),
      ),
      TextField(
        controller: _phoneController,
        decoration: const InputDecoration(labelText: 'Teléfono'),
      ),
      TextField(
        controller: _addressController,
        decoration: const InputDecoration(labelText: 'Dirección'),
      ),
    ],
  ),
),
);
}
}

```

```

class SettingsScreen extends StatefulWidget {
  const SettingsScreen({super.key});

  @override
  State<SettingsScreen> createState() => _SettingsScreenState();
}

```

```

class _SettingsScreenState extends State<SettingsScreen> {
  late Timer _refreshTimer;

  @override
  void initState() {
    super.initState();
    _refreshTimer = Timer.periodic(
      const Duration(seconds: 30),
      (_) => _refreshSettings(),
    );
  }

  void _refreshSettings() {
    // Refrescar configuración
  }

  @override
  void dispose() {
    _refreshTimer.cancel(); // <- Cancelar timer
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Configuración')),
      body: const Center(child: Text('Settings')),
    );
  }
}

```

Resultado esperado

Métrica	Antes	Después
Memory después de 20 min	Creciente continuamente	Estable
Controllers activos (leak)	Múltiples por pantalla	0 después de dispose
Timers activos (leak)	Múltiples	0 después de dispose
GC frequency	> 5/min	< 1/min

8. Ejercicio integrador: Diagnosticar una app "enferma"

Contexto

Recibes una app Flutter de un proyecto existente. El equipo reporta múltiples problemas:

- 1. El feed principal se congela al hacer scroll
- 2. La pantalla de login parpadea cuando escribes
- 3. La memoria crece continuamente durante el uso
- 4. Las imágenes del perfil tardan mucho en cargar

Tu tarea es **diagnosticar y corregir cada problema**.

Archivos del proyecto

```
// === lib/models/post.dart ===
class Post {
  final String id;
  final String title;
  final String content;
  final String authorId;
  final DateTime createdAt;

  Post({
    required this.id,
    required this.title,
    required this.content,
    required this.authorId,
    required this.createdAt,
  });

  factory Post.fromJson(Map<String, dynamic> json) {
    return Post(
      id: json['id'],
      title: json['title'],
      content: json['content'],
      authorId: json['author_id'],
      createdAt: DateTime.parse(json['created_at']),
    );
  }
}

// === lib/blocs/feed/feed_state.dart ===
class FeedState {
  final List<Post> posts;
  final bool isLoading;
  final String? error;

  FeedState({
    this.posts = const [],
    this.isLoading = false,
    this.error,
  });
}

// === lib/screens/feed_screen.dart ===
class FeedScreen extends StatefulWidget {
  const FeedScreen({super.key});

  @override
  State<FeedScreen> createState() => _FeedScreenState();
}

class _FeedScreenState extends State<FeedScreen> {
  StreamSubscription? _postsSubscription;
  final List<Post> _posts = [];

  @override
  void initState() {
    super.initState();
    _postsSubscription = Supabase.instance.client
      .from('posts')
      .stream(primaryKey: ['id'])
      .order('created_at', ascending: false)
      .listen((data) {
        setState(() {

```

```

        _posts.clear();
        _posts.addAll(data.map((p) => Post.fromJson(p)));
    });
});
}

@override
Widget build(BuildContext context) {
    return Scaffold(
        appBar: AppBar(title: const Text('Feed')),
        body: ListView(
            children: _posts.map((post) => PostCard(post: post)).toList(),
        ),
    );
}

class PostCard extends StatelessWidget {
    final Post post;
    const PostCard({super.key, required this.post});

    @override
    Widget build(BuildContext context) {
        return Card(
            margin: const EdgeInsets.all(8),
            child: Padding(
                padding: const EdgeInsets.all(16),
                child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                        Row(
                            children: [
                                CircleAvatar(
                                    child: Image.network(
                                        'https://api.dicebear.com/7.x/avataaars/svg?seed=${post.authorId}',
                                    ),
                                ),
                                const SizedBox(width: 12),
                                Text(post.title, style: const TextStyle(fontWeight: FontWeight.bold)),
                            ],
                        ),
                        const SizedBox(height: 8),
                        Text(post.content),
                        const SizedBox(height: 8),
                        Text('${post.createdAt}', style: TextStyle(color: Colors.grey[600])),
                    ],
                ),
            ),
        );
    }
}

// === lib/screens/login_screen.dart ===
class LoginScreen extends StatelessWidget {
    const LoginScreen({super.key});

    @override
    Widget build(BuildContext context) {
        return Scaffold(
            body: BlocBuilder<AuthBloc, AuthState>(
                builder: (context, state) {
                    return Padding(
                        padding: const EdgeInsets.all(24),
                        child: Column(

```

```

        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          const Text('Login', style: TextStyle(fontSize: 32)),
          const SizedBox(height: 32),
          TextField(
            onChanged: (v) => context.read<AuthBloc>().add(EmailChanged(v)),
          ),
          const SizedBox(height: 16),
          TextField(
            obscureText: true,
            onChanged: (v) => context.read<AuthBloc>().add>PasswordChanged(v)),
          ),
          const SizedBox(height: 24),
          if (state.isLoading) const CircularProgressIndicator(),
          ElevatedButton(
            onPressed: state.isValid ? () => context.read<AuthBloc>().add(LoginSubmitted()) :
null,
            child: const Text('Entrar'),
          ),
          Text(state.error ?? '', style: const TextStyle(color: Colors.red)),
        ],
      ),
    ),
  },
),
);
},
),
);
}
}

```

```
// === lib/screens/profile_screen.dart ===
```

```
class ProfileScreen extends StatefulWidget {
  const ProfileScreen({super.key});
```

```

  @override
  State<ProfileScreen> createState() => _ProfileScreenState();
}

```

```

class _ProfileScreenState extends State<ProfileScreen> {
  final _nameController = TextEditingController();
  final _bioController = TextEditingController();
  final _urlController = TextEditingController();

```

```

  @override
  void initState() {
    super.initState();
    _loadProfile();
  }

```

```

  void _loadProfile() {
    _nameController.text = 'Usuario';
    _bioController.text = 'Bio del usuario';
  }

```

```

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Perfil')),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            Image.network('https://api.dicebear.com/7.x/avataaars/svg?seed=user'),
            const SizedBox(height: 16),
            TextField(controller: _nameController, decoration: const InputDecoration(labelText:

```

```

'Nombre')),
      TextField(controller: _bioController, decoration: const InputDecoration(labelText:
'Bio')),
      TextField(controller: _urlController, decoration: const InputDecoration(labelText: 'URL
foto')),
    ],
  ),
),
);
}
}

```

Guía de diagnóstico

Sigue estos pasos para encontrar y corregir cada problema:

PROBLEMA 1: Feed se congela al hacer scroll

- └─ Diagnóstico: ListView() construye TODOS los posts
- └─ Solución: Cambiar a ListView.builder
- └─ Bonus: Agregar const a PostCard si es posible
- └─ Verificar: Performance > build time < 8ms

PROBLEMA 2: Login parpadea al escribir

- └─ Diagnóstico: BlocBuilder reconstruye todo el form
- └─ Solución: BlocSelector para botón + indicador
- └─ Verificar: Enhance Tracing > menos rebuilds
- └─ Resultado: Build time < 2ms por tecla

PROBLEMA 3: Memoria crece continuamente

- └─ Diagnóstico: FeedScreen no tiene dispose()
- └─ Solución: Agregar dispose() con cancel de subscription
- └─ Bonus: ProfileScreen no tiene dispose() de controllers
- └─ Verificar: Memory > diff snapshot sin crecimiento
- └─ Resultado: Memoria estable después de 15 nav

PROBLEMA 4: Imágenes del perfil tardan

- └─ Diagnóstico: Image.network sin cacheWidth/cacheHeight
- └─ Solución: Agregar cacheWidth/cacheHeight
- └─ Bonus: precacheImage en initState
- └─ Verificar: Memory > menos uso por imagen
- └─ Resultado: Imágenes cargan instantáneamente

Solución completa

```
// === lib/screens/feed_screen.dart (CORREGIDO) ===
class FeedScreen extends StatefulWidget {
  const FeedScreen({super.key});

  @override
  State<FeedScreen> createState() => _FeedScreenState();
}

class _FeedScreenState extends State<FeedScreen> {
  StreamSubscription? _postsSubscription;
  final List<Post> _posts = [];

  @override
  void initState() {
    super.initState();
    _postsSubscription = Supabase.instance.client
      .from('posts')
      .stream(primaryKey: ['id'])
      .order('created_at', ascending: false)
      .listen((data) {
        if (mounted) {
          setState(() {
            _posts.clear();
            _posts.addAll(data.map((p) => Post.fromJson(p)));
          });
        }
      });
  }

  @override
  void dispose() {
    _postsSubscription?.cancel(); // CORRECCIÓN #1: cancelar subscription
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Feed')),
      body: ListView.builder( // CORRECCIÓN #2: ListView.builder
        itemCount: _posts.length,
        itemBuilder: (context, index) {
          return PostCard(post: _posts[index]);
        },
      ),
    );
  }
}

// === lib/screens/login_screen.dart (CORREGIDO) ===
class LoginScreen extends StatelessWidget {
  const LoginScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Padding(
        padding: const EdgeInsets.all(24),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
```

```

const Text('Login', style: TextStyle(fontSize: 32)),
const SizedBox(height: 32),
TextField(
  onChanged: (v) => context.read<AuthBloc>().add(EmailChanged(v)),
),
const SizedBox(height: 16),
TextField(
  obscureText: true,
  onChanged: (v) => context.read<AuthBloc>().add>PasswordChanged(v)),
),
const SizedBox(height: 24),
// CORRECCIÓN #3: BlocSelector para indicador
BlocSelector<AuthBloc, AuthState, bool>(
  selector: (state) => state.isLoading,
  builder: (context, isLoading) {
    return isLoading
      ? const CircularProgressIndicator()
      : const SizedBox.shrink();
  },
),
const SizedBox(height: 16),
// CORRECCIÓN #4: BlocSelector para botón
BlocSelector<AuthBloc, AuthState, bool>(
  selector: (state) => state.isValid,
  builder: (context, isValid) {
    return ElevatedButton(
      onPressed: isValid
        ? () => context.read<AuthBloc>().add(LoginSubmitted())
        : null,
      child: const Text('Entrar'),
    );
  },
),
// CORRECCIÓN #5: BlocSelector para error
BlocSelector<AuthBloc, AuthState, String?>(
  selector: (state) => state.error,
  builder: (context, error) {
    return Text(
      error ?? '',
      style: const TextStyle(color: Colors.red),
    );
  },
),
],
),
),
);
}
}

// === lib/screens/profile_screen.dart (CORREGIDO) ===
class ProfileScreen extends StatefulWidget {
  const ProfileScreen({super.key});

  @override
  State<ProfileScreen> createState() => _ProfileScreenState();
}

class _ProfileScreenState extends State<ProfileScreen> {
  final _nameController = TextEditingController();
  final _bioController = TextEditingController();
  final _urlController = TextEditingController();

  @override

```

```

void initState() {
  super.initState();
  _loadProfile();
}

void _loadProfile() {
  _nameController.text = 'Usuario';
  _bioController.text = 'Bio del usuario';
}

@override
void dispose() {
  // CORRECCIÓN #6: dispose de todos los controllers
  _nameController.dispose();
  _bioController.dispose();
  _urlController.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Perfil')),
    body: SingleChildScrollView(
      padding: const EdgeInsets.all(16),
      child: Column(
        children: [
          // CORRECCIÓN #7: cacheWidth/cacheHeight en imágenes
          Image.network(
            'https://api.dicebear.com/7.x/avataaars/svg?seed=user',
            width: 120,
            height: 120,
            cacheWidth: 240, // 120 * 2.0 devicePixelRatio
            cacheHeight: 240,
          ),
          const SizedBox(height: 16),
          TextField(controller: _nameController, decoration: const InputDecoration(labelText:
'Nombre')),
          TextField(controller: _bioController, decoration: const InputDecoration(labelText:
'Bio')),
          TextField(controller: _urlController, decoration: const InputDecoration(labelText: 'URL
foto')),
        ],
      ),
    ),
  );
}

```

Resumen de correcciones

#	Archivo	Problema	Corrección
1	feed_screen.dart	StreamSubscription sin cancelar	Agregar <code>dispose()</code> con <code>cancel()</code>
2	feed_screen.dart	<code>ListView</code> construye todos los items	Cambiar a <code>ListView.builder</code>
3	login_screen.dart	<code>BlocBuilder</code> reconstruye todo	<code>BlocSelector</code> para <code>isLoading</code>
4	login_screen.dart	Botón se reconstruye innecesariamente	<code>BlocSelector</code> para <code>isValid</code>

#	Archivo	Problema	Corrección
5	<code>login_screen.dart</code>	Texto de error se reconstruye	<code>BlocSelector</code> para error
6	<code>profile_screen.dart</code>	Controllers sin dispose	Agregar <code>dispose()</code> completo
7	<code>profile_screen.dart</code>	Imagen sin cacheWidth/cacheHeight	Agregar dimensiones en cache

Resumen

Escenario	Problema raíz	Técnica
Lista congelada	<code>ListView</code> construye todo	<code>ListView.builder</code>
Login parpadea	<code>BlocBuilder</code> reconstruye todo	<code>BlocSelector</code>
Chat con leak	Stream sin cancelar	<code>dispose()</code> + <code>cancel()</code>
Dashboard pesado	Una build gigante	Secciones + <code>BlocSelector</code>
Imágenes lentas	Decodificación completa	<code>cacheWidth</code> / <code>cacheHeight</code>
App que crece	Controllers/timers sin dispose	<code>dispose()</code> completo

La regla de oro: **mide en profile mode, encuentra el cuello de botella con DevTools y optimiza solo esa parte.**

Referencias

- Flutter | [Performance best practices](#) — Buenas prácticas oficiales de rendimiento
- Flutter | [Lazy loading de listas](#) — Por qué usar builders en listas
- [bloc](#) | `BlocSelector` — Rebuilds selectivos con BLoC

 **Siguiente:** [25-debugging-asincrono.md](#) — Debugging de código asíncrono en Dart

25 — Debugging Asíncrono en Dart/Flutter

El 80% de bugs difíciles en Flutter son problemas asíncronos. Futures que nunca completan, streams que pierden datos, microtasks que bloquean el UI thread.

1. Por qué es tan difícil

```
// Este código parece correcto pero tiene 3 bugs ocultos
Future<void> loadData() async {
  final user = await fetchUser();           // Bug 1: no maneja error
  final posts = await fetchPosts(user.id); // Bug 2: secuencial innecesariamente
  setState(() {
    _user = user;           // Bug 3: setState después de await sin mounted check
    _posts = posts;
  });
}
```

2. Los 5 bugs asíncronos más comunes

2.1 Future que nunca completa

```
// × BUG: Si fetch falla, el loading queda para siempre
Future<void> load() async {
  setState(() => _loading = true);
  final data = await fetch(); // Si lanza excepción...
  setState(() {
    _data = data;
    _loading = false; // ← Nunca se ejecuta
  });
}

// ✅ CORRECTO
Future<void> load() async {
  setState(() => _loading = true);
  try {
    final data = await fetch();
    setState(() {
      _data = data;
      _loading = false;
    });
  } catch (e) {
    setState(() {
      _error = e.toString();
      _loading = false;
    });
  }
}
```

2.2 setState sin mounted check

```
// × BUG: Widget desmontado mientras awaited
Future<void> load() async {
  final data = await fetch(); // 2 segundos
  setState(() { _data = data; }); // Widget puede no existir
}

// ✅ CORRECTO
Future<void> load() async {
  final data = await fetch();
  if (!mounted) return; // ← Verificar
  setState(() { _data = data; });
}
```

2.3 Llamadas secuenciales innecesarias

```
// × LENTO: Espera 4 segundos (2+2)
Future<void> loadAll() async {
  final users = await fetchUsers(); // 2 seg
  final posts = await fetchPosts(); // 2 seg (espera a que termine users)
}

// ✅ RÁPIDO: Espera 2 segundos (paralelo)
Future<void> loadAll() async {
  final results = await Future.wait([
    fetchUsers(),
    fetchPosts(),
  ]);
  final users = results[0];
  final posts = results[1];
}
```

2.4 Stream que se suscribe mal

```
// × BUG: Suscripción que nunca se cancela (memory leak)
void initState() {
  super.initState();
  _supabase.from('posts').stream(primaryKey: ['id']).listen((data) {
    setState(() => _posts = data);
  });
}

// ✅ CORRECTO
late final StreamSubscription _subscription;

@override
void initState() {
  super.initState();
  _subscription = _supabase
    .from('posts')
    .stream(primaryKey: ['id'])
    .listen((data) {
      if (mounted) setState(() => _posts = data);
    });
}

@override
void dispose() {
  _subscription.cancel(); // ← SIEMPRE cancelar
  super.dispose();
}
```

2.5 Completer manual innecesario

```
// × COMPLEJIDAD INNECESARIA
final completer = Completer<String>();
fetch().then((v) => completer.complete(v)).catchError((e) => completer.completeError(e));
final result = await completer.future;

// ✔ SIMPLE
final result = await fetch();
```

3. Patrón de carga segura

```
mixin SafeLoad<T> {
  bool _loading = false;
  String? _error;
  T? _data;

  bool get loading => _loading;
  String? get error => _error;
  T? get data => _data;

  Future<T> loadSafely(Future<T> Function() loader) async {
    _loading = true;
    _error = null;
    try {
      _data = await loader();
      _loading = false;
      return _data!;
    } catch (e) {
      _error = e.toString();
      _loading = false;
      rethrow;
    }
  }
}
```


4. Debugging de Futures

4.1 Prints de debug

```
Future<T> debugFuture<T>(String name, Future<T> Function() fn) async {
  print('🕒 [$name] Iniciando...');
  final sw = Stopwatch()..start();
  try {
    final result = await fn();
    print('✅ [$name] Completado en ${sw.elapsedMilliseconds}ms');
    return result;
  } catch (e) {
    print('❌ [$name] Error: $e');
    rethrow;
  }
}

// Uso
final user = await debugFuture('fetchUser', () => fetchUser());
```

4.2 Timeout

```
// Nunca dejes un Future sin timeout en producción
final data = await fetch().timeout(
  Duration(seconds: 10),
  onTimeout: () => throw TimeoutException('Fetch tardó más de 10s'),
);
```

5. Debugging de Streams

5.1 Inspeccionar eventos

```
// Envuelve el stream para ver cada evento
Stream<T> debugStream<T>(String name, Stream<T> stream) {
    return stream.transform(
        StreamTransformer.fromHandlers(
            handleData: (data, sink) {
                print('📦 [$name] evento: $data');
                sink.add(data);
            },
            handleError: (error, stack, sink) {
                print('✖ [$name] error: $error');
                sink.addError(error, stack);
            },
            handleDone: (sink) {
                print('🔌 [$name] stream cerrado');
                sink.close();
            },
        ),
    );
}

// Uso
final posts = debugStream('posts', _supabase.from('posts').stream(primaryKey: ['id']));
```

5.2 Detectar streams que nunca emiten

```
// Timeout sobre la primera emisión
final firstPost = await posts
    .first
    .timeout(
        const Duration(seconds: 5),
        onTimeout: () => throw TimeoutException('El stream no emitió en 5s'),
    );
```

6. Debugging de microtasks y event loop

6.1 Cuándo se ejecuta tu código

```
void debugEventLoop() {
  print('1: síncrono');

  Future(() => print('2: microtask')); // Microtask

  Future.delayed(Duration.zero, () {
    print('3: event loop (timer)');
  });

  Future.microtask(() => print('4: microtask explícita'));

  print('5: síncrono (último)');
}

// Orden de salida:
// 1: síncrono
// 5: síncrono (último)
// 2: microtask
// 4: microtask explícita
// 3: event loop (timer)
```

6.2 UI bloqueada por trabajo pesado síncrono

```
// × BLOQUEA el UI thread: el frame no se renderiza
void onPressed() {
  final bigList = List.generate(10_000_000, (i) => i);
  final sum = bigList.reduce((a, b) => a + b); // CPU bloqueada aquí
}

// ✅ Delegar a un isolate
void onPressed() async {
  final sum = await compute(_computeSum, 10_000_000);
}

int _computeSum(int n) {
  var total = 0;
  for (var i = 0; i < n; i++) {
    total += i;
  }
  return total;
}
```

7. Checklist asíncrono

- ☐ ¿Todos los `await` tienen `try/catch`?
- ☐ ¿Hay `mounted` check después de cada `await` en widgets?
- ☐ ¿Las llamadas independientes usan `Future.wait`?
- ☐ ¿Los streams se cancelan en `dispose`?
- ☐ ¿Hay `timeout` en llamadas de red?
- ☐ ¿Completer es realmente necesario? (probablemente no)
- ☐ ¿El trabajo pesado usa `compute()` o `isolates`?

Resumen

Problema	Síntoma	Solución
Future nunca completa	Loading infinito	<code>try/catch</code> + manejo de error
setState tras desmontar	Crash de framework	<code>if (!mounted) return</code>
Llamadas secuenciales	App lenta	<code>Future.wait</code>
Stream sin cancelar	Memory leak	<code>dispose()</code> + <code>cancel()</code>
Completer manual	Código complejo	Usar <code>await</code> directo
UI bloqueada	Jank al hacer scroll	<code>compute()</code> / <code>isolates</code>

Referencias

- [dart:async | Future API](#) — Futures y `async/await`
- [dart:async | Stream API](#) — Streams y suscripciones
- [Dart | Asynchronous programming](#) — Guía de programación asíncrona
- [Dart | Event loop y microtasks](#) — Cómo funciona el loop de eventos

 **Siguiente:** [26-workflow-debugging-por-tipo.md](#) — Workflow de debugging según el tipo de problema

26 — Workflow de Debugging por Tipo de Bug

Diferentes tipos de bugs requieren diferentes estrategias. Este archivo te da el workflow exacto para cada tipo: UI, estado, performance, red, memoria, compilación y hot reload.

1. Framework general de debugging

1. REPRODUCIR → ¿Cómo hago que pase otra vez?
2. AISLAR → ¿Dónde exactamente falla?
3. DIAGNOSTICAR → ¿Por qué falla?
4. CORREGIR → ¿Cómo lo arreglo sin romper algo?
5. VERIFICAR → ¿Sigue funcionando todo?

2. Tipo 1: UI Bug (algo se ve mal)

Flujo:

1. Widget Inspector (DevTools)
2. Select Widget Mode (VSCode: Ctrl+Shift+P / Cmd+Shift+P → Flutter: Select Widget)
3. Inspeccionar propiedades
4. Verificar constraints (SizedBox, Expanded, Flexible)
5. Revisar MediaQuery si es responsive

Herramienta: DevTools → Flutter Inspector → Select Widget

Desde código (sin DevTools):

```
// Ver bounds y constraints de cada widget
debugPaintSizeEnabled = true;

// Ver árbol de widgets completo
debugDumpWidgetTree();

// Ver qué se repinta
debugProfilePaintsEnabled = true;
```

Ver [28-debugging-programatico-flutter.md](#) para todas las herramientas programáticas.

Ejemplo:

```
// ✗ Widget se desborda
Container(
  width: double.infinity,
  height: 300,
  child: Text('Muy largo texto que no cabe y se desborda del contenedor'),
)

// ✔ Con overflow controlado
Container(
  width: double.infinity,
  child: Text(
    'Texto largo',
    maxLines: 2,
    overflow: TextOverflow.ellipsis,
  ),
)
```

3. Tipo 2: State Bug (estado inconsistente)

Flujo:

1. Agregar prints en cada cambio de estado
2. Verificar si mounted antes de setState
3. Revisar si hay múltiples setState en secuencia
4. Usar DevTools → extensión Bloc Inspector (si usa BLoC)
5. Verificar inicialización del state

Herramienta: Print debugging + Bloc Inspector

Ejemplo:

```
// ✗ Estado se resetea
class MyCubit extends Cubit<MyState> {
  MyCubit() : super(MyInitial());

  void load() async {
    emit>Loading(); // Emit loading
    final data = await fetch();
    // Si fetch lanza error, nunca llega aquí
    emit(Loaded(data));
  }
}

// ✔ Manejo correcto
void load() async {
  emit>Loading();
  try {
    final data = await fetch();
    emit(Loaded(data));
  } catch (e) {
    emit(Error(e.toString()));
  }
}
```

4. Tipo 3: Performance Bug (app lenta)

Flujo:

1. DevTools → Performance Overlay (Enable)
2. DevTools → Performance → Record
3. Buscar "jank" (frames > 16ms)
4. Identificar widget que causa rebuilds
5. Aplicar const, RepaintBoundary, o lazy loading

Herramienta: DevTools → Performance

Checklist:

- ☐ ¿Usas const en widgets que no cambian?
- ☐ ¿ListView.builder en vez de ListView?
- ☐ ¿Imágenes con BoxFit adecuado y cacheWidth/cacheHeight?
- ☐ ¿Evitas rebuilds innecesarios?
- ☐ ¿Usas RepaintBoundary en animations?

5. Tipo 4: Network Bug (API falla)

Flujo:

1. DevTools → Network → Ver request/response
2. Copiar como cURL para probar externamente
3. Verificar headers (Authorization, Content-Type)
4. Verificar status code (200, 400, 401, 403, 500)
5. Verificar body del request

Herramienta: DevTools → Network

Debugging de Supabase:

```
// Agregar logging
final supabase = SupabaseClient(url, key);

// Ver queries en consola
final data = await supabase
  .from('users')
  .select()
  .order('created_at', ascending: false);

// Imprimir query result
print('Query result: $data');
print('Rows: ${data.length}');
```

6. Tipo 5: Memory Leak

Flujo:

1. DevTools → Memory → Take Heap Snapshot
2. Hacer acción que debería liberar memoria
3. Take another snapshot
4. Comparar snapshots
5. Si objetos crecen → buscar suscripciones no canceladas

Herramienta: DevTools → Memory

Comunes:

```
// × Suscripción no cancelada
void initState() {
  _stream.listen((_) {}); // Nunca se cancela
}

// × Controller no disposed
final _controller = StreamController(); // Nunca se cierra

// × Timer no cancelado
Timer.periodic(Duration(seconds: 1), (_) {}); // Nunca se cancela
```

7. Tipo 6: Build Error (compilación)

Flujo:

1. Leer el error completo (no solo la primera línea)
2. Buscar "error:" en el output
3. Identificar archivo y línea
4. Verificar tipos de datos
5. Verificar imports
6. Verificar null safety

Errores comunes en Flutter:

```
// "RenderFlex overflowed" → Widget se desborda
// "Incorrect use of ParentDataWidget" → Expanded/Flexible sin ListView/Row/Column
// "A RenderFlex overflowed" → Text muy largo sin wrap
```

8. Tipo 7: Hot Reload no funciona

Flujo:

1. Verificar si el cambio es "hot reloadable"
2. Cambios en initState NO se recargan con hot reload
3. Cambios en main() NO se recargan
4. Cambios en imports NO se recargan
5. Usar Hot Restart (no Hot Reload)

No recarga con Hot Reload:

- `initState()`
- `main()`
- `const` values
- Imports
- Static fields

9. Cheat Sheet: ¿Qué herramienta para qué?

Bug	Herramienta	Atajo
UI se ve mal	Widget Inspector	DevTools → Flutter Inspector
Estado inconsistente	Bloc Inspector / Prints	—
App lenta	Performance View	DevTools → Performance
API falla	Network View	DevTools → Network
Memory leak	Memory View	DevTools → Memory
Build error	Terminal output	Leer error completo
Hot Reload no funciona	Hot Restart	R (consola) / Shift+F5 (VS Code)

Resumen

Tipo de bug	Primer paso	Herramienta principal
UI	Select Widget Mode	Flutter Inspector
Estado	Prints en cada cambio	Bloc Inspector
Performance	Performance Overlay	Performance View
Network	Ver request en Network	Network View
Memoria	Heap snapshot base	Memory View
Compilación	Leer el error completo	Terminal

Tipo de bug	Primer paso	Herramienta principal
Hot Reload	Cambiar a Hot Restart	Consola / VS Code

Este workflow cierra la guía: combina todo lo aprendido en los capítulos anteriores — DevTools, rendering, rendimiento, debugging de UI y código asíncrono — para diagnosticar y resolver cualquier problema que aparezca en tu app Flutter.

Para un enfoque paso-a-paso más sistemático, ver [29-playbook-debugging-sistematico.md](#). Para verificar **cada capa de Clean Architecture de forma aislada** mientras desarrollas una feature (modo andamiaje SDD), ver [31-debugging-por-capa.md](#).

Referencias

- Flutter | [Debugging tools](#) — Panorama de herramientas de debugging
- Flutter | [DevTools](#) — Documentación de DevTools
- Flutter | [Hot reload](#) — Qué se recarga y qué no

 **Volver al índice:** [README.md](#)

27 — Mentalidad de Debugging: Piensa como un Debugger, no como un Prompt

Antes de aprender herramientas, aprende a pensar. Este capítulo establece la filosofía que te hará independiente de la IA para diagnosticar bugs.

1. El problema real: debugging dependiente

La mayoría de developers en 2026 tienen este ciclo:

Bug aparece → Pedirle a la IA que lo resuelva → Copiar la solución → Siguiendo bug

Esto es problemático porque:

- **No entiendes POR QUÉ** algo falló, solo que "ahora funciona"
- **No sabes detectar** si la solución de la IA introduce nuevos bugs
- **No puedes distinguir** entre una solución correcta y una que "parece" correcta
- **Te vuelves más lento** sin IA, porque nunca desarrollaste la habilidad

El debugging no es una herramienta. Es una **mentalidad**.

2. La filosofía: "No adivines, observa"

Un desarrollador principiante adivina:

"Creo que el problema es que el BLoC no emite el estado"

Un desarrollador experimentado observa:

"BLoC emite LoginLoading en la línea 195. Breakpoint ahí. event.email es null.
El problema está en el handler del formulario, no en el BLoC."

La diferencia es **evidencia vs hipótesis**.

2.1 Regla #1: Nunca supongas, verifica

✗ Suposición	✓ Verificación
"Creo que es un null"	Inspecciona la variable: ¿es null?
"Creo que el estado no cambia"	Pon un breakpoint en el emit: ¿se ejecuta?
"Creo que la API falla"	Mira Network View: ¿qué status code devuelve?
"Creo que es un rebuild innecesario"	Agrega <code>debugPrintRebuildDirtyWidgets</code> : ¿quién rebuilda?

2.2 Regla #2: Reduce el espacio de búsqueda

Cuando no sabes dónde está el bug, no revises todo el código. Usa **binary search debugging**:

Tu app tiene 100 archivos. El bug está en alguno.

1. Pon un breakpoint a mitad del flujo (archivo 50)
2. ¿El bug está antes o después?
3. Repite con la mitad correcta
4. En 7 pasos ($\log_2$ de 100) encuentras el archivo exacto

2.3 Regla #3: Reproduce antes de arreglar

Si no puedes reproducir el bug, no puedes arreglarlo. Punto.

✗ "Me pasó una vez, voy a cambiar esto y espero que se arregle"

✓ "Puedo reproducir el bug en 3 pasos exactos. Ahora puedo testear mi fix."

3. Framework de debugging en 5 pasos

Estos 5 pasos funcionan para **cualquier bug**, en cualquier lenguaje:

Paso 1: REPRODUCIR

Preguntas clave:

- ¿Qué hago exactamente para que el bug aparezca?
- ¿Pasa siempre o solo a veces?
- ¿En qué dispositivo/plataforma?
- ¿Con qué datos de entrada?

Acción:

1. Abre un documento nuevo (puede ser un comment en el código)
2. Escribe los steps exactos para reproducir
3. Ejecuta esos steps. Si el bug no aparece, ajusta hasta que sea consistente

Paso 2: AISLAR

Preguntas clave:

- ¿En qué función/método ocurre?
- ¿En qué capa de la arquitectura? (UI, Domain, Data)
- ¿Es un problema de datos, lógica o presentación?

Acción:

1. Pon un breakpoint al inicio del flujo (ej: handler del evento)
2. F10 (Step Over) hasta que el comportamiento anormal aparezca
3. Ahora sabes en qué rango de código está el bug
4. Repite: pon un breakpoint a mitad de ese rango
5. En 3-5 iteraciones, encuentras la línea exacta

Paso 3: DIAGNOSTICAR

Preguntas clave:

- ¿Qué valor tiene la variable en el momento del error?
- ¿Qué camino tomó el código? (call stack)
- ¿Qué era el estado antes del error?

Acción:

1. En el breakpoint, mira VARIABLES: ¿qué valores hay?
2. Agrega WATCH para las variables sospechosas
3. Usa Debug Console para evaluar expresiones
4. Mira el CALL STACK para ver cómo llegaste ahí
5. La respuesta está en los datos, no en tu intuición

Paso 4: CORREGIR

Preguntas clave:

- ¿Cuál es el fix más mínimo que resuelve el problema?
- ¿Este fix rompe algo más?
- ¿Puedo probar que el fix funciona?

Acción:

1. Escribe el fix más pequeño posible
2. NO refactorices al mismo tiempo (un cambio a la vez)
3. Ejecuta los steps del Paso 1: ¿el bug ya no aparece?
4. Ejecuta tests existentes: ¿algo se rompió?

Paso 5: VERIFICAR

Preguntas clave:

- ¿El bug original está resuelto?
- ¿Introduje nuevos bugs?
- ¿Los tests pasan?
- ¿Puedo reproducir el escenario original sin el bug?

Acción:

1. Reproduce los steps exactos del Paso 1
2. Verifica que el comportamiento sea el esperado
3. Ejecuta la suite de tests
4. Si todo pasa, documenta: qué era el bug y cómo lo arreglaste

4. Cuándo SÍ usar la IA (y cuándo NO)

La IA es una herramienta poderosa. El problema no es usarla, es **usarla sin entender**.

4.1 SÍ usa la IA cuando:

Situación	Por qué
Ya entiendes el bug y necesitas help con la sintaxis del fix	La IA es rápida para code generation
Necesitas un snippet boilerplate (ej: configurar un interceptor HTTP)	No es lógica de negocio
Quieres aprender un patrón nuevo (ej: cómo usar un package)	Es documentación en formato conversacional
Tienes un error de compilación que no entiendes	La IA puede explicar el mensaje de error

4.2 NO uses la IA cuando:

Situación	Por qué
No sabes POR QUÉ algo falla	Necesitas diagnosticar, no recibir una solución ciega
El bug es de lógica de negocio	Solo tú (y tu team) entienden la lógica
La solución "parece" funcionar pero no sabes por qué	Estás introduciendo deuda técnica oculta
Es un bug de performance o memory leak	La IA no puede medir tu app en tu dispositivo

Situación	Por qué
Necesitas entender el call stack completo	La IA no tiene contexto de tu estado en runtime

4.3 El test definitivo

Antes de pedir ayuda a la IA, hazte esta pregunta:

"¿Podría explicarle a otro developer exactamente qué está pasando y por qué?"

Si la respuesta es NO, **no entiendes el bug todavía**. Usar la IA ahora solo enmascara el problema.

Si la respuesta es SÍ, **úsala** para acelerar el fix.

5. La checklist universal: 15 preguntas ANTES de pedir ayuda a la IA

Imprime esto y ponlo al lado de tu monitor:

- ☐ 1. ¿Puedo reproducir el bug consistentemente?
- ☐ 2. ¿Sé exactamente qué pasos lo provocan?
- ☐ 3. ¿En qué archivo/función ocurre?
- ☐ 4. ¿Qué valor tiene la variable sospechosa en el momento del error?
- ☐ 5. ¿Qué dice el call stack?
- ☐ 6. ¿Qué cambió recientemente? (git log)
- ☐ 7. ¿Es un error de compilación o de runtime?
- ☐ 8. ¿Es un error nuevo o existente?
- ☐ 9. ¿Puedo hacer un breakpoint en el lugar correcto?
- ☐ 10. ¿Inspeccioné las variables con WATCH/Debug Console?
- ☐ 11. ¿Leí el error COMPLETO (no solo la primera línea)?
- ☐ 12. ¿Busqué el error en Google/Stack Overflow primero?
- ☐ 13. ¿Revisé la documentación oficial del package/API?
- ☐ 14. ¿Puedo explicar el problema a otro developer?
- ☐ 15. ¿Entiendo QUÉ está fallando (no solo QUÉ hacer para arreglarlo)?

Si más de 3 están sin marcar, **estás pidiendo ayuda prematuramente**.

6. Ejercicio mental: mapea tu debugging

Dado este escenario:

Tu app de Flutter tiene un BLoC de login. Cuando el usuario presiona "Login", la app se queda en loading indefinidamente. No muestra error, no navega.

Antes de tocar cualquier herramienta, responde:

1. ¿Dónde pondrías tu primer breakpoint?
2. ¿Qué variable inspeccionarías primero?

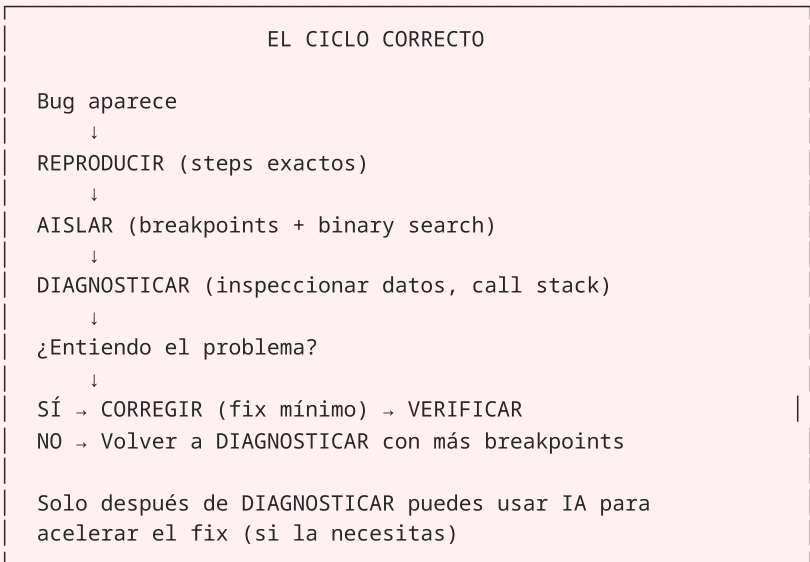
- 3. ¿Qué Expressions agregarías al WATCH?
- 4. ¿Qué buscarías en el CALL STACK?
- 5. ¿Qué consultarías en la Debug Console?

Respuesta esperada:

- 1. Breakpoint en el handler del evento LoginSubmitted (línea donde se llama loginUseCase)
- 2. Inspeccionar: event.email, event.password, y el resultado de loginUseCase
- 3. WATCH: state.runtimeType (para ver si cambia de LoginInitial a LoginLoading)
- 4. CALL STACK: verificar que el BLoC recibió el evento correctamente
- 5. Debug Console: evaluar await loginUseCase(LoginParams(email: "test", password: "123")) directamente para ver si el UseCase retorna algo

Si podías responder esto **sin pedir ayuda a la IA**, ya estás desarrollando la mentalidad.

7. Mentalidad anti-AI-dependency: resumen



Resumen

Concepto	Descripción
"No adivines, observa"	Usa breakpoints y datos, no intuición
Binary search	Reduce el espacio de búsqueda a la mitad en cada paso
Reproducir primero	Si no puedes reproducir, no puedes arreglar
Fix mínimo	Un cambio a la vez, sin refactorizar

Concepto	Descripción
IA como acelerador	Úsala para coding, no para diagnóstico
Checklist universal	15 preguntas ANTES de pedir ayuda

Referencias

- [VS Code | Debugging](#) — Fundamentos del debugger
- [Flutter | Debugging tools](#) — Panorama de herramientas de debugging
- [The Debugging Book](#) — Metodología de debugging (general, no Flutter)

 **Siguiente:** [01-fundamentos-debugging.md](#) — Fundamentos del debugging en VS Code: modos de compilación, interfaz del debugger y flujo completo

28 — Debugging Programático: Herramientas de Debug desde el Código

Flutter ofrece un arsenal de flags, métodos y widgets para diagnosticar problemas directamente desde tu código Dart, sin necesidad de breakpoints ni DevTools. Conócelos para cuando el debugger tradicional no es la mejor opción.

1. ¿Cuándo usar debugging programático?

Escenario	Herramienta debugger (VS Code)	Herramienta programática
Necesitas pausar en una línea específica	Breakpoint	—
Necesitas ver qué rebuilda un widget	—	<code>debugPrintRebuildDirtyWidgets</code>
Necesitas ver el árbol de widgets	—	<code>debugDumpWidgetTree()</code>
Necesitas ver bounds de cada widget	—	<code>debugPaintSizeEnabled</code>
El bug solo pasa en release (no puedes debuggear)	—	<code>FlutterError.onError</code>
Necesitas logging continuo sin pausas	Logpoint	<code>debugPrint()</code>
Necesitas ver qué se repinta	—	<code>debugProfilePaintsEnabled</code>

El debugging programático es tu arsenal para bugs que **no puedes pausar** o que necesitas **monitorear continuamente**.

2. `debugPrint()` — La alternativa inteligente a `print()`

2.1 Por qué no usar `print()`

```
// x print() envía todo al log de Android/iOS de golpe
for (var i = 0; i < 1000; i++) {
  print('Item $i'); // Puede causar "lost connection to VM"
}
```

2.2 Por qué usar `debugPrint()`

```
// ✅ debugPrint() hace throttling (limita la velocidad de salida)
for (var i = 0; i < 1000; i++) {
  debugPrint('Item $i'); // Salida gradual, no se corta
}
```

2.3 `debugPrintThrottled()` — El throttling explícito

```
// Útil cuando quieres controlar la velocidad manualmente
debugPrintThrottled('Mensaje que puede repetirse mucho');
```

2.4 En producción se desactiva automáticamente

```
// debugPrint() NO se ejecuta en release mode
// print() SÍ se ejecuta (y puede filtrar información)
debugPrint('Esto solo se ve en debug/profile'); // ← No aparece en release
```

Regla de oro: Usa `debugPrint()` en vez de `print()` siempre. Es más seguro (no filtra en producción) y más estable (no causa lost connections).

3. Dump de árboles: ver la estructura de tu UI

3.1 `debugDumpWidgetTree()` — Árbol de widgets

```
// En cualquier punto de tu código (ej: dentro de un onPressed)
onPressed: () {
  debugDumpWidgetTree(); // Imprime el árbol completo de widgets
}
```

Salida típica:

```

MaterialApp
└─Directionality
  └─Overlay
    └─[Stack]
      └─Positioned
        └─Positioned
          └─RawDialogRoute
            └─_ModalScope
              └─Stack
                └─_SubtreeGuard
                  └─StatefulBuilder
                    └─Center
                      └─Padding
                        └─ SizedBox
                          └─Column ← Tu widget está aquí
                            └─Text("Login")
                            └─TextField
                            └─ElevatedButton

```

Cuándo usarlo:

- Cuando no entiendes por qué un widget no se ve
- Cuando necesitas ver qué widgets envuelven al tuyo
- Cuando un widget tiene constraints inesperados

3.2 `debugDumpRenderTree()` — Árbol de render

```
debugDumpRenderTree(); // Imprime el árbol de render objects
```

Cuándo usarlo:

- Cuando sospechas un problema de layout
- Cuando necesitas ver el tamaño y posición exactos de cada widget
- Cuando hay un render overflow que no entiendes

3.3 `debugDumpLayerTree()` — Árbol de capas

```
debugDumpLayerTree(); // Imprime las capas de composición (GPU)
```

Cuándo usarlo:

- Cuando hay problemas visuales (artefactos, parpadeos)
- Cuando necesitas entender cómo Flutter compone la UI en capas

3.4 `debugDumpSemanticsTree()` — Árbol de accesibilidad

```
debugDumpSemanticsTree(); // Imprime el árbol de semántica (accesibilidad)
```

Cuándo usarlo:

- Cuando estás implementando soporte para screen readers
- Cuando los widgets no tienen la semántica correcta (Semantics widget)

3.5 `debugDumpSemanticsTree(SemanticsBinding.instance.rootPipelineOwner)` — Versión completa

```
// Para ver el árbol completo incluyendo nodos no visibles
debugDumpSemanticsTree(
  SemanticsBinding.instance.rootPipelineOwner,
);
```

4. Flags de debug: activar overlays visuales

4.1 `debugPaintSizeEnabled` — Bounds y constraints

```
import 'package:flutter/rendering.dart';

void main() {
  debugPaintSizeEnabled = true; // Activa overlay de bounds
  runApp(MyApp());
}
```

Qué muestra:

- Bordes azules para cada widget (muestra sus bounds)
- Padding en amarillo
- Margins en verde claro
- Constraints con líneas punteadas

4.2 `debugPaintLayerBordersEnabled` — Bordes de capas

```
import 'package:flutter/rendering.dart';

void main() {
  debugPaintLayerBordersEnabled = true; // Bordes de cada capa
  runApp(MyApp());
}
```

Qué muestra:

- Bordes de colores aleatorios para cada capa de composición
- Útil para entender por qué hay parpadeos o regiones que no se repintan

4.3 `debugPaintPointersEnabled` — Área de touch

```
import 'package:flutter/rendering.dart';

void main() {
  debugPaintPointersEnabled = true; // Muestra área de touch
  runApp(MyApp());
}
```

Qué muestra:

- Regiones en rojo donde el touch está siendo procesado
- Útil para entender por qué un widget no responde a taps

4.4 `debugPaintBaselinesEnabled` — Baselines de texto

```
import 'package:flutter/rendering.dart';

void main() {
  debugPaintBaselinesEnabled = true; // Muestra baselines
  runApp(MyApp());
}
```

Qué muestra:

- Líneas que muestran las baselines de texto
- Útil para alinear texto manualmente

5. Debug de rebuilds: quién se actualiza y cuándo

5.1 `debugPrintRebuildDirtyWidgets` — Log cada rebuild

```
import 'package:flutter/widgets.dart';

void main() {
  debugPrintRebuildDirtyWidgets = true; // Log cada rebuild
  runApp(MyApp());
}
```

Salida típica:

```
Rebuilding: MyHomePage
Rebuilding: Text("Counter: 5")
Rebuilding: ElevatedButton
Rebuilding: MyHomePage ← ¡Este se rebuilda demasiado!
```

Cuándo usarlo:

- Cuando tu app se siente lenta y no sabes qué widget rebuilda de más
- Cuando sospechas rebuilds innecesarios
- Para validar que `const` y `RepaintBoundary` están funcionando

5.2 `debugPrintLayouts` — Log cada layout

```
import 'package:flutter/rendering.dart';

void main() {
  debugPrintLayouts = true; // Log cada layout
  runApp(MyApp());
}
```

Cuándo usarlo:

- Cuando un widget tiene tamaño inesperado
- Cuando hay constraints que no entiendes
- Para entender el flujo de layout de un widget complejo

5.3 `debugProfilePaintsEnabled` — Qué widgets se repintan

```
import 'package:flutter/rendering.dart';

void main() {
  debugProfilePaintsEnabled = true; // Log cada repaint
  runApp(MyApp());
}
```

Cuándo usarlo:

- Cuando hay jank visual (parpadeos, tartamudeo)
 - Para identificar qué widget se repinta innecesariamente
 - Para validar que `RepaintBoundary` está funcionando
-

6. Performance Overlay desde código

6.1 `showPerformanceOverlay`

```
import 'package:flutter/material.dart';

void main() {
  runApp(
    Stack(
      children: [
        MyApp(),
        PerformanceOverlay.all(), // Overlay de performance
      ],
    ),
  );
}
```

Qué muestra:

- Barra superior: GPU rendering time (verde < 16ms, rojo > 16ms)
- Barra inferior: UI thread time (verde < 16ms, rojo > 16ms)

6.2 `WidgetsFlutterBinding.ensureInitialized()` + flag

```
import 'package:flutter/material.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();

  // Solo en debug mode
  assert(() {
    debugPaintSizeEnabled = true;
    debugPrintRebuildDirtyWidgets = true;
    return true;
  }());

  runApp(MyApp());
}
```

Nota: Los `assert()` solo se ejecutan en debug mode. En profile/release se ignoran automáticamente.

7. Manejo de errores: personalizar la respuesta

7.1 `FlutterError.onError` — Capturar errores de Flutter

```
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';

void main() {
  FlutterError.onError = (FlutterErrorDetails details) {
    // Log personalizado
    FlutterError.presentError(details);

    // O enviar a un servicio de crash reporting
    if (kReleaseMode) {
      // Sentry.captureException(details.exception, stackTrace: details.stack);
    }
  };

  runApp(MyApp());
}
```

Cuándo usarlo:

- Para personalizar cómo se reportan los errores de Flutter
- Para enviar crashes a un servicio (Sentry, Crashlytics)
- Para mostrar un UI de error en vez del widget rojo por defecto

7.2 `ErrorWidget.builder` — Personalizar pantalla de error

```
import 'package:flutter/material.dart';

void main() {
  ErrorWidget.builder = (FlutterErrorDetails details) {
    return Material(
      child: Center(
        child: Padding(
          padding: const EdgeInsets.all(16),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.min,
            children: [
              const Icon(Icons.error, color: Colors.red, size: 48),
              const SizedBox(height: 16),
              Text(
                'Algo salió mal',
                style: Theme.of(context).textTheme.headlineSmall,
              ),
              const SizedBox(height: 8),
              Text(details.exceptionAsString()),
            ],
          ),
        ),
      );
    };
  runApp(MyApp());
}
```

7.3 `debugCheckElevationsEnabled` — Detectar sombras incorrectas

```
import 'package:flutter/rendering.dart';

void main() {
  debugCheckElevationsEnabled = true; // Detecta problemas de elevación
  runApp(MyApp());
}
```

8. Assertions personalizadas

8.1 `assert()` con mensajes descriptivos

```
class LoginUseCase {
    final UserRepository repository;

    LoginUseCase({required this.repository});

    Future<User> call(LoginParams params) {
        // Assertions solo se ejecutan en debug mode
        assert(
            params.email.isNotEmpty,
            'Email no puede estar vacío. Verificar que el formulario valide antes de llamar al UseCase.',
        );
        assert(
            params.password.length >= 8,
            'Password debe tener al menos 8 caracteres. Recibido: ${params.password.length}',
        );

        return repository.login(params);
    }
}
```

8.2 Assertions con lógica compleja

```
class CartState {
    final List<CartItem> items;

    CartState({required this.items}) {
        assert(
            items.every((item) => item.quantity > 0),
            'Todos los items deben tener cantidad > 0. '
            'Items con cantidad inválida: ${items.where((i) => i.quantity <= 0).map((i) => i.id)}',
        );
        assert(
            items.length <= 50,
            'Carrito no puede tener más de 50 items únicos. Recibido: ${items.length}',
        );
    }
}
```

8.3 Assertions en development pero no en producción

```
// Los assert() solo se ejecutan en debug mode
// En profile/release, se ignoran completamente
assert(
  () {
    // Tu lógica de validación aquí
    _validateState();
    return true;
  }(),
  'Descripción del error',
);
```

9. Resumen: tabla de todas las herramientas programáticas

Herramienta	Qué hace	Cuándo usarla
<code>debugPrint()</code>	Logging con throttling	Siempre (en vez de <code>print()</code>)
<code>debugDumpWidgetTree()</code>	Imprime árbol de widgets	No entiendes la estructura de UI
<code>debugDumpRenderTree()</code>	Imprime árbol de render	Problemas de layout/tamaño
<code>debugDumpLayerTree()</code>	Imprime capas GPU	Artefactos visuales
<code>debugDumpSemanticsTree()</code>	Imprime árbol de accesibilidad	Screen readers
<code>debugPaintSizeEnabled</code>	Overlay de bounds	Constraints inesperados
<code>debugPaintLayerBordersEnabled</code>	Bordes de capas	Parpadeos, regiones
<code>debugPaintPointersEnabled</code>	Área de touch	Widgets no responden a taps
<code>debugPaintBaselinesEnabled</code>	Baselines de texto	Alineación de texto
<code>debugPrintRebuildDirtyWidgets</code>	Log cada rebuild	Performance, rebuilds innecesarios
<code>debugPrintLayouts</code>	Log cada layout	Widgets con tamaño inesperado
<code>debugProfilePaintsEnabled</code>	Log cada repaint	Jank visual
<code>showPerformanceOverlay</code>	Overlay de frames	Medir performance en runtime
<code>FlutterError.onError</code>	Capturar errores Flutter	Crash reporting personalizado
<code>ErrorWidget.builder</code>	Pantalla de error custom	UI de error en producción
<code>debugCheckElevationsEnabled</code>	Detectar sombras	Problemas de elevación
<code>assert()</code> con mensajes	Validaciones debug-only	Invariantes en development

10. Patrón recomendado: debug flags condicionales

```
import 'package:flutter/foundation.dart';
import 'package:flutter/rendering.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();

  // Solo en debug mode: activar herramientas de debugging
  assert(() {
    debugPaintSizeEnabled = false;          // Cambiar a true cuando necesites
    debugPrintRebuildDirtyWidgets = false; // Cambiar a true cuando necesites
    debugPaintLayerBordersEnabled = false;
    debugProfilePaintsEnabled = false;
    return true;
  }());

  // En todos los modos: manejo de errores personalizado
  FlutterError.onError = (details) {
    FlutterError.presentError(details);
    if (kReleaseMode) {
      // Reportar a servicio de crashes
    }
  };

  ErrorWidget.builder = (details) {
    return const Material(
      child: Center(child: Text('Error inesperado')),
    );
  };

  runApp(MyApp());
}
```

Consejo: No actives todos los flags al mismo tiempo. Activa uno a la vez según el tipo de bug que estés investigando. Los flags se pueden togglear en caliente sin reiniciar la app (solo cambia el valor y haz hot reload).

Resumen

Concepto	Descripción
<code>debugPrint()</code>	Logging seguro con throttling (no filtra en release)
Dump methods	Ver árboles de widgets, render, layers, semantics
Debug flags	Overlays visuales para bounds, touch, rebuilds
<code>FlutterError.onError</code>	Capturar y personalizar errores de Flutter
<code>assert()</code>	Validaciones que solo corren en debug mode
Patrón condicional	Activar flags con <code>assert(() { ... }())</code>

Referencias

- [Flutter | Debug from code](#) — Documentación oficial de debugging programático
- [Flutter | Common errors](#) — Errores comunes y sus soluciones
- [Flutter | Debug flags](#) — Todos los flags de rendering

 **Anterior:** [27-mentalidad-debugging.md](#) — Mentalidad de debugging: piensa como un debugger 

Siguiente: [01-fundamentos-debugging.md](#) — Fundamentos del debugging en VS Code

29 — Playbook de Debugging Sistemático

Un paso-a-paso reproducible para diagnosticar CUALQUIER bug sin pedir ayuda a la IA. Imprime este archivo y tenlo al lado de tu monitor.

1. El playbook en 6 pasos

PASO 1: ENTENDER	→ ¿Qué se esperaba vs qué pasó?
PASO 2: REPRODUCIR	→ Steps exactos, siempre falla
PASO 3: AISLAR	→ Binary search con breakpoints
PASO 4: DIAGNOSTICAR	→ Datos, call stack, watch
PASO 5: CORREGIR	→ Fix mínimo, un cambio a la vez
PASO 6: VERIFICAR	→ Reproducir + tests + regresión

2. Paso 1: ENTENDER

2.1 Preguntas obligatorias

Responde estas preguntas **por escrito** (en un comment, un doc, o un notepad):

1. ¿Qué behavior esperaba ver?
2. ¿Qué behavior veo en realidad?
3. ¿Cuándo empezó a pasar? (¿antes funcionaba?)
4. ¿Pasa siempre o solo a veces?
5. ¿En qué dispositivo/plataforma?
6. ¿Con qué datos de entrada?
7. ¿Cambie algo recientemente? (git log --oneline -10)

2.2 Patrón de error: "No sabes qué esperabas"

Si no puedes responder la pregunta #1, **el bug no es técnico, es de requirements**. Habla con tu team/product owner antes de debugging.

2.3 Git como herramienta de debugging

```
# Ver cambios recientes
git log --oneline -10

# Ver qué cambió en un archivo específico
git log --oneline --follow lib/presentation/bloc/login/login_bloc.dart

# Comparar con una versión que funcionaba
git diff HEAD~5 lib/presentation/bloc/login/login_bloc.dart

# Si sabes cuándo funcionaba, volver temporalmente
git stash # Guardar cambios actuales
git checkout <commit-hash> # Ir a la versión que funcionaba
# Probar → si funciona, el bug está en los cambios entre ese commit y HEAD
git checkout - # Volver al HEAD
git stash pop # Recuperar cambios
```

3. Paso 2: REPRODUCIR

3.1 Escribir los steps exactos

Steps para reproducir:

1. Abrir la app
2. Ir a la pantalla de login
3. Escribir "test@email.com" en el campo de email
4. Escribir "12345678" en el campo de password
5. Presionar el botón "Login"
6. OBSERVAR: la app se queda en loading indefinidamente

3.2 Si el bug es intermitente

¿Cuándo falla?

- ¿Con datos específicos? → Probar con esos datos exactos
- ¿Después de cierto tiempo? → Esperar ese tiempo
- ¿En cierto orden de acciones? → Repetir ese orden exacto
- ¿Con cierto estado previo? → Forzar ese estado primero

3.3 Si no puedes reproducir

Opciones:

1. Agregar logging detallado (debugPrint) en el flujo sospechoso
2. Reproducir en el mismo entorno (mismo dispositivo, misma versión OS)
3. Buscar reports de usuarios: ¿qué tenían en común?
4. Usar DevTools → Logging para ver qué pasa en background

4. Paso 3: AISLAR (Binary Search Debugging)

4.1 El principio

En vez de leer 50 archivos buscando el bug, usa **binary search**:

Tu flujo tiene 10 pasos. El bug está en alguno.

1. Pon breakpoint en el paso 5 (mitad)

2. Ejecuta hasta el breakpoint

3. ¿El estado es correcto hasta aquí? → Bug está entre 6-10

4. ¿El estado ya es incorrecto? → Bug está entre 1-5

5. Repite con la mitad correcta

6. En 3-4 iteraciones encuentras el paso exacto

4.2 Ejemplo práctico: BLoC de login

Flujo: LoginSubmitted → validate → LoginUseCase → repository → api → emit(result)

Breakpoint 1: LoginSubmitted (línea 10) → event.email = "test@test.com" ✓

Breakpoint 2: validate (línea 15) → datos válidos ✓

Breakpoint 3: LoginUseCase (línea 20) → params correctos ✓

Breakpoint 4: repository.login (línea 25) →Aquí el result es Left(failure) ×

→ El bug está entre el UseCase y el repository

→ Investigar el repository

4.3 Qué inspeccionar en cada breakpoint

Capa	Qué inspeccionar
Evento	<code>event.email</code> , <code>event.password</code> , campos del evento
Validate	Resultado de la validación, errores
UseCase	<code>params</code> que recibe, <code>result</code> que retorna
Repository	Datos que envía, respuesta que recibe
API/Datasource	Request body, response body, status code, headers
State	<code>state.runtimeType</code> , campos del estado emitido

5. Paso 4: DIAGNOSTICAR

5.1 Herramientas de diagnóstico en orden

1. VARIABLES panel → ¿Qué valor tiene la variable sospechosa?
2. WATCH → Agregar 表达式 clave: state.runtimeType, result.runtimeType
3. CALL STACK → ¿Cómo llegué aquí? ¿Qué función me llamó?
4. Debug Console → Evaluar expresiones, probar "what if"
5. debugPrint() → Agregar logging para el siguiente paso

5.2 Expresiones útiles para WATCH

```
// Tipo de estado actual
state.runtimeType

// ¿Es un error?
result.runtimeType == Left<Failure, User>

// Contenido de un mapa
myMap.keys.toList()

// Tamaño de una lista
myList.length

// Primer elemento
myList.first

// ¿Está vacío?
myList.isEmpty

// Valor de un campo específico
(user as User).email
```

5.3 Debug Console como laboratorio

```
// En la Debug Console, puedes probar cosas SIN modificar el código:

// Crear un objeto y probarlo
final params = LoginParams(email: "test@test.com", password: "12345678");
params.isValid() // → true/false

// Llamar un método y ver el resultado
await repository.login(params) // → Left(failure) o Right(user)

// Verificar un controller
controller.text // → "valor actual"

// Probar una expresión condicional
state is LoginLoading // → true/false
```

6. Paso 5: CORREGIR

6.1 Reglas del fix

1. UN CAMBIO A LA VEZ (no refactoros mientras debuggeas)
2. FIX MÍNIMO (lo más pequeño que resuelva el problema)
3. NO AGREGUES FEATURES (eso es para después)
4. COMMIT DESPUÉS DEL FIX (para poder volver si algo se rompe)

6.2 Tipos de fix comunes

Tipo de bug	Fix típico
Null safety	Agregar <code>?</code> o <code>?? defaultValue</code>
Estado no se actualiza	Verificar <code>emit()</code> se ejecuta
Widget no se ve	Verificar constraints, <code>Expanded</code> / <code>Flexible</code>
API falla	Verificar headers, URL, body
Memory leak	Agregar <code>cancel()</code> en <code>dispose()</code>
Performance	Agregar <code>const</code> , <code>RepaintBoundary</code> , <code>ListView.builder</code>

6.3 El fix estándar de null safety

```
// ✗ Antes (crash)
final userName = user.name;

// ✔ Después (seguro)
final userName = user?.name ?? 'Unknown';
```

7. Paso 6: VERIFICAR

7.1 Checklist de verificación

- ☐ 1. Reproducir los steps del Paso 2: ¿el bug ya no aparece?
- ☐ 2. Ejecutar tests existentes: ¿todos pasan?
- ☐ 3. Probar el happy path completo: ¿la app funciona normal?
- ☐ 4. Probar edge cases: ¿qué pasa con datos vacíos, límites, etc.?
- ☐ 5. Probar en otro dispositivo/emulador: ¿especifico de plataforma?
- ☐ 6. Verificar que no hay regressions en funcionalidad relacionada

7.2 Si algo se rompió

1. Revert el fix: `git checkout <archivo>`
 2. Vuelve al Paso 3 (Aislar)
 3. Busca una solución que no rompa lo existente
 4. 0: agrega un test que cubra el caso que se rompió

8. Matriz de debugging: tipo de bug → herramienta

Tipo de bug	Primer breakpoint	Qué inspeccionar	Fix típico
UI se ve mal	Widget builder o initState	Constraints, tamaño, bounds	<code>Expanded</code> , <code>Flexible</code> , <code>SizerBox</code>
Estado no cambia	<code>emit()</code> del BLoC/Cubit	<code>state.runtimeType</code> , params	Verificar <code>emit()</code> se ejecuta
API falla	Llamada HTTP	Request/response, headers	URL, Authorization, body
Navegación falla	<code>context.push()</code> / <code>go()</code>	Ruta, parámetros, auth guard	Verificar ruta en GoRouter
Build error	—	Mensaje de error completo	Tipos, imports, null safety
Performance	—	Frames > 16ms	<code>const</code> , <code>ListView.builder</code>
Memory leak	—	Objetos que crecen	<code>cancel()</code> , <code>dispose()</code>
Hot reload no funciona	—	Tipo de cambio	Hot Restart en vez de Reload

9. Checklist universal: 15 preguntas ANTES de pedir ayuda

- ☐ 1. ¿Puedo reproducir el bug consistentemente?
 - ☐ 2. ¿Sé exactamente qué pasos lo provocan?
 - ☐ 3. ¿En qué archivo/función ocurre?
 - ☐ 4. ¿Qué valor tiene la variable sospechosa en el momento del error?
 - ☐ 5. ¿Qué dice el call stack?
 - ☐ 6. ¿Qué cambió recientemente? (git log)
 - ☐ 7. ¿Es un error de compilación o de runtime?
 - ☐ 8. ¿Es un error nuevo o existente?
 - ☐ 9. ¿Puedo hacer un breakpoint en el lugar correcto?
 - ☐ 10. ¿Inspeccioné las variables con WATCH/Debug Console?
 - ☐ 11. ¿Leí el error COMPLETO (no solo la primera línea)?
 - ☐ 12. ¿Busqué el error en Google/Stack Overflow primero?
 - ☐ 13. ¿Revisé la documentación oficial del package/API?
 - ☐ 14. ¿Puedo explicar el problema a otro developer?
 - ☐ 15. ¿Entiendo QUÉ está fallando (no solo QUÉ hacer para arreglarlo)?

10. Ejercicio integrador: 3 bugs graduales

Bug 1 (Fácil): El contador no aumenta

Tu app tiene un CounterCubit. Cuando presionas "+", el contador no cambia.

Steps para reproducir:

1. Abrir la app
2. Presionar el botón "+"
3. OBSERVAR: el contador sigue en 0

Tu trabajo:

1. ¿Dónde pondrías el primer breakpoint?
2. ¿Qué variable inspeccionarías?
3. Encuentra el fix (pista: revisa el handler del evento)

Respuesta:

1. Breakpoint en increment() del Cubit
2. Inspeccionar: state.counter, el valor antes del emit
3. Fix probable: el Cubit no hace emit(state.copyWith(counter: state.counter + 1))

Bug 2 (Medio): La lista no carga

Tu app tiene un ListCubit que carga items de una API. La lista queda vacía siempre.

Steps para reproducir:

1. Abrir la app
2. Navegar a la pantalla de lista
3. OBSERVAR: "No hay items" aunque la API tiene datos

Tu trabajo:

1. Binary search: ¿dónde está el bug? (evento → usecase → repository → datasource → API)
2. ¿Qué inspeccionarías en cada breakpoint?
3. Encuentra el fix

Respuesta:

1. Breakpoint en datasource → ver qué retorna la API
2. Si la API retorna datos pero el repository no los parsea → bug en el mapper
3. Si la API retorna vacío → bug en la query o en los headers

Bug 3 (Difícil): La app se congela después de navegar

Tu app tiene una pantalla de chat. Después de 30 segundos de chat, la app se congela.

Steps para reproducir:

1. Abrir la app
2. Ir a la pantalla de chat
3. Enviar 5-10 mensajes
4. Esperar 30 segundos
5. OBSERVAR: la app se congela

Tu trabajo:

1. ¿Qué tipo de bug es? (memory leak, timer, stream)
2. ¿Qué herramienta usarías primero?
3. Encuentra el fix (pista: busca streams/timers no cancelados)

Respuesta:

1. Memory leak o timer leak
2. DevTools → Memory → Heap Snapshot (comparar antes y después)
3. Fix: cancelar streams/timers en dispose()
 - Buscar: Stream.listen() sin cancel()
 - Buscar: Timer.periodic() sin cancel()
 - Buscar: StreamController sin close()

Resumen

Paso	Qué hacer	Herramienta
1. Entender	Responder 7 preguntas	Notepad, git log
2. Reproducir	Steps exactos	Tu app
3. Aislar	Binary search con breakpoints	VS Code debugger
4. Diagnosticar	Variables, watch, call stack	VS Code debugger
5. Corregir	Fix mínimo, un cambio a la vez	Editor de código
6. Verificar	Reproducir + tests	Tu app, terminal



Referencias

- [27-mentalidad-debugging.md](#) — La filosofía detrás de este playbook
- [26-workflow-debugging-por-tipo.md](#) — Workflows específicos por tipo de bug
- [VS Code | Debugging](#) — Herramientas del debugger
- [Flutter | Debugging tools](#) — Herramientas de debugging de Flutter

 **Anterior:** [26-workflow-debugging-por-tipo.md](#) — Workflow por tipo de bug  **Siguiente:** [30-referencia-rapida-flutter-debugging.md](#) — Referencia rápida de todo el módulo

31 — Debugging por Capa: Ejecución Aislada de cada Capa con el Debugger

Cuando desarrollas en **modo andamiaje SDD** (02-SPEC-DRIVEN-DEVELOPMENT), la IA genera el esqueleto de cada fichero con `throw UnimplementedError()` y tú **implementas los cuerpos**. El problema: una feature recién andamiada **no está cableada** — no ves su resultado desde la app completa porque las capas superiores todavía no existen. Este capítulo te da la técnica para **correr cada capa de forma aislada con el debugger** y verificar, en tiempo real, que hace lo que debe — sin tests obligatorios y sin depender de la IA para debugear.

1. El problema que resuelve

En el flujo SDD, para llegar a la feature `add-cart` tienes 6+ ficheros por capa:

```
domain/entities/cart.dart           → throw UnimplementedError()
domain/usecases/get_cart.dart       → throw UnimplementedError()
data/models/cart_model.dart         → throw UnimplementedError()
data/datasources/cart_remote_datasource.dart → throw UnimplementedError()
data/repositories/cart_repository_impl.dart → throw UnimplementedError()
presentation/cubit/cart_cubit.dart → throw UnimplementedError()
presentation/pages/cart_page.dart   → throw UnimplementedError()
```

No puedes arrancar la app y ver el resultado porque:

- La ruta `/cart` aún no existe o no está guardada por sesión.
- `CartPage` depende de `CartCubit`, que depende de usecases, que dependen del repository, que depende del datasource... y nada está implementado aún.
- `GetIt` (`service_locator.dart`) registra dependencias que aún lanzan `UnimplementedError`.

La solución: crear un **entrypoint aislado (scratchpad)** por capa que instancie SOLO esa capa, le inyecte a mano lo que necesita, la invoque y deje el **debugger** para pisar línea a línea. Así verificas cada capa **en el momento exacto en que la implementas**, sin esperar a cablear toda la feature ni depender de la app.

Esta es la diferencia entre **testing** (que automatiza verificación y vive en 05-TESTING) y **este capítulo**: ejecución *manual e interactiva* con el debugger para **entender y verificar el comportamiento real** mientras escribes. No son excluyentes — se complementan. Cuando una capa te cuesta "verla bien" con el debugger, ese es el mejor momento para escribirle un test (usa la skill `flutter-test-generator`).

2. Los dos modos de verificación (clave)

Antes de escribir código, decide **qué le inyectas** a la capa para correrla aislada. Hay dos modos:

Modo A: REAL (integración) — para capas de datos

Las capas que hablan hacia afuera (datasource → Supabase; repository → datasource) se verifican mejor contra sus **dependencias reales**. El objetivo es **ver la respuesta real** de la API, la excepción real, el request HTTP real.

```
// SCRATCH – datasource REAL contra Supabase
// No hay fake. Construyes el client real y ves el JSON que devuelve.
final client = SupabaseClient(
    const String.fromEnvironment('SUPABASE_URL'),
    const String.fromEnvironment('SUPABASE_ANON_KEY'),
);
final ds = CartRemoteDataSource(client: client);

// Pones AQUÍ un breakpoint → ves la respuesta realmente devuelta
final items = await ds.getCart(userId: 'test-123');
print(items); // ElevatedButton? No – el JSON real mapeado
```

Modo B: AISLADO CON STUB — para capas superiores

Las capas que dependen de capas **internas aún no implementadas** (usecase → repository; cubit → usecases) se verifican con un **stub manual** — objeto que imita a la dependencia para ejercitar la lógica de la capa sin esperar a tenerla.

```
// SCRATCH – usecase con repository STUB (aún no implementado)
class _CartRepositoryStub implements CartRepository {
    @override
    Future<Either<Failure, List<Cart>>> getCart({required String userId}) async {
        return Right([Cart(id: '1', items: [...])]); // dato de prueba fijo
    }
}

void main() async {
    final usecase = GetCart(repository: _CartRepositoryStub());
    final result = await usecase(GetCartParams(userId: 'test-123'));
    // Breakpoint aquí → ves el Either<Failure, List<Cart>> resultante
    result.fold(
        (l) => print('Failure: $l'),
        (r) => print('Éxito: ${r.length} items'),
    );
}
```

Regla práctica: usa **Modo A (real)** siempre que la dependencia externa ya exista y quieras ver el comportamiento real. Usa **Modo B (stub)** SOLO cuando la dependencia inferior todavía no está lista y quieres verificar la lógica de la capa sin bloquearte. La matriz de la sección 5 te dice cuál elegir por capa.

3. El patrón scratchpad: dónde y cómo

3.1 Dónde vivir

Crea una carpeta `tool/scratch/` (recomendado — no se compila a producción) o `lib/scratch/`. Un `main` por capa:

```
tool/scratch/
├─ main_entity.dart      → dart run
├─ main_model.dart       → dart run
├─ main_state.dart       → dart run
├─ main_usecase.dart     → dart run
├─ main_datasource.dart  → flutter run -t tool/scratch/main_datasource.dart
├─ main_repository.dart  → flutter run -t
├─ main_cubit.dart       → flutter run -t
└─ main_page.dart        → flutter run -t
```

¿ `dart run` o `flutter run -t` ? Si la capa NO toca Flutter (no importa `flutter/*`, no usa widgets, no usa paquetes que requieran motor Flutter) → `dart run`. Si toca Flutter (datasource con `supabase_flutter`, cubit con `flutter_bloc`, page) → `flutter run -t <archivo>`.

3.2 Cómo lanzarlo con el debugger

Añade entradas al `launch.json` (ver `02-configuracion-launch-json.md`) para cada scratchpad. Así lo lanzas con `F5` y pisas breakpoints como en la app:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Scratch: UseCase (dart)",
      "type": "dart",
      "request": "launch",
      "program": "tool/scratch/main_usecase.dart"
    },
    {
      "name": "Scratch: DataSource (flutter)",
      "type": "dart",
      "request": "launch",
      "program": "tool/scratch/main_datasource.dart",
      "dartDefine": [
        "SUPABASE_URL=http://localhost:54321",
        "SUPABASE_ANON_KEY=eyJ..."
      ]
    },
    {
      "name": "Scratch: Cubit (flutter)",
      "type": "dart",
      "request": "launch",
      "program": "tool/scratch/main_cubit.dart"
    }
  ]
}
```

Usa el `presentation` de 02 §13 para agrupar todos los `Scratch: *` en un submenú y ocultar el ruido.

4. Inyección manual (sin GetIt)

El andamiaje registra dependencias en GetIt, pero en el scratchpad **no quieres depender del DI global** (esas dependencias aún lanzan `UnimplementedError`). Inyectas **a mano** en el `main`:

```
// × No hagas esto en el scratch (depende de capas aún rotas)
final ds = GetIt.I<CartRemoteDataSource>();

// ✓ Haz esto: construyes SOLO lo que tu capa necesita
final ds = CartRemoteDataSource(client: realClient); // Modo A
final repo = CartRepositoryImpl(datasource: ds); // Modo A (ds ya real)
final usecase = GetCart(repository: stubRepo); // Modo B
final cubit = CartCubit(usecases: [stubUsecase]); // Modo B
```

Esto te da **control total del harness**: decides exactamente qué se ejecuta y qué se simula, y el debugger se centra en tu capa, no en el cableado.

5. Matriz: cómo ejecutar y verificar cada capa

Capa	Runtime	Qué inyecto	Modo	Qué verifico (breakpoint/debugPrint)
Entity	<code>dart run</code>	nada	—	Cálculos puros, invariantes, <code>Equatable</code> / <code>copyWith</code> , bordes (total, descuento)
Model	<code>dart run</code>	nada	—	<code>fromJson</code> / <code>toJson</code> , roundtrip, mapeo snake↔camel, conversión a entity
State	<code>dart run</code>	nada	—	Igualdad/ <code>props</code> , que cada estado sea distinto del anterior
UseCase	<code>dart run</code>	repository	B (stub) si repo aún no está, o A si ya existe	<code>Either<Failure,T></code> , mensajes EXACTOS de validación (RN001-RN006)
DataSource	<code>flutter run -t</code>	client real	A (real)	Respuesta JSON real, excepciones reales (<code>AuthException</code> , <code>SocketException</code>), RPC
RepositoryImpl	<code>flutter run -t</code>	datasource	A (si ya) o B (stub)	Mapeo excepción→ <code>Failure</code> , delega al datasource
Cubit	<code>flutter run -t</code>	usecases	A (si ya) o B (stub)	Transiciones de estado una a una, <code>BlocObserver</code>

Capa	Runtime	Qué inyecta	Modo	Qué verifico (breakpoint/debugPrint)
Page/Widget	<code>flutter run</code>	cubit	A o B + Widget Inspector	Render por cada estado (Loading/Loaded/Error)

Entity / Model / State no inyectan nada y no tocan I/O → se corren con `dart run` puro, lo más rápido y simple para verificar lógica pura. Los **debugPrint** aquí son muy útiles para ver el resultado de cálculos sin abrir la app.

6. Capa a capa: qué hace cada scratchpad

6.1 Entity — lógica pura

Verificas cálculos e invariantes con `dart run`, sin dependencias.

```
// tool/scratch/main_entity.dart
import '../...' ; // ruta a tu entidad

void main() {
  final cart = Cart(
    items: [
      CartItem(name: 'Camiseta', price: 100, quantity: 2),
      CartItem(name: 'Gorra', price: 50, quantity: 1),
    ],
  );
  // Breakpoint aquí → inspecciona subtotal, impuesto, total
  print('Subtotal: ${cart.subtotal}'); // 250
  print('Impuesto: ${cart.tax}');      // 40 (16%)
  print('Total: ${cart.total}');       // 290
  // if (cart.cartItems.isEmpty) { } // prueba bordes
}
```

Verifica también `copyWith` y `==` / `hashCode` (Equatable): crea dos carts iguales y comprueba `a == b`.

6.2 Model — roundtrip JSON

```
// tool/scratch/main_model.dart
void main() {
  const json = {'id': '1', 'name': 'Camiseta', 'price': 100.0, 'createdAt': '2024-01-01T00:00:00Z'};
  // Breakpoint en fromJson → pisa el mapeo snake→camel
  final model = CartModel.fromJson(json);
  print(model.toJson()); // roundtrip: sale igual que entró
  print(model.toEntity()); // conversión a entidad de dominio
}
```

6.3 UseCase — lógica + stub de repository

```
// tool/scratch/main_usecase.dart
class _CartRepositoryStub implements CartRepository { /* ... */ }

void main() async {
  final usecase = GetCart(repository: _CartRepositoryStub());
  final result = await usecase(GetCartParams(userId: 'u1'));
  // Breakpoint aquí → ver el Either. Escribe el mensaje EXACTO del escenario.
  result.fold((l) => print('Failure: ${l.message}'), (r) => print('OK: ${r.length}'));
}
```

Los mensajes de failure deben coincidir **exactamente** con los escenarios del spec (RN001-RN006 del [tasks.md](#)). El debugger te deja verlos antes de commitear.

6.4 DataSource — REAL contra Supabase (Modo A)

```
// tool/scratch/main_datasource.dart
import 'package:flutter/material.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Supabase.initialize(
    url: const String.fromEnvironment('SUPABASE_URL'),
    anonKey: const String.fromEnvironment('SUPABASE_ANON_KEY'),
  );
  final ds = CartRemoteDataSource(client: Supabase.instance.client);
  try {
    final items = await ds.getCart(userId: 'test-123');
    // Breakpoint aquí → ves la respuesta REAL devuelta por la API
    print('Items: $items');
  } on AuthException catch (e) {
    print('Auth real: ${e.message}');
  } on SocketException catch (e) {
    print('Red real: $e'); // sin conexión
  }
}
```

No necesitas fake aquí: quieres ver la respuesta real. Abre **DevTools → Network View** mientras corres este scratchpad para ver el request HTTP real, su status y su payload (ver [13-network-view.md](#)).

6.5 RepositoryImpl — mapeo excepción → Failure

```
// tool/scratch/main_repository.dart
// Si el datasource ya está listo → usa el REAL (Modo A).
// Si no → stub del datasource que lanza una excepción, y verificas que
// el repository la convierte en el Failure correcto.
```

6.6 Cubit — transiciones de estado

```
// tool/scratch/main_cubit.dart
void main() {
  final cubit = CartCubit(usecases: [GetCart(repository: stubRepo)]);
  // Registra un observer para ver cada estado emitido
  final sub = cubit.stream.listen((state) => print('→ $state'));
  cubit.loadCart(); // Breakpoint en el cubit → pisa la transición
  // Verifica: Initial → Loading → Loaded (o Error)
}
```

Con `flutter run -t` + DevTools puedes conectar el **Bloc Inspector** y ver las transiciones visualmente (ver 16-BLOC-CUBIT).

6.7 Page/Widget — render por estado

```
// tool/scratch/main_page.dart
void main() {
  runApp(MaterialApp(
    home: BlocProvider(
      create: (_) => CartCubit(usecases: [stubUsecase]),
      child: const CartPage(),
    ),
  ));
}
```

Usa **Flutter Inspector → Select Widget Mode** para ver cómo renderiza cada estado (Loading/Loaded/Error). Puedes inyectar el cubit con un estado `Error` fijo para ver ese render sin esperar un fallo real.

7. Secuencia de build order SDD: verificar capa a capa

Este es el flujo completo conforme implementas el andamiaje. El bucle por capa es:

1. Escribes el cuerpo de la capa (reemplazas `throw UnimplementedError`)
2. Creas el scratchpad para esa capa (o reutilizas uno)
3. Lo corres con el debugger (F5) y pisas los breakpoints
4. Verificas que hace lo que dice el criterio "Éxito" del `tasks.md`
5. ¿OK? → pasas a la siguiente capa. ¿Mal? → corriges e iteras

Orden sugerido (dependencia de abajo hacia arriba)

Paso	Capa	Verificas	Modo
1	Entity	cálculos puros, invariantes	—

Paso	Capa	Verificas	Modo
2	Model	roundtrip JSON ↔ entity	—
3	DataSource	respuesta REAL de la API	A real
4	State	igualdad de estados	—
5	RepositoryImpl	mapeo excepción → Failure	A (ds real)
6	UseCase	Either + mensajes exactos	B stub (repo ya real)
7	Cubit	transiciones de estado	B stub (useCases reales)
8	Page/Widget	render por estado	B + Widget Inspector
9	Wiring (DI + ruta)	app completa ya funciona	—

El orden real lo decide la estructura del `tasks.md` de tu feature, pero el principio es el mismo: **verifica cada capa en cuanto la terminas, antes de construir la siguiente**. Así los bugs se aíslan a la capa recién escrita — nunca arrastras un error de la capa 2 mientras depuras la 6.

8. Checklist: "¿esta capa está verificada?"

Antes de pasar a la siguiente capa, confirma:

- ☐ Corrí la capa aislada con el debugger (F5) y pisé los breakpoints clave
- ☐ Vi el resultado real esperado (respuesta API, cálculo, estado, render)
- ☐ Probé al menos el camino feliz Y el camino de error/borde
- ☐ Si el spec define mensajes/valores exactos, los verifiqué literalmente
- ☐ No necesité la app completa ni la IA para saber que funciona
- ☐ (Si la capa fue difícil de "ver") escribí un test con flutter-test-generator

Resumen

Concepto	Valor
Problema	Feature andamiada no está cableada → no se ve desde la app
Solución	Scratchpad (<code>tool/scratch/main_*.dart</code>) por capa, corrido con el debugger
Modo A (real)	datos: datasource/repository contra Supabase real, ver respuesta real
Modo B (stub)	superiores: usecase/cubit con stub manual mientras la dependencia no está
<code>dart run</code>	capas puras sin Flutter (entity, model, state, usecase)
<code>flutter run -t</code>	capas que tocan Flutter (datasource, repository, cubit, page)

Concepto	Valor
Puente a tests	si una capa cuesta verla bien, ese es el momento de escribir su test (05)

Para lanzar cada scratchpad configura entradas en el `launch.json` — ver 02-configuracion-launch-json.md. Para las capas de datos, combínalo con Network View. Para estados, con el guard de 07-practicas-vscode. Este capítulo es la pieza que une "escribo una capa" con "la verifico aislada, sin depender de la IA".



Módulos relacionados

- 05-TESTING — Verificación automatizada (complemento: tests por capa)
- 02-SPEC-DRIVEN-DEVELOPMENT — Origen del andamiaje y criterios "Éxito" por capa
- 01-CLEAN-ARCHITECTURE — Las capas que recorres aquí
- 16-BLOC-CUBIT — Debugging de estados (BlocObserver)

 **Volver al índice:** [README.md](#)

30 — Referencia Rápida: Todo el Debugging en una Página

Cheatsheet ultra-densa de TODO el módulo. Imprime esto y tenlo al lado de tu monitor.

1. Atajos de VS Code

Atajo	Acción
F5	Start / Continue debugging
Ctrl+F5	Run without debugging
Shift+F5	Stop debugging
Ctrl+Shift+F5	Restart (Hot Restart en Flutter)
F9	Toggle breakpoint
F10	Step over
F11	Step into
Shift+F11	Step out
Ctrl+Shift+F9	Inline breakpoint
Ctrl+Shift+D	Run and Debug panel
Ctrl+Shift+Y	Debug Console

2. Panel de Debug

VARIABLES

WATCH

CALL STACK

BREAKPOINTS

| Locales, campos, closures

| Expresiones que monitoreas

| Pila de llamadas (frames)

| Lista de todos los breakpoints

DEBUG CONSOLE

(Ctrl+Shift+Y)

| REPL para evaluar expresiones

3. Breakpoints

Tipo	Icono	Cómo	Cuándo
Línea		Click gutter / F9	Pausar en una línea
Expression		Click derecho → Conditional	Cuando se cumple condición
Hit Count		Conditional → Hit Count	Cada N veces
Logpoint		Click derecho → Add Logpoint	Log sin pausar
Inline	 small	Ctrl+Shift+F9	Statement específico

4. launch.json — Templates

Debug Flutter

```
{ "name": "Flutter (Debug)", "type": "dart", "request": "launch", "program": "lib/main.dart" }
```

Profile

```
{ "name": "Flutter (Profile)", "type": "dart", "request": "launch", "program": "lib/main.dart", "flutterMode": "profile" }
```

Con flavors

```
{ "name": "Dev", "type": "dart", "request": "launch", "program": "lib/main.dart", "toolArgs": [ "--flavor", "development", "-t", "lib/main_development.dart" ] }
```

Con dart-define

```
{ "name": "Dev+Env", "type": "dart", "request": "launch", "program": "lib/main.dart", "dartDefine": [ "ENV=dev", "SUPABASE_URL=http://localhost:54321" ] }
```

Attach

```
{ "name": "Attach", "type": "dart", "request": "attach", "program": "lib/main.dart" }
```

Test

```
{"name":"Test","type":"dart","request":"launch","program":"test/widget_test.dart","args":["--name","test name"]}
```

Compound (App + Worker)

```
"compounds":[{"name":"App+Worker","configurations":["Flutter","Worker"]}]
```

5. Debug Flags de Flutter

Desde `main()`

```
import 'package:flutter/rendering.dart';

void main() {
  // Bounds y constraints
  debugPaintSizeEnabled = true;

  // Bordes de capas
  debugPaintLayerBordersEnabled = true;

  // Área de touch
  debugPaintPointersEnabled = true;

  // Baselines de texto
  debugPaintBaselinesEnabled = true;

  // Log cada rebuild
  debugPrintRebuildDirtyWidgets = true;

  // Log cada layout
  debugPrintLayouts = true;

  // Log cada repaint
  debugProfilePaintsEnabled = true;

  runApp(MyApp());
}
```

Debugging programático

```
// Árbol de widgets
debugDumpWidgetTree();

// Árbol de render
debugDumpRenderTree();

// Capas GPU
debugDumpLayerTree();

// Accesibilidad
debugDumpSemanticsTree();
```

Manejo de errores

```
FlutterError.onError = (details) {
  FlutterError.presentError(details);
};

ErrorWidget.builder = (details) {
  return const Material(child: Center(child: Text('Error')));
};
```

Performance Overlay

```
Stack(children: [MyApp(), PerformanceOverlay.all()]);
```

Assertions (solo debug mode)

```
assert(myValue != null, 'myValue no puede ser null');
```

6. Debug Console — Expresiones útiles

```
// Tipo
variable.runtimeType

// Contenido
myList → [1, 2, 3]
myMap → {"key": "value"}

// Operaciones
counter + 1
myList.length
myList.isEmpty
user?.name ?? "Anonymous"

// Condicionales
state is LoginLoading ? "cargando" : "listo"
```

7. Hot Reload vs Hot Restart

	Hot Reload (r)	Hot Restart (Ctrl+Shift+F5)
Estado	Se mantiene	Se pierde todo
Breakpoints	Se mantienen	Se limpian
Uso	Cambios visuales	Cambios en main/initState
Velocidad	~segundos	~5-10 segundos

NO se recarga con Hot Reload:

- `initState()`
- `main()`
- `const` values
- Imports
- Static fields

8. Workflow de 6 pasos

1. ENTENDER

→ ¿Qué se esperaba vs qué pasó?

2. REPRODUCIR

→ Steps exactos, siempre falla

3. AISLAR

→ Binary search con breakpoints

4. DIAGNOSTICAR

→ Variables, watch, call stack

5. CORREGIR

→ Fix mínimo, un cambio a la vez

6. VERIFICAR

→ Reproducir + tests + regresión

9. Matriz: tipo de bug → herramienta

Bug	Primera acción	Herramienta
UI se ve mal	Select Widget	Flutter Inspector
Estado no cambia	Breakpoint en emit()	Debugger
API falla	Network View	DevTools
Build error	Leer error completo	Terminal
Performance	Performance Overlay	DevTools
Memory leak	Heap Snapshot	DevTools Memory
Hot reload no funciona	Hot Restart	Ctrl+Shift+F5

11. Ejecución aislada de una capa (modo andamiaje SDD)

Verifica cada capa de Clean Architecture **por separado** con el debugger, sin la app completa ni la IA.

11.1 Scratchpad

Crea `tool/scratch/main_<capa>.dart` y lánzalo con el debugger (F5). Entradas en `launch.json` (ver 02).

11.2 Matrix por capa

Capa	Runtime	Modo	Qué verifico
Entity	<code>dart run</code>	—	cálculos, invariantes, Equatable/copyWith
Model	<code>dart run</code>	—	fromJson/tojson, roundtrip
State	<code>dart run</code>	—	igualdad/props
UseCase	<code>dart run</code>	B stub	Either<Failure,T>, mensajes exactos

Capa	Runtime	Modo	Qué verifico
DataSource	<code>flutter run -t</code>	A real	respuesta JSON real, excepciones reales
Repository	<code>flutter run -t</code>	A o B	excepción→Failure
Cubit	<code>flutter run -t</code>	A o B	transiciones de estado
Page	<code>flutter run</code>	A o B	render por estado

Modo A (real): inyecta dependencias reales (datasource→Supabase). Ver respuesta real + Network View.

Modo B (stub): inyecta stub manual cuando la dependencia inferior aún no está. Verifica la lógica de la capa.

Detalle completo en [31-debugging-por-capa.md](#)

12. Checklist universal

- ☐ Puedo reproducir el bug
- ☐ Sé los steps exactos
- ☐ Sé en qué archivo/función ocurre
- ☐ Inspeccioné las variables
- ☐ Leí el error completo
- ☐ Revisé git log
- ☐ Busqué en Google/Stack Overflow
- ☐ Revisé la documentación oficial
- ☐ Puedo explicar el problema
- ☐ Entiendo QUÉ falla (no solo QUÉ hacer)

Módulos relacionados

Tema	Archivo
Mentalidad	27-mentalidad-debugging.md
Fundamentos VS Code	01-fundamentos-debugging.md
launch.json	02-configuracion-launch-json.md
Breakpoints	03-breakpoints-avanzados.md
Inspección datos	04-inspeccion-datos-consola.md
Multi-target	05-multi-target-remoto.md
Cheatsheet VS Code	06-cheatsheet-vscode.md
Prácticas VS Code	07-practicas-vscode.md
Debug programático	28-debugging-programatico-flutter.md

Tema	Archivo
Playbook sistemático	29-playbook-debugging-sistematico.md
Workflow por tipo	26-workflow-debugging-por-tipo.md
Debugging por capa	31-debugging-por-capa.md

 **Volver al índice:** README.md